C++ Roadmap

Introduction to C++

C++ is a general-purpose, high-performance programming language. It was developed by Bjarne Stroustrup at Bell Labs starting in 1979.

C++ is widely used in various applications like game development, system programming, embedded systems, and high-performance computing.

C++ is a statically-typed language, meaning that the type of a variable is determined during compilation, and has an extensive library called the C++ Standard Library, which provides a rich set of functions, algorithms, and data structures for various tasks.

C++ is a popular and widely used programming language for various reasons.

- C++ is designed to provide high performance and efficiency. It offers fine-grained control over system resources, making it easier to optimize your software.
- C++ is supported on different computer architectures and operating systems, allowing you to write portable code that runs on various platforms without making major modifications.
- C++ supports object-oriented programming (OOP)
- C++ allows you to write both low-level code, like memory manipulation, as well as high-level abstractions, like creating classes and using the Standard Template Library (STL).
- C++ offers a vast range of libraries and tools, such as the Standard Template Library (STL), Boost, and Qt, among others, that can aid in the development of your projects and make it more efficient.
- **Compiled language:** C++ is a compiler language where the complete code is converted into machine language, which makes it a faster programming language
- Standard template library: C++ provides many data structures and algorithms-related library collections, such as template libraries for contains, iterators, algorithms, etc.
- The ANSI standard is an attempt to ensure that C++ is portable; that code you write for Microsoft's compiler will compile without errors, using a compiler on a Mac, UNIX, a Windows box, or an Alpha

Applications of C++ Programming:

- Application Software Development - C++ programming has been used in developing almost all the major Operating Systems like Windows, Mac OSX and Linux. Apart from the operating systems, the core part of many browsers, like Mozilla Firefox and Chrome have been written using C++. C++ also has been used in developing the most popular database system called MySQL.
- Programming Languages Development - C++ has been used extensively in developing new programming languages like C#, Java, JavaScript, Perl, UNIX's C Shell, PHP, Python, and Verilog, etc.
- Games Development - C++ is extremely fast, which allows programmers to do procedural programming for CPU-intensive functions and provides greater control over hardware, because of which it has been widely used in the development of gaming engines.
- Embedded System - C++ is being heavily used in developing medical and engineering applications like software for MRI machines, high-end CAD/CAM systems, etc.

Executable's Platform Dependency

The key reason C++ is considered platform-dependent is that the compiler translates the source code directly into **native machine code** for a specific target platform (a combination of operating system and CPU architecture).

- An executable file (`.exe` on Windows, `.out` on Linux/macOS, etc.) generated on one platform **cannot** run on another platform's operating system or different CPU architecture.
- For example, an `.exe` file compiled on Windows will not work on a Linux machine. To run the program on Linux, you must recompile the source code using a Linux-specific compiler.

Key disadvantages of C++ include:

- **Complexity:** C++ is a large, multi-paradigm language with an extensive feature set, making it difficult to learn and master, especially for beginners.
- **Manual Memory Management:** Unlike many modern languages (e.g., Java, Python) that have automatic garbage collection, C++ requires the programmer to manually allocate and deallocate memory. If not handled carefully, this can lead to memory leaks, corruption, and program crashes.
- **Security Vulnerabilities:** C++ allows low-level memory access and the use of pointers, which increases the risk of writing unsafe code susceptible to bugs like buffer overflows and null pointer dereferences, leading to security issues.
- **Long Compile Times:** Large C++ projects can have relatively long compilation times, which can slow down the development and testing cycles. C++ is platform-dependent.

Steps involved in Compilation:

1. Source Code (.cpp, .h files)

Preprocessing - The preprocessor handles lines that start with # (like #include, #define, etc.) and prepares the code for the compiler. It:

- Replaces #include <....> with actual code from the included files.
- Expands macros
- Handles conditional compilation (#ifdef, etc.)

```
g++ -E main.cpp -o main.i
```

The output (main.i) contains pure C++ code with all header files inserted and macros expanded.

Compilation - The compiler translates preprocessed C++ code into assembly language, which is human readable low-level code, It :

- Does Syntax checking
- Converts C++ to Assembly code (.s file)

```
g++ -S main.i -o main.s
```

main.s contains low-level assembly instructions.

```
int result = add(10, 5);
assembly - movl $10, -4(%rbp)
           movl $5, -8(%rbp)
           call add
```

Assembly - The assembler converts the assembly code into object code, which is machine-readable binary (.o file).

g++ -c main.cpp -o main.o

.o (object file) that contains machine code, not yet executable on its own

Linking - The linker combines multiple object files (main.o, math_utils.o) and links in external libraries (like iostream), creating a final executable. It:

- Connects the main() function in main.o to add() in math_utils.o
- Resolves symbols from the standard library

C vs C++

c	c++
C is a procedural programming language	C++ is both procedural and object-oriented.
Focuses on functions and structured programming.Does not support objects or classes.	Supports both functions and classes.
Memory management is manual, using functions like **malloc** and **free**.	Memory management can be manual (like C) or rely on constructors/destructors and smart pointers.
Code reusability is achieved through functions and modular programming.	Offers better code reusability with classes, inheritance, and polymorphism.
Function libraries can be created and included through headers.	Code modularity is enhanced through namespaces and well-designed object-oriented hierarchy.
Error handling in C is done primarily through return codes.Lacks support for exceptions or any built-in error handling mechanism.	Offers exception handling, which can be used to handle errors that may occur during program execution.Enables catching and handling exceptions with **try**, **catch**, and **throw** keywords, providing more control over error handling.

Setting Up C++

Setting up C++ requires a few steps, including installing a compiler, configuring an Integrated Development Environment (IDE), and creating a new C++ project.

- GCC (GNU Compiler Collection) for Linux and macOS, but can also be used on Windows through MinGW
- MSVC (Microsoft Visual C++) for Windows.

An IDE is a software application that provides facilities for programming, such as code editing, debugging, and building. Some popular C++ IDEs include:

- **@article@Visual Studio** (Windows, macOS)
- **@article@Eclipse** (Windows, macOS, Linux)
- **@article@Code::Blocks** (Windows, macOS, Linux)

There are many C++ compilers available:

- Apple C++. Xcode
- Bloodshed Dev-C++
- Clang C++
- Cygwin (GNU C++)
- Mentor Graphics
- MINGW - "Minimalist GNU for Windows"
- GNU CC source
- IBM C++
- Intel C++
- Microsoft Visual C++
- Oracle C++
- HP C++

Tokens :

Constants/Literals - Constants refer to fixed values that the program may not alter and they are called literals. Define Using #define pre-processor or Using const keyword.

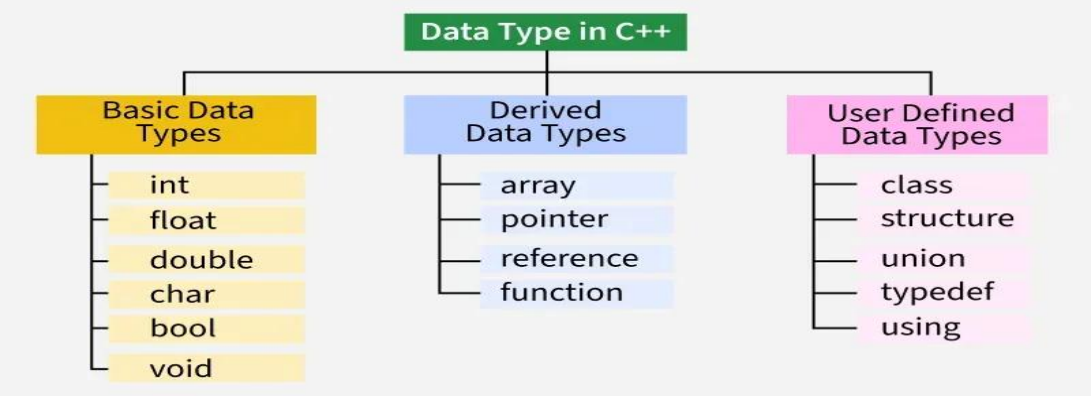
Keywords - are reserved words that have special meanings to the compiler.

Identifiers - C++ identifiers are unique names assigned to identify variables, functions, classes, arrays, and other user-defined items in a program.

DATA Types

Primitive Built-in Types

Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.



char	1byte	-127 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-127 to 127
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	2bytes	0 to 65,535
signed short int	2bytes	-32768 to 32767
long int	8bytes	-9223372036854775808 to 9223372036854775807
signed long int	8bytes	same as long int
unsigned long int	8bytes	0 to 18446744073709551615
long long int	8bytes	-(2^63) to (2^63)-1
unsigned long long int	8bytes	0 to 18,446,744,073,709,551,615
float	4bytes	
double	8bytes	
long double	12bytes	
wchar_t	2 or 4 bytes	1 wide character

References vs Pointers:

You cannot have NULL references. You must always be able to assume that a reference is connected to a legitimate piece of storage.

Once a reference is initialized to an object, it cannot be changed to refer to another object. Pointers can be pointed to another object at any time.

A reference must be initialized when it is created. Pointers can be initialized at any time.

```
int i = 17;

int& r = i;

cout << "Value of i : " << i << endl;

cout << "Value of i reference : " << r << endl;
```

Variable Scope in C++

Local Variables and Global Variables

You can access a global variable when there is a local variable with the same name by using the **SRO** (Scope Resolution Operator) **::** before the name of that variable

cout << "Value of g (Global variable): " << ::var_name;

Manipulators in C++ are **special functions or objects** that are used with insertion **(<<)** and extraction **(>>)** operators **(cin/cout)** to control the input and output streams.

endl	It inserts a newline and flushes the output buffer.	cout << "Tutorials" << endl;
setw(n)	It sets the field width for the next output operation.	cout << setw(5) << 57;
setfill()	It sets the fill character.	cout << setfill('*') << setw(5) << 42;

C++ Modifier Types

C++ allows the char, int, and double data types to have modifiers preceding them. The modifiers are signed, unsigned, long, and short.

storage classes : A storage class defines the scope (visibility) and life-time of variables and/or functions within a C++ Program.

- The **auto** storage class is the default storage class for all local variables.
- The **register** storage class is used to define local variables that should be stored in a register instead of RAM. This means that the variable has a maximum size equal to the register size (usually one word). You cannot apply the address operator (&) to a register variable because it might not have a memory location in RAM. The register should only be used for variables that require quick access such as counters. It should also be noted that defining 'register' does not mean that the variable will be stored in a register. It means that it MIGHT be stored in a register depending on hardware and implementation restrictions.{ register int miles; }
- The **static** storage class instructs the compiler to keep a local variable in existence during the life-time of the program instead of creating and destroying it each time it comes into and goes out of scope. Therefore, making local variables static allows them to maintain their values between function calls. Automatically initialized to zero (0) if no explicit value is provided. The static modifier may also be applied to global variables. When this is done, it causes that variable's scope to be restricted to the file in which it is declared. Other files cannot access it using the extern keyword. In C++, when static is used on a class data member, it causes only one copy of that member to be shared by all objects of its class.

  file1.c

  static int count = 5;

  file2.c

  extern int count; // ✕ ERROR

- The **extern** storage class is used to give a reference of a global variable that is visible to ALL the program files. When you use 'extern' the variable cannot be initialized as all it does is point the variable name at a storage location that has been previously defined.When you have multiple files and you define a global variable or function, which will be used in other files also, then extern will be used in another file to give reference of defined variable or function. Just for understanding extern is used to declare a global variable or function in another file.The extern modifier is most commonly used when there are two or more files sharing the same global variables or functions. 0 (zero) if the definition doesn't provide an initializer

The **constexpr** specifier was introduced in C++11. It calculates the value of a variable or the return value of a function at **compile time.** Calculating the values during compilation time improves the performance of the code. It helps in faster execution of programs; since the value is already known during compilation, the runtime overhead is reduced.

```
#include <iostream>

 using namespace std;

constexpr int sum(int a, int b) { return a + b; }

int main() {

constexpr int x = sum(10, 2);

cout << "10 + 2 = " << x << endl;

return 0; }
```

`constexpr` variable cannot be changed.

`constexpr` is stronger than `const`.

```
const int b = a; // allowed
constexpr int c = a; // ✕ error
```

Limitations of Constexpr Specifier

The **input and output operations** (cin/cout) cannot be used in the functions declared with constexpr specifier.

It does not support dynamic memory allocation.

One constexpr function can only call another constexpr function. We can not call any other normal functions.

There is no support for exception handling.

Function parameters or return type can only be of **literal type**.

Basic Operations in C++

Arithmetic Operations

Addition (+): Adds two numbers.

```
int a = 5; int b = 6; int sum = a + b; // sum is 11
```

- **Subtraction (-)**: Subtracts one number from the other.

```
int a = 10; int b = 6; int diff = a - b; // diff is 4
```

- **Multiplication (*)**: Multiplies two numbers.

int a = 3; int b = 4; int product = a * b; // product is 12

- **Division (/)**: Divides one number by another, yields quotient.

int a = 12; int b = 4; int quotient = a / b; // quotient is 3

- **Modulus (%)**: Divides one number by another, yields remainder.

int a = 15; int b = 4; int remainder = a % b; // remainder is 3

Increment Operator (++)

```
int x = 5;
int y = ++x; // x = 6, y = 6

int z = x++; // x = 7, z = 6
```

Decrement Operator (++)

```
int x = 5;
int y = --x; // x = 4, y = 4

int z = x--; // x = 3, z = 4
```

Relational Operators

- **Equal to (==)**: Returns true if both operands are equal.

5 == 5 // true 3 == 4 // false

- **Not equal to (!=)**: Returns true if operands are not equal.

5 != 2 // true 1 != 1 // false

- **Greater than (>)**: Returns true if the first operand is greater than the second.

5 > 3 // true 2 > 3 // false

- **Less than (<)**: Returns true if the first operand is less than the second.

3 < 5 // true 6 < 5 // false

- **Greater than or equal to (>=)**: Returns true if the first operand is greater than or equal to the second.

5 >= 5 // true 6 >= 2 // true 3 >= 4 // false

- **Less than or equal to (<=)**: Returns true if the first operand is less than or equal to the second.
- 4 <= 4 // true 2 <= 3 // true 5 <= 4 // false

Logical Operators

Logical operators are used for combining multiple conditions or boolean values.

- **AND (&&)**: Returns true if both operands are true.

true && true // true true && false // false

- **OR (||)**: Returns true if any one of the operands is true.

true || false // true false || false // false

- **NOT (!)**: Returns true if the operand is false and vice versa.

!true // false !false // true

Bitwise Operations

Bitwise operations are operations that directly manipulate the bits of a number. Bitwise operations are useful for various purposes, such as optimizing algorithms, performing certain calculations, and manipulating memory in lower-level programming languages like C and C++.

Bitwise AND (&)

The bitwise AND operation (**&**) is a binary operation that takes two numbers, compares them bit by bit, and returns a new number where each bit is set (1) if the corresponding bits in both input numbers are set (1); otherwise, the bit is unset (0).
Example:

int result = 5 & 3; // result will be 1 (0000 0101 & 0000 0011 = 0000 0001)

Bitwise OR (|)

The bitwise OR operation (|) is a binary operation that takes two numbers, compares them bit by bit, and returns a new number where each bit is set (1) if at least one of the corresponding bits in either input number is set (1); otherwise, the bit is unset (0).
Example:

int result = 5 | 3; // result will be 7 (0000 0101 | 0000 0011 = 0000 0111)

Bitwise XOR (^)

The bitwise XOR (exclusive OR) operation (^) is a binary operation that takes two numbers, compares them bit by bit, and returns a new number where each bit is set (1) if the corresponding bits in the input numbers are different; otherwise, the bit is unset (0).
Example:

int result = 5 ^ 3; // result will be 6 (0000 0101 ^ 0000 0011 = 0000 0110)

Bitwise NOT (~)

The bitwise NOT operation (~) is a unary operation that takes a single number, and returns a new number where each bit is inverted (1 becomes 0, and 0 becomes 1).
Example:

int result = ~5; // result will be -6 (1111 1010)

Most Significant Bit (MSB)

- 0 → positive number
- 1 → negative number

Example (8-bit):

01111111 = 127
10000000 = -128

~x = -(x + 1)

Bitwise Left Shift (<<)

The bitwise left shift operation (**<<**) is a binary operation that takes two numbers, a value and a shift amount, and returns a new number by shifting the bits of the value to the left by the specified shift amount. The vacated bits are filled with zeros.
Example:

int result = 5 << 1; // result will be 10 (0000 0101 << 1 = 0000 1010)

x << 1 // multiply by 2

x << 2 // multiply by 4

11111111 << 1 => 11111110

this happened in case of 8 bit (255) << 1 => (254)

but when we cosidered as 32 bit then it become (510)

00000000 00000000 00000000 11111111 << 1 = > 00000000 00000000 00000001 11111110

Bitwise Right Shift (>>)

The bitwise right shift operation (**>>**) is a binary operation that takes two numbers, a value and a shift amount, and returns a new number by shifting the bits of the value to the right by the specified shift amount. The vacated bits are filled with zeros or sign bit depending on the input value being signed or unsigned.
Example:

int result = 5 >> 1; // result will be 2 (0000 0101 >> 1 = 0000 0010)

Right shift by 1 = divide by 2

Assignment Operators

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand.	C = A + B will assign value of A + B into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2

&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
|=	Bitwise inclusive OR and assignment operator.	C |= 2 is same as C = C | 2

The **sizeof** is a keyword, but it is a compile-time operator that determines the size, in bytes, of a variable or data type. sizeof (data type). The sizeof operator can be used to get the size of classes, structures, unions and any other user defined data type.

C++ Conditional ? : Operator

if(y < 10) { var = 30; } else { var = 40; }

Is equal to => var = (y < 10) ? 30 : 40;

C++ Member (dot & arrow) Operators

The . (dot) operator and the -> (arrow) operator are used to reference individual members of classes, structures, and unions. Simply saying: To access members of a structure, use the dot operator. To access members of a structure through a pointer, use the arrow operator.

Example:

```
struct Employee {
 char first_name[20];
};
 int main() {
Employee emp;
strcpy(emp.first_name, "zara");
Employee* p_emp = &emp; // Using arrow operator to assign a value
strcpy(p_emp->first_name, " "zara");
 return 0; }
```

Casting Operators in C++:

A cast is a special operator that forces one data type to be converted into another. As an operator, a cast is unary and has the same precedence as any other unary operator.

Type Casting is the process of **converting a value from one data type to another data type**, either **automatically by the compiler** or **manually by the programmer.**

Implicit Type Casting (Automatic):

Implicit type casting is a conversion performed **automatically by the compiler** when a smaller data type is converted into a larger compatible data type.

```
int a = 10;
double b = a; // implicit
```

Explicit Type Casting (Manual):

Explicit type casting is a conversion in which the programmer **forcibly converts one data type into another** using a cast operator.

```
int a = 5, b = 2;
float result = (float)a / b;
```

C++ supports 4 special casting operators:

static_cast:

Used for compile-time type conversion.

It performs **checked conversions** that are known at compile time and does **not perform runtime checks**.

Performs conversions between related types that the compiler considers safe, such as int to double, enum to int, or upcasting a derived class pointer to a base class pointer.

Example:

```
int a = 10;
float b = static_cast<float>(a);
```

eg.2

```
class Base {};
class Derived : public Base {};
Base* b = new Derived();
Derived* d = static_cast<Derived*>(b);//its ok
Base* b = new Base();
Derived* d = static_cast<Derived*>(b); // ✕ Undefined behaviour
```

dynamic_cast:

Runtime polymorphism . Base ↔ Derived class

Used for conversions involving polymorphic classes (those with at least one virtual function) at runtime. It checks the validity of the conversion and returns a nullptr (for pointers) or throws a std::bad_cast exception (for references) if the cast fails.

It checks the **actual object type at runtime** using RTTI (Run-Time Type Information). Used mainly for **downcasting**. **downcasting** is the process of converting a pointer or reference from a **base class** to a pointer or reference of a **derived class**.

- Base class must have at least one virtual function
- Works only with pointers or references

```
Base* b = new Derived();
Derived* d = dynamic_cast<Derived*>(b);
```

✓ Runtime safety
✓ Slower than static_cast
✓ Returns nullptr if cast fails
✓ Best for polymorphism

✕ What NOT to Do

```
class Base {};   // ✕ No virtual function
Derived* d = dynamic_cast<Derived*>(b); // Compile-time error
```

```
Base* b = new Base();
Derived* d = dynamic_cast<Derived*>(b);
if (d == nullptr)
    cout << "Invalid cast";
```

const_cast:

const_cast is used to **add or remove** const **or** volatile **qualifiers** from a variable.

Used to add or remove the const or volatile attributes from a pointer or reference. Modifying a *truly* constant object via const_cast results in undefined behavior.

```
void modify(int* p) {
    *p = 20;
```

```
}
int a = 10;
const int* ptr = &a;
modify(const_cast<int*>(ptr));
```

Not to do:

```
const int a = 10;
int* p = const_cast<int*>(&a);
```

*p = 20; // ✕ Undefined behaviour Modifying a **truly const object** is illegal.

const_cast is considered safer than simple type casting. It'safer in the sense that the casting won't happen if the type of cast is not same as original object.

```
  int a1 = 40;
   const int* b1 = &a1;
   char* c1 = const_cast <char *> (b1); // compiler error
```

reinterpret_cast:

reinterpret_cast performs **low-level, unsafe type conversions** by **reinterpreting the bit pattern** of the data.

he reinterpret_cast operator changes a pointer to any other type of pointer. It also allows casting from pointer to an integer type and vice versa.

```
int a = 65;
char* c = reinterpret_cast<char*>(&a);
int x = 1000;
int* p = reinterpret_cast<int*>(x); // ✕ ⊘ Invalid memory access possible. Integer to ponter.
```

A unary operator is operators that act upon a single operand to produce a new value, It manipulates or modifies only the single operand or value. It is used to perform operations like incrementing, negating, or changing the sign of a value.

C++ Operators Precedence

The operator precedence determines the order in which operations are performed in an expression.

Operators with higher precedence are evaluated first.The operators are listed from top to bottom in descending order of precedence:

Precedence	Operators	Description	Associativity
1	() [] -> .	Grouping, member access	Left → Right
2	++ --	Unary increment/decrement	Right → Left
3	! ~ + - (type)	Unary operators	Right → Left
4	* / %	Multiplicative	Left → Right
5	+ -	Additive	Left → Right
6	<< >>	Shift	Left → Right
7	< <= > >=	Relational	Left → Right
8	== !=	Equality	Left → Right
9	&	Bitwise AND	Left → Right
10	^	Bitwise XOR	Left → Right
11	`	`	Bitwise OR
12	&&	Logical AND	Left → Right
13	`		`
14	?:	Ternary	Right → Left
15	= += -= *=	Assignment	Right → Left
16	,	Comma	Left → Right

Functions in C++

A function **is a group of statements that perform a specific task, organized as a separate unit in a program.**

Pre-defined functions available in the C++ standard library, such as **sort()**, **strlen()**, **sqrt()**, and many more.

User-defined functions: Functions created by the programmer to perform a specific task.

A function prototype is a declaration of the function without its body, and it informs the compiler about the function's name, return type, and parameters.

int multiplyNumbers(int x, int y);

A **function** is a group of statements that together perform a task.

Every C++ program has at least one function, which is **main()**

Sr.No	Call Type & Description
1	**Call by Value** This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.
2	**Call by Pointer**

	This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.
3	**Call by Reference** This method copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

Variable Number of Parameters

A function that can accept a variable number of parameters or arguments. This is typically implemented using variadic functions or template parameter packs.

```
// Variadic template function

template<typename... Args>

void printNumbers(Args... args) {

(cout << ... << args) << endl;

}

int main() {

printNumbers(1, 2, 3);

// Outputs: 123

printNumbers(4, 5, 6, 7, 8);

// Outputs: 45678

return 0;

}
```

Default Parameters

The default parameters are the parameters with default values that can be omitted when calling the function. The function uses the default values if no arguments are provided for those parameters.

Once a parameter has a default, **all parameters to its right must also have defaults. void f(int x = 3, int y, int k) // this is not legal**

Once default arguments are defined in the declaration, they cannot be modified in the function definition.

```
void f(int a = 10);

// This definintion will throw and error

void f(int a = 222){

// statements

}
```

If a function containing default arguments is overloaded, then we need to make sure it is not ambiguous to the compiler, otherwise it will throw an error.

```
void f(int a = 10){
cout << a << endl;
}
int main(){
f();
f(221);
return 0;
}
```

```
void greet(const string& name = "Guest", int age = 0) {
cout << "Hello, " << name;
if (age > 0) {
cout << ". You are " << age << " years old.";
}
cout << endl;
}
int main() {
greet(); // Outputs: Hello, Guest
greet("Alice"); // Outputs: Hello, Alice
greet("Bob", 30); // Outputs: Hello, Bob. You are 30 years old.
return 0;
}
```

Default Arguments must be specified in Function Declaration, not Definition.

```
void greet(string name = "TutorialsPoint");
// Function Definition without default arguments (Correct)
void greet(string name) {
cout << message << ", " << name << "!" << endl;
}
```

When assigning the values for default arguments, it must start from the **rightmost** parameters to the **leftmost** ones, where the user can't skip providing parameters in the middle.

Advantages of Default Arguments

It simplifies the function calls by reducing the number of overloads, where users don't need to create multiple overloaded functions when one function can handle different cases by using default values for parameters.

Default arguments help avoid code duplication and reduce redundancy.

It improves code readability and increases flexibility, which makes further easier to modify in the future.

Named Parameters (C++ Specific):

[Named parameters (or keyword arguments) in programming allow you to pass arguments to functions/methods using the parameter's name instead of its position
Positional arguments match parameters by order (e.g., func(1, 2) for func(a, b)), while named arguments use name: value (e.g., func(b: 2, a: 1)).]

Though C++ does not support named parameters directly, you can achieve similar functionality using a class or struct to encapsulate parameters.

```
struct Person {

string name;

int age;

double height;

};

// Function that takes a parameter object

void printPerson(const Person& p) {

cout << "Name: " << p.name << ", Age: " << p.age << ", Height: " << p.height << endl;

}
```

Lambda Functions in C++

A lambda function, or simply "lambda", is an anonymous (unnamed) function that is defined in place, within your source code, and with a concise syntax. Lambda functions were introduced in C++11.

```
[capture-list](parameters) -> return_type {
   // function body

};
```

- **capture-list**: A list of variables from the surrounding scope that the lambda function can access.
- **parameters**: The list of input parameters, just like in a regular function. Optional.
- **return_type**: The type of the value that the lambda function will return. This part is optional, and the compiler can deduce it in many cases.
- **function body**: The code that defines the operation of the lambda function.

+Lambda function with no capture, parameters, or return type.

```
auto printHello = []() {
   std::cout << "Hello, World!\n";

};
```

```
printHello(); // Output: Hello, World!
```

Lambda function with parameters.

```
auto add = [](int a, int b) {
    return a + b;
};
int result = add(3, 4); // result = 7
```

Lambda function with capture-by-value.

```
int multiplier = 3;
auto times = [multiplier](int a) {
    return a * multiplier;
};
int result = times(5); // result = 15
```

Lambda function with capture-by-reference.

```
int expiresInDays = 45;
auto updateDays = [&expiresInDays](int newDays) {
    expiresInDays = newDays;
};
updateDays(30); // expiresInDays = 30
```

Inline Function in C++

An **inline function** is a function for which the **compiler tries to replace the function call with the actual function code** at the place where it is called, instead of performing a normal function call.

☞ This **reduces function call overhead** and can improve performance.

Why do we need inline functions?

Normal function call involves:

- Pushing arguments on stack
- Jumping to function code
- Returning back

This takes **extra time**, especially for **small functions**.

Inline function avoids this by **copy-pasting function code** at call location.

How inline function works (concept)

Normal function call:

```
result = add(2, 3);
```

Inline expansion (conceptually):

result = 2 + 3;

Advantages of inline functions

✓ Faster execution for small functions
✓ Reduces function call overhead
✓ Improves performance in time-critical code

Disadvantages of inline functions

✗ Increases program size (code duplication)
✗ Not suitable for large functions
✗ Compiler may ignore inline request

Advantages of Inline Function

The following are the advantages of using inline function −

• In the case of an inline function, the function call overhead does not occur.

• The inline function saves the overhead of the push and pop variables on the stack, when the function is called.

• An inline function saves the overhead of a return call from a function.

• When an inline function is created, the compiler may perform context specific optimization on the body of the function. This kind of optimization is not performed for normal functions.

• Using small inline functions may be useful for embedded systems, because inline can yield less code than the function call preamble and return.

Disadvantages of Inline Function

Some of the disadvantages of inline functions are as follows –

• When we use an inline function, the size of the code is increased because the compiler replaces each function call with the inline function code.

• Large sized code takes more memory and time to compiler the code.

• The compilation process slows down because the compiler evaluates and replaces the function code at the place function calls.

• In some of the cases, the program's performance may slow down.

• Based on the function code complexity, the compiler may ignore the inline keyword. Thus, the application of the inline function is limited.

Static Polymorphism

Static polymorphism, also known as compile-time polymorphism, is a type of polymorphism that resolves the types and method calls at compile time rather than at runtime.

C++ Operator Overloading

returnType operator symbol (arguments) {

```
    ... .. ...
}
```

Operator overloading means giving a new meaning to an operator (like +, -, *, []) when it is used with **objects**.

With operator overloading, we can make operators work for user defined classes structures.

Operator overloading means **using the same operator for different purposes** depending on the data type.

Operator overloading is done in class and structure

Why use Operator Overloading?

- Allows objects to behave like basic data types.
- Useful for mathematical objects like Complex numbers and Vectors.
- Reduces the need for extra function calls.

```
Complex operator+(Complex c2)
  {
    Complex c3(0, 0);
    c3.real = this->real + c2.real;
    c3.imaginary = this->imaginary + c2.imaginary;
    return c3;
  }
```

Complex c3 = c1 + c2;

"c1 + c2" is translated internally to "c1.operator+(c2)"

Operator Functions vs Normal Functions

Feature	Operator Function	Normal Function
Syntax	Uses operator keyword	Standard function name
Invocation	Triggered by using an operator	Called explicitly by name
Purpose	Redefines behavior of operators	Performs defined actions

Feature	Operator Function	Normal Function
Example	operator+()	add()

The following operators cannot be overloaded in C++:

1. sizeof Operator

- Evaluated at compile time
- Used internally for memory layout and pointer arithmetic
- Overloading would break fundamental language behavior

2. typeid Operator

- Used for runtime type identification
- Must uniquely identify a type
- Altering its meaning would break RTTI and polymorphism guarantees

3. Scope resolution (::) Operator

- Works on names, not values
- Fully resolved at compile time
- There is no syntax to intercept or overload name resolution

4. Class Member Access Operators (. and .*)

- Implicitly used by the compiler to access members
- Overloading would break object access semantics

Operator overloading can be done in struct in c++ but it not support c language why?

C is a *procedural* language

No function dispatch on operators in C . operators are **hardcoded into the compiler**

Function Overloading

Function overloading is a way to create multiple functions with the same name but different parameter lists. The compiler determines the correct function to call based on the types and number of arguments used when the function is called.
Static polymorphism occurs at compile-time with the help of overloaded functions and templates

Example:

```
#include <iostream>
void print(int i) {

   std::cout << "Printing int: " << i << '\n';

}

void print(double d) {

   std::cout << "Printing double: " << d << '\n';

}
```

```
void print(const char* s) {
    std::cout << "Printing string: " << s << '\n';
}
int main() {
    print(5);          // Calls print(int i)
    print(3.14);       // Calls print(double d)
    print("Hello");    // Calls print(const char* s)
    return 0;
}
```

Function Overloading Based on Number of Parameters

```
void display(int a); // takes one parameter
void display(int a, double b); // takes two parameters
```

Function Overloading Based on Different Parameter Types

```
void show(int a); // parameter with int type
void show(double a); // parameter with double type
```

Function Overloading Based on Different Parameter Order

```
void display(int a, double b);
void display(double a, int b) ;
```

Functions that Cannot be Overloaded

1) Function declarations that differ only in the return type.

```
int foo() {
  return 10;
}
char foo() {
  return 'a';
}
```

2) Member function declarations with the same name and the name parameter-type-list cannot be overloaded if any of them is a static member function declaration.

```
class Test {
   static void fun(int i) {}
   void fun(int i) {}
};
```

3) Parameter declarations that differ only in a pointer * versus an array [] are equivalent.

```
int fun(int *ptr);

int fun(int ptr[]);
```

4) Parameter declarations that differ only in that one is a function type and the other is a pointer to the same function type are equivalent.

```
void h(int ());

  void h(int (*)());
```

5) Parameter declarations that differ only in the presence or absence of const and/or volatile are equivalent.

```
#include<iostream>

#include<stdio.h>

using namespace std;

 int f ( int x) {

    return x+10;

}

int f ( const int x) {

    return x+10;

}

int main() {

  getchar();

  return 0;

}
```

6) Default Arguments Causing Ambiguity ❌

```
void test(int a, int b = 10);
void test(int a);  // ERROR (ambiguous call)
```

❌ Function Overloading in C

C does **NOT** support function overloading.

C uses simple symbol names (no name mangling).

The compiler cannot distinguish functions with the same name.

_Generic is a C11 compile-time feature that selects expressions based on the type of an argument, allowing function-like overloading using macros.

_Generic((a), ...) - Look at the **type of** a

Example:

```
#include <stdio.h>

int add_int(int a, int b) {
```

```
    return a + b;
}
float add_float(float a, float b) {
    return a + b;
}
#define add(a,b) _Generic((a), \
    int: add_int, \
    float: add_float \
)(a,b)
int main() {
    printf("%d\n", add(2, 3));          // calls add_int
    printf("%f\n", add(2.5f, 3.5f));    // calls add_float
}
```

What is Function Overriding in C++?

Function overriding is a concept of object-oriented programming which allows a derived class to redefine a function that is already defined in the base class

Example:

```
class base {
public:
void notice( )
      cout << "This is my Base Class" ;
}
class derived: public base {
public:
void notice( )
     cout << "This is my Derived Class";
}
```

it resolved at runtime polymorphism(dynamic binding), it only take place in presence of inheritance

Function overriding is an essential concept in object-oriented programming, enabling polymorphism and dynamic binding.

Example:

```
#include <iostream>
using namespace std;
// Base class
class Shape {
```

```
public:
// Virtual method to be overridden
virtual void draw() const {
cout << "Drawing a shape" << endl;
}
};
// Derived class Circle
class Circle : public Shape {
public:
// Overriding the base class method
void draw() const override {
cout << "Drawing a circle" << endl;
}
};
// Derived class Square
class Square : public Shape {
public:
// Overriding the base class method
void draw() const override {
cout << "Drawing a square" << endl;
}
};
// Main function
int main() {
Shape* shapePtr;
Circle circle;
Square square;
// Point to Circle and call draw()
shapePtr = &circle;
shapePtr->draw(); // Outputs: Drawing a circle
// Point to Square and call draw()
shapePtr = &square;
shapePtr->draw(); // Outputs: Drawing a square
return 0;
```

}

1. Virtual Destructor

A virtual destructor makes sure that the destructor of the derived class will be executed when an object is removed through a base class pointer. Function overriding with virtual destructors avoids resource leaks and unpredictable behavior by ensuring proper deletion of objects through base class pointers.

```
#include <iostream>
using namespace std;

class BaseClass {

public:

virtual ~BaseClass() { // Virtual destructor

cout << "BaseClass destructor" << endl;

}

};

class DerivedClass : public BaseClass {

public:

~DerivedClass() override { // Overriding destructor

cout << "DerivedClass destructor" << endl;

}

};

int main() {

BaseClass* a = new DerivedClass();

// when delete a call it Calls DerivedClass's destructor followed

// by BaseClass's destructor

delete a;

return 0;

}
```

Output:

DerivedClass destructor

BaseClass destructor

Overriding and final Keyword alternate word

The `final` keyword stops any further subclassing of a class or overriding of a method.Function overriding with the `final` keyword is important because it guarantees that a class or method cannot be further changed or extended.

```
#include <iostream>

using namespace std;

class Subject {
```

```
public:
// Final method
virtual void examType() final {
cout << "This subject has a written exam." << endl;
}
};
class Math : public Subject {
// Cannot override examType() here
};
int main() {
Subject* s = new Math();
s->examType(); // Calls Subject's examType() method
delete s;
return 0;
}
```

Virtual Inheritance

Virtual inheritance in C++ tackles problems which arise with multiple inheritance, particularly the diamond problem

```
#include <iostream>
using namespace std;
class Base {
public:
void present() { cout << "Display from Base class" << endl; }
};
class A : virtual public Base { };
class B : virtual public Base { };
class Final : public A, public B {
public:
void get() { cout << "Display from Final class" << endl; }
};
int main() {
Final obj;
// Displays: Display from Base class
```

```
obj.present();
// Displays: Display from Final class
obj.get();
return 0;
}
```

typedef Declarations

You can create a new name for an existing type using **typedef**.

```
typedef type newname;
typedef int feet;
feet distance;
```

c++ Strings

The C-Style Character String

The C-style character string originated within the C language and continues to be supported within C++. This string is actually a one-dimensional array of characters which is terminated by a **null** character '\0'. Thus a null-terminated string contains the characters that comprise the string followed by a **null**.

The following declaration and initialization create a string consisting of the word "Hello". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello."

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

If you follow the rule of array initialization, then you can write the above statement as follows −

```
char greeting[] = "Hello";
```

```
strcpy(s1, s2);
```

Copies string s2 into string s1.

```
strcat(s1, s2);
```

Concatenates string s2 onto the end of string s1.

```
strlen(s1);
```

Returns the length of string s1.

```
strcmp(s1, s2);
```

Returns 0 if s1 and s2 are the same; less than 0 if s1<s2; greater than 0 if s1>s2.

```
strchr(s1, ch);
```

Returns a pointer to the first occurrence of character ch in string s1.

```
strstr(s1, s2);
```

Returns a pointer to the first occurrence of string s2 in string s1.

strcmp compares **lexicographically (dictionary order)** using **ASCII values**, character by character.

str1 = "hello"

str2 = "Tutorialspoint Learner"

here str1>str2 because ‘h’> ‘t’

The String Class in C++

C++ string is an array of characters that can be extracted and manipulated with the help of various member functions.

To create an object of a string, you must use the std::string class from the <string> header.

Strings are defined as character arrays which are accessed using the pointer to the first iterator of the array

Comparison operators give access to compare two **std::string** objects directly.
== (Equality) != (Inequality) < (Less than) > (Greater than) <= (Less than or equal to) >= (Greater than or equal to)

Array (in C / C++):

An array is a collection of elements of the same data type stored in contiguous (continuous) memory locations, and each element is accessed using an index (position number).

C++ Pointer to an Array

An array name is a constant pointer to the first element of the array.herefore, in the declaration −

balance is a pointer to &balance[0], which is the address of the first element of the array balance.

double balance[5] = {1000.0, 2.0, 3.4, 17.0, 50.0};

double *p;

p = balance;

And for second value => *(p + 1) or *(balance+1)

C++ Passing Arrays to Functions

C++ does not allow to pass an entire array as an argument to a function. If you want to pass a single-dimension array as an argument in a function, you would have to declare function formal parameter in one of following three ways and all three declaration methods produce similar results because each tells the compiler that an integer pointer is going to be received.

void myFunction(int *param)

void myFunction(int param[10])

```
void myFunction(int param[])
```

Return Array from Functions in C++

C++ does not allow to return an entire array as an argument to a function. However, you can return a pointer to an array by specifying the array's name without an index.

```
int * myFunction()
```

Second point to remember is that C++ does not advocate to return the address of a local variable to outside of the function so you would have to define the local variable as **static** variable.

Wrong example:

```
int* foo() {
   int x = 10;
   return &x;   // ✕ BAD
}
```

right example:

```
int* foo() {
static int r[10];
   return r;   // GOOD
}
```

Array decay means:

> When an array is used in most expressions (especially when passed to a function), it is automatically converted into a pointer to its first element.

Example:

a is now int* → pointer size (8 bytes) . The array has decayed into a pointer.

```
void print(int a[])
{
   cout << sizeof(a) << endl; //8
}
int main()
{
   int arr[5] = {1,2,3,4,5};
   cout << sizeof(arr) << endl; // size of full array = 20
   print(arr);
}
```

How to Pass a 2D Array to a Function:

When passing this to a function, **you MUST specify the second dimension.**

void func(int arr[][4], int rows)

void func(int (*arr)[4], int rows)

void func(int** arr) // ✕ wrong

Returning a 2D Array via Pointer (C-Style)

```
int (*createMatrix())[4]
{
   static int arr[3][4] =
   {
      {1, 2, 3, 4},
      {5, 6, 7, 8},
      {9, 10, 11, 12}
   };
   return arr;
}
int main()
{
   // Pointer to array of 4 ints
   int (*p)[4] = createMatrix();
 for(int i = 0; i < 3; i++)
   {
      for(int j = 0; j < 4; j++)
      {
         cout << p[i][j] << " ";
      }
      cout << endl;
   }
}
```

You cannot use the **"="** operator to **copy one array to another array** as the array decays into a pointer. The '=' operator **copies the address** but we **can not reassign the array memory** as it is already fixed. So, it will show an **error**.

int a[3] = {1, 2, 3};

int b[3];

// Array decay happens here

// 'a' decays to a pointer to first element

b = a; // here int * = &a[0] = a

C++ Structures

A structure in C++ is a user-defined data type that allows grouping variables of different data types under a single name, so they can be used and managed as one unit. Structures are also known as structs.

Example:

struct Books {

char title[50];

char author[50];

char subject[100];

int book_id;

};

int main() {

Books Book1; // In c++ we can write it without struct keyword

struct Books Book2; // Declare Book2 of type Book

}

Structures as Function Arguments

You can pass a structure as a function argument in very similar way as you pass any other variable or pointer.

```
void printBook( struct Books book );
```

Pointers to Structures

You can define pointers to structures in very similar way as you define pointer to any other variable as follows –

```
struct Books *struct_pointer;
```

```
struct_pointer = &Book1;
```

```
struct_pointer->title;
```

The typedef Keyword:

There is an easier way to define structs or you could "alias" types you create.

Now, you can use *Books* directly to define variables of *Books* type without using struct keyword.

```
typedef struct {
```

```
char title[50];
} Books;
Books Book1, Book2;
```

memory allocation: In a structure, memory is allocated for all members separately, along with additional padding for proper alignment, when a structure variable is created.

```
struct C {
   int x;    // 4 bytes
   char c;   // 1 byte
};
```

x → 4 bytes

c → 1 byte

padding → 3 bytes (at end)

Total = 8 bytes

Padding is added automatically by the compiler to make access faster.

Structs in C are just data containers

No methods

- No constructors
- No operator overloads
- No encapsulation

C Structures	C++ Structures
Only data members are allowed, it cannot have member functions.	Can hold both: member functions and data members.
Cannot have static members.	Can have static members.
Cannot have a constructor inside a structure.	Constructor creation is allowed.
Direct Initialization of data members is not possible.	Direct Initialization of data members is possible.
Writing the 'struct' keyword is necessary to declare structure-type variables.	Writing the 'struct' keyword is not necessary to declare structure-type variables.
Do not have access modifiers.	Supports access modifiers.
Only pointers to structs are allowed.	Can have both pointers and references to the struct.
Sizeof operator will generate 0 for an empty structure.	Sizeof operator will generate 1 for an empty structure.
Data Hiding is not possible.	Data Hiding is possible.

C Structures	C++ Structures
Inheritance not supported	Supported inheritence

In C++, struct **and** class **are almost identical.**
The **only differences are defaults**:

Feature	struct	class
Default member access	public	private
Default inheritance	public	private

Enumeration (Enum)

Enumeration or simply enum is a user-defined data type that is used to give name to integer constants in a program. This increases the user-readability of a program.

enum Day { Sunday, // 0 Monday, // 1 Tuesday, // 2 Wednesday, // 3 Thursday, // 4 Friday = 45, // 45 Saturday // 46 };

int main() { // Declaring a variable for a day

Day get1 = Wednesday;

cout<<get1<<endl; cout<<Saturday<<endl;

return 0; }

enum class which is also referred to as scoped enumerations. To solve issue of potential name clashes and the absence of type safety.

enum class Day { Monday, // 0 Tuesday, // 1 Wednesday, // 2 Thursday, // 3 Friday, // 4 Saturday, // 5 Sunday // 6 };

int main()

{ Day today = Day::Friday;

if (today == Day::Friday) { cout << "Today is Friday!" << endl; }

return 0; }

Unions in C++

In C++, a **union** is a user-defined data type that allows you to store different data types in the same memory location. However, the union can store only one of its member variables at a time, which means that if you assign a value to one member, the previous value stored in another member is overwritten. Where the size of a union is determined by the size of its largest member.

EXAMPLE:

union Data {

int intValue; // Member for integer value

```
float floatValue; // Member for float value

char strValue[50]; // Member for string value

};

int main() {

// Defining a union variable

Data data;

data.intValue = 2006;

data.floatValue = 5.75f;

cout << "My Float value is: " << data.floatValue << endl; // this is ok

// Accessing the integer after assigning a string

cout << "Integer after string assignment: " << data.intValue << endl;

// Undefined behaviour print garbage value

}
```

Anonymous Unions

Anonymous unions are a special type of union which don't have a name. This helps in simplifying code by allowing you to direct access to union members without specifying the union variable name. but we can only write inside same scope.

```
union {

int intValue;

float floatValue;

char strValue[50];

};

// Assigning an integer value

intValue = 2006;

cout << "Integer Value: " << intValue << endl;
```

Union-like Classes

In C++ a **union-like** class is defined as a class that includes at least one anonymous union as a member. The data members defined within these anonymous unions are known as variant members.

```
class UnionLikeClass {
```

```
public:
// Anonymous union declaration
union {
int intValue; // Member for integer value
float floatValue; // Member for float value
char strValue[50]; // Member for string value
};
// Method to display the current value
void display() {
cout << "Integer Value: " << intValue << endl;
cout << "Float Value: " << floatValue << endl;
cout << "String Value: " << strValue << endl;
}
};
int main() {
// Creating an instance of UnionLikeClass
UnionLikeClass data;
data.intValue = 2006;
cout << "TutorialsPoint: Founded in " << data.intValue << endl;
}
```

Difference between Structure, Union, and Enum (C++)

Feature	Structure (struct)	Union (union)	Enum (enum)
Definition	User-defined data type that groups variables of different types	User-defined data type where all members share the same memory	User-defined data type consisting of named constant values
Memory allocation	Separate memory for each member	Shared memory for all members	Memory for a single integer type
Total size	Sum of all members + padding	Size of the largest member	Size of int (usually)
Members active at a time	All members can be used simultaneously	Only one member at a time	Only one value at a time
Data types allowed	Different data types	Different data types	Only integral constants

Supports functions	✅ Yes (in C++)	✅ Yes (in C++)	❌ No
Supports inheritance	✅ Yes	❌ No	❌ No
Supports polymorphism	✅ Yes	❌ No	❌ No
Memory efficiency	❌ Less efficient	✅ Very efficient	✅ Very efficient
Use case	Represent real-world objects	Save memory when only one value is needed	Represent fixed set of choices
Example use	Student, Employee	Variant data, protocol fields	Days, States, Colors

C++ Pointers:

A pointer is a variable that stores the memory address of another variable.

dereferencing is the process that helps in accessing the value that a pointer points to.

you can use the dereference operator denoted by (*).

```
Type value = *pointer;
```

Special Types of Pointers

NULL Pointer

- The NULL Pointers are those pointers that do not point to any memory location.
- They can be created by assigning NULL value to the pointer. A pointer of any type can be assigned the NULL value.

```
int *ptr = NULL;
```

```
int* ptrName = nullptr
int* ptrName = 0; // by assigning the value 0
```

Void Pointer(Generic Pointer)

- The void pointers in C are the pointers of type void.
- It means that they do not have any associated data type.
- They are also called generic pointers as they can point to any type and can be typecasted to any type.

```
 int a = 10;

 char b = 'x';

   // void pointer holds address of int 'a'

   void* p = &a;
```

```
    // void pointer holds address of char 'b'
    p = &b;
```

Properties of Void Pointers

1. Void pointers cannot be dereferenced.
Example:

```
#include <stdio.h>
int main()
{
    int a = 10;
    void* ptr = &a;
    printf("%d", *ptr);
    return 0;
}
```

Output: Compiler Error: 'void*' is not a pointer-to-object type

But we can write printf("%d", *(int*)ptr);

2. **The C standard doesn't allow pointer arithmetic with void pointers. However, in GNU C it is allowed by considering the size of the void as 1**

   This not work in c++ because void has no size

```
#include <stdio.h>
int main()
{
    // Declare and initialize an integer array 'a' with two
    // elements
    int a[2] = { 1, 2 };
    // Declare a void pointer and assign the address of
    // array 'a' to it
    void* ptr = &a;

    // Increment the pointer by the size of an integer
    ptr = ptr + sizeof(int);

    // The void pointer 'ptr' is cast to an integer
    // pointer using '(int*)ptr' Then, the value is
```

```
    // dereferenced with `*(int*)ptr` to get the value at
    // that memory location
    printf("%d", *(int*)ptr);
    return 0;
}
```

Advantages of Void Pointers in C
Following are the advantages of void pointers

- malloc() and calloc() return void * type and this allows these functions to be used to allocate memory of any data type (just because of void *).
- void pointers used along with Function pointers of type **void (*)(void) point** to the functions that take no arguments and not return any value.
- void pointers are mainly used in the implementation of data structures such as linked lists, trees, and queues i.e. dynamic data structures. Because data structures often store **different data types**, and `void*` allows this flexibility.

```
struct Node {
    void *data;
    struct Node *next;
};
int x = 10;
node->data = &x;
```

- void pointers are also commonly used for typecasting.

```
int x = 10;
void *vp = &x;
int *ip = (int *)vp;
```

Wild Pointers

The wild pointers are pointers that have not been initialized with something yet. These types of C-pointers can cause problems in our programs and can eventually cause them to crash. If values are updated using wild pointers, they could cause data abort or data corruption. they point to some arbitrary memory location and may cause a program to crash or behave unexpectedly.
int *ptr is wild because it points to some garbage memory by default, but int *ptr = NULL is not wild because it clearly points to nothing.

```
int *ptr;
  /* Some unknown memory location is being corrupted.
This should never be done. */
*ptr = 12;
```

How can we avoid wild pointers?

If a pointer points to a known variable then it's not a wild pointer.

Example
In the below program, p is a wild pointer till this points to a.

```
int main()
```

```
{
   int* p; /* wild pointer */
   int a = 10;
   /* p is not a wild pointer now*/
   p = &a;
   /* This is fine. Value of a is changed */
   *p = 12;
}
```

Dangling Pointer

- A pointer pointing to a memory location that has been deleted (or freed) is called a dangling pointer.
- Such a situation can lead to unexpected behavior in the program and also serve as a source of bugs in C programs.

There are three different ways where a pointer acts as a dangling pointer:

- **1. De-allocation of Memory**

When a memory pointed by a pointer is deallocated the pointer becomes a dangling pointer.

```
int* ptr = (int*)malloc(sizeof(int));

// After below free call, ptr becomes a dangling pointer

free(ptr);

printf("Memory freed\n");

// removing Dangling Pointer

ptr = NULL;
```

2. Function Call

When the local variable is not static and the function returns a pointer to that local variable. The pointer pointing to the local variable becomes dangling pointer.

Example

```
#include <stdio.h>


int* fun()

{

   // x is local variable and goes out of

   // scope after an execution of fun() is

   // over.

   int x = 5;

   return &x;

}

// Driver Code

int main()

{
```

```
    int* p = fun();
    fflush(stdin);
    // p points to something which is not
    // valid anymore
    printf("%d", *p);
    return 0;
}
```

3. Variable Goes Out of Scope
When a variable goes out of scope the pointer pointing to that variable becomes a dangling pointer.

Example

```
#include <stdio.h>
#include <stdlib.h>
// driver code
int main()
{
    int* ptr;
    // creating a block
    {
        int a = 10;
        ptr = &a;
    }
    // ptr here becomes dangling pointer
    printf("%d", *ptr);
    return 0;
}
```

Pointer to Pointer (Double Pointer)

A pointer that stores the **address of another pointer**. size of the variable of the double-pointer and the size of the normal pointer variable is always equal.

```
int x = 10;
int *p = &x;
int **pp = &p;
printf("%d", **pp); // 10
```

Common Use Cases

1. Modifying a Pointer in a Function: If you pass a pointer to a function and want to change where that original pointer points (e.g., reallocating memory or updating a linked list head), you must pass its address as a double pointer.

   If you pass a pointer normally, you **cannot change where it points**.

   ```
   void changePointer(int *p) {//here we take p as value
      int x = 20;
      p = &x;   // changes only local copy
   }
   int main() {
      int a = 10;
      int *p = &a;
      changePointer(p);
      printf("%d\n", *p); // still 10 ✗
   }
   ```

   Solution: Pass pointer's address (double pointer)

   ```
   void changePointer(int **pp) {
      static int x = 20;
      *pp = &x;   // modifies original pointer
   }
   int main() {
      int a = 10;
      int *p = &a;

      changePointer(&p);
      printf("%d\n", *p); // 20 ✓
   }
   ```

2. **Dynamic 2D Arrays:** A double pointer is often used to create a 2D array where each row can be of a different length or is allocated separately on the heap.

   A dynamic 2D array using int ** is NOT a real matrix.it is An array of pointers, where each pointer points to a row

   int **arr;

   This means:

   arr → will point to many int *

   Each int * → will point to a row

   ## Allocate memory for ROW pointers

   arr = malloc(rows * sizeof(int *));

   ## Allocate memory for EACH ROW

   ```
   for (int i = 0; i < rows; i++) {
      arr[i] = malloc(cols * sizeof(int));
   }
   ```

3. **Command-Line Arguments:** The standard char** argv in a main function is a pointer to an array of string pointers.

   Example:

```
#include <stdio.h>

int main(int argc, char **argv) {
   for (int i = 0; i < argc; i++) {
      printf("argv[%d] = %s\n", i, argv[i]);
   }
   return 0;
}
```

   - Each argument is a char * (string)
   - argv points to multiple char *
   - So → pointer to pointer → char **

Function Pointer:

A pointer that stores the **address of a function**. **The function pointer's type must exactly match the function signature**

Signature =

- return type
- parameter types

```
int add(int a, int b) {
   return a + b;
}
int (*fp)(int, int) = add; //or int (*fp)(int, int)  and do fp=add
printf("%d", fp(2, 3)); // 5
```

What is a callback?

Passing a function to another function, so it can be called later.

Example: Callback

```
#include <stdio.h>

void success() {
```

```
    printf("Operation successful!\n");
}

void process(void (*callback)()) {
    printf("Processing...\n");
    callback();   // call later
}

int main() {
    process(success);
    return 0;
}
```

Handling Ctrl+C (SIGINT)

```
#include <stdio.h>
#include <signal.h>

void handler(int signum) {
    printf("Signal %d caught!\n", signum);
}

int main() {
    signal(SIGINT, handler);
    while (1);
}//handler is passed as a function pointer to signal()
```

Constant Pointer

The **address cannot change**, but the value can.

✓ Example

```
int x = 10;
int *const p = &x;

*p = 20;   // ✓ allowed
// p = &y; ✗ not allowed
```

Pointer to Constant

The **value cannot change**, but the pointer can.

✓ Example

```
int x = 10, y = 20;
const int *p = &x;

// *p = 30; ✗ not allowed
p = &y;    // ✓ allowed
```

Constant Pointer to Constant

Neither **address nor value** can change.

✓ Example

```
int x = 10;
const int *const p = &x;
```

Array Pointer

A pointer that points to an **entire array**, not just its first element.

`p` is a pointer. `*p` is an array. array size is `5` . array element type is `int`

✓ Example

```
int arr[5];
int (*p)[5] = &arr;// (*p)[0] for accessing
```

⚠ Different from:

```
int *p = arr; // points to first element
```

Why do we even need array pointers?

Passing 2D arrays to functions

```
void print(int (*arr)[4], int rows) {// here this pointer point to address of
zero'th array that is address of arr[0]
    for(int i = 0; i < rows; i++)
        for(int j = 0; j < 4; j++)
            printf("%d ", arr[i][j]);
}
```

Call:

```
int a[3][4];
print(a, 3);//here a means &a[0]
```

Why this works:

- `arr` points to an array of **4 ints**
- Compiler knows row size

int (*p2)[5] = &arr; **one 1D array** of 5 ints

int (*p)[4] = arr; **first row of a 2D array**

int **p = arr; // WRONG

Because:

int** expects pointers to pointers

2D array is contiguous memory

No row pointers exist

Structure Pointer

A pointer pointing to a **structure variable**.

✓ Example

```
struct Student {
    int id;
```

```
};

struct Student s;
struct Student *p = &s;
p->id = 10;
```

Modifying Structure Inside Function

❌ Without pointer (won't change original)

```
void update(struct Student s) {
    s.marks = 100;
}
```

✅ With pointer (changes original)

```
void update(struct Student *s) {
    s->marks = 100;
}
```

Dynamic Structure (Heap Allocation)

```
struct Student *p;
p = malloc(sizeof(struct Student));
```

Array of Structures using Pointer

```
struct Student arr[3];
struct Student *p = arr;// access using p[1].id because p[1].id  ≡  (*(p + 1)).id = (p+1)->id
```

Using pointer without memory

```
struct Student *p;
p->id = 10;  // 💥 crash
```

Smart Pointers :

since C++11, in the `<memory>` header

Smart pointers in C++ are **class templates that wrap around a raw pointer to automatically manage the lifetime of dynamically allocated memory**, thereby preventing memory leaks and dangling pointers. They implement the Resource Acquisition Is Initialization (RAII) idiom, ensuring that resources are released when the smart pointer object goes out of scope.

RAII (Resource Acquisition Is Initialization) **means**

a resource is acquired when an object is created and automatically released when the object is destroyed.

auto_ptr

Ownership is transferred, leaving the original pointer invalid.

auto_ptr was an early smart pointer that automatically deleted the managed object when it went out of scope.

std::auto_ptr is deprecated because copying it transfers ownership, which causes **unexpected behavior and bugs**.

When you **copy an auto_ptr**, the original pointer **loses ownership** and becomes NULL.

This breaks:

- Normal copy semantics
- STL containers (like vector)
- Programmer expectations

Example:

```
#include <iostream>
#include <memory>
using namespace std;
int main() {
    auto_ptr<int> ptr1(new int(10));
    cout << *ptr1 << endl;

    auto_ptr<int> ptr2 = ptr1;  // ownership transfer
    cout << *ptr2;
    return 0;
}
```

`std::unique_ptr` :

Enforces **sole ownership** of the dynamically allocated object.
It cannot be copied, only moved to a new owner using `std::move()`, which makes the original pointer null.

It is lightweight and efficient, having the same size as a raw pointer. Creation is typically done using `std::make_unique` (available since C++14).

unique_ptr stores one pointer only at a time. We cannot copy unique_ptr, only transfer ownership of the object to another unique_ptr using the move() method.

unique_ptr automatically deletes the object it points to when it goes out of scope, so there's no need to manually call delete.

We can write it also = > unique_ptr<A> ptr1 = make_unique<A>();
// will give compile time error

```
unique_ptr<A> p2 = p1;
```

Example:

```
#include <iostream>

#include <memory>

using namespace std;

class Rectangle {

    int length, breadth;

public:

    Rectangle(int l, int b) : length(l), breadth(b) {}

    int area() { return length * breadth; }

};

int main() {
```

```
    unique_ptr<Rectangle> P1(new Rectangle(10, 5));
    cout << P1->area() << endl;
    unique_ptr<Rectangle> P2;
    P2 = move(P1);  // ownership transfer
    cout << P2->area();
    return 0;
}
```

shared_ptr

shared_ptr allows multiple pointers to share ownership of the same object. It uses reference counting to manage memory.

it has an associated control block that keeps track of the reference count for the managed object.

Initialization using a New Pointer

```
std::shared_ptr<T> ptr(new T());
std::shared_ptr<T> ptr = std::make_shared<T>();
```

Initialization using existing Pointer

```
shared_ptr<T> ptr(already_existing_pointer);
shared_ptr<T> ptr = make_shared(already_existing_pointer);
```

Example:

```
#include <iostream>
#include <memory>
using namespace std;
class Rectangle {
    int length, breadth;
public:
    Rectangle(int l, int b) : length(l), breadth(b) {}
    int area() { return length * breadth; }
};
int main() {
    shared_ptr<Rectangle> P1(new Rectangle(10, 5));
    shared_ptr<Rectangle> P2 = P1;
    cout << P1->area() << endl;
    cout << P2->area() << endl;
```

```
    cout << P1.use_count();
    return 0;
}
```

weak_ptr

The weak_ptr, just like shared_ptr has the capability to point to the resource owned by another shared_ptr but without owning it. In other words, they are able to create a non-owning reference to the object managed by shared_ptr.

Key Characteristics

- Does not increase reference count
- Prevents circular dependency
- Must be converted using lock() before use

Need of weak_ptr
To understand the need for weak_ptr, we need to first understand the use case of shared_ptr which leads to a common problem called a circular link. It occurs when two or more objects reference each other using a shared_ptr. For example, if ObjectA has a shared_ptr to ObjectB and ObjectB has a shared_ptr to ObjectA, they form a circular reference. This can be problematic because neither ObjectA nor ObjectB will ever be deleted, leading to a memory leak.

Here, weak_ptr comes to the rescue by providing a way to break these circular references. It allows you to create a non-own reference to an object managed by shared_ptr without affecting the reference count or preventing the object from being deleted.

Example:

```
#include <iostream>
#include <memory>
using namespace std;
// declaring a dummy object
class Object {
public:
    Object(int value)
        : data(value)
    {
        cout << "Object created with value: " << data
             << endl;
    }
    ~Object()
    {
        cout << "Object destroyed with value: " << data
```

```
            << endl;
    }
    int data;
};
// driver code
int main()
{
    // creating shared pointer with resource ownership
    shared_ptr<Object> sharedObjectA
        = make_shared<Object>(42);
    // creating weak pointer to the previously created
    // shared objects
    weak_ptr<Object> weakObjectA = sharedObjectA;
    // Access objects using weak_ptr
    if (!weakObjectA.expired()) {
        cout << "The value stored in sharedObjectA:"
            << (*weakObjectA.lock()).data << endl;
    }
    // deleting object
    sharedObjectA.reset();
    cout << "End of the Program";
    return 0;
}
```

Applications of weak_ptr
The following are the primary applications of the weak_ptr:

- **Preventing Circular References:** The primary application of weak_ptr is to prevent circular references. When an object wants to reference another object without owning it, it can use weak_ptr. This ensures that no circular references are created and objects can be safely freed when they are no longer needed.
- **Cache Systems:** weak_ptr is commonly used in cache implementations. Caches often need to temporarily store references to objects without preventing the deletion of those objects when they are no longer in use. weak_ptr provides an elegant solution for this use case.

C++ Object Oriented

The prime purpose of C++ programming was to add object orientation to the C programming language, which is in itself one of the most powerful programming languages.The core of the pure

object-oriented programming is to create an object, in code, that has certain properties and methods. While designing C++ modules, we try to see whole world in the form of objects. For example a car is an object which has certain properties such as color, number of doors, and the like. It also has certain methods such as accelerate, brake, and so on.

Object

An **object** is a real instance of a **class**.
It represents a real-world entity and contains **data (variables)** and **behavior (functions)** defined by its class.

Class

A **class** is a **user-defined data type** that acts as a **blueprint** for creating objects.It defines **data members (variables)** and **member functions (methods)**.
Defining a member function within the class definition declares the function inline, even if you do not use the inline specifier.
They are not always inline if function is large the compiler treat it as normal funciton.

Encapsulation

Definition:
Encapsulation is the process of **wrapping data and functions together** into a single unit (class) and **hiding data** using access specifiers.

Abstraction

Definition:
Abstraction means **showing only essential details** and **hiding internal implementation** from the user.

Inheritance

Definition:
Inheritance allows a **new class (child)** to acquire the **properties and behavior** of an **existing class (parent)**.

Polymorphism

Definition:
Polymorphism means **one function name having many forms**, depending on the object or arguments.

C++ Class Access Modifiers

C++ access modifiers are used for data hiding implementation.

Public Access Modifier

The public access modifier defines public data members and member functions that are accessible from anywhere outside the class but within a program. You can set and get the value of public variables without any member function.

Example:

```
#include <iostream>
```

```
 using namespace std;

class Line {

public:

double length;

void setLength( double len );

 double getLength( void );

};

// Member functions definitions

double Line::getLength(void) {

return length ;

}

void Line::setLength( double len) {

 length = len;

}

// Main function for the program

 int main() {

Line line;

// set line length line.setLength(6.0);

cout << "Length of line : " << line.getLength() <<endl;

// set line length without member function

 line.length = 10.0; // OK: because length is public

cout << "Length of line : " << line.length <<endl;

return 0;

}
```

Private Access Modifier

The access modifier defines private data members and member functions that cannot be accessed, or even viewed from outside the class. Only the class and friend functions can access private members.By default all the members of a class would be private.

Example:

```
#include <iostream>
using namespace std;
class Box {
public:
double length;
void setWidth( double wid );
double getWidth( void );
private:
double width;
};
// Member functions definitions d
ouble Box::getWidth(void) {
return width ;
}
void Box::setWidth( double wid ) {
width = wid;
}
// Main function for the program
int main() {
Box box;
// set box length without member function
box.length = 10.0; // OK: because length is public
cout << "Length of box : " << box.length <<endl;
// set box width without member function
// box.width = 10.0; // Error: because width is private
box.setWidth(10.0); // Use member function to set it.
cout << "Width of box : " << box.getWidth() <<endl;
return 0;
```

```
}
```

Protected Access Modifier

The access modifier defines protected data members and member functions that are very similar to a private member, but it provides one additional benefit that they can be accessed in child classes, which are called derived classes.

Example:

```
#include <iostream>

using namespace std;

class Box {

protected:

double width;

};

class SmallBox:Box { // SmallBox is the derived class.

public:

void setSmallWidth( double wid );

double getSmallWidth( void );

};

// Member functions of child class

double SmallBox::getSmallWidth(void) {

return width ;

}

void SmallBox::setSmallWidth( double wid ) {

width = wid;

}

// Main function for the program

 int main() {

SmallBox box;

// set box width using member function
```

box.setSmallWidth(5.0);

cout << "Width of box : "<< box.getSmallWidth() << endl;

return 0;

}

Inheritance Type Public becomes Protected becomes

public public protected

protected protected protected

private private private

Access modifiers are compile-time restrictions only.Does not affect on memory.

class A {

public:

int a;

private:

int b;

protected:

int c;

};

| a | b | c | //All stored contiguously.

Static Members of a C++ Class

We can define class members static using keyword. When we declare a member of a class as static it means no matter how many objects of the class are created, there is only one copy of the static member.

Example:

#include <iostream> using namespace std; class Box {

public:

static int objectCount;

// Constructor definition

Box(double l = 2.0, double b = 2.0, double h = 2.0) {

cout <<"Constructor called." << endl;

length = l; breadth = b; height = h;

// Increase every time object is created objectCount++;

}

double Volume() {

return length * breadth * height;

}

private:

double length; // Length of a box double breadth; // Breadth of a box double height; // Height of a box

};

// Initialize static member of class Box int Box::objectCount = 0;

int main(void) {

Box Box1(3.3, 1.2, 1.5); // Declare box1

Box Box2(8.5, 6.0, 2.0); // Declare box2

// Print total number of objects.

cout << "Total objects: " << Box::objectCount << endl; //object count is 2 return 0;

}

Static Function Members

A function that belongs to the class, not to objects.

By declaring a function member as static, you make it independent of any particular object of the class. A static member function can be called even if no objects of the class exist and the scope resolution operator functions are accessed using only the class name and the :: . A static member function can only access static data member, other static member functions and any other functions from outside the class.

Example:

class Test {

public:

int x = 5; static int y;

static void show() { cout << y; // ✓ OK

// cout << x; ✗ ERROR

}

};

Example2:

void hello() {

cout << "Hello";

}

class Test { public:

static void show() { hello(); // ✅ Allowed

}

};

Example3:

static void show(Demo obj) {

cout << obj.x; // ✅ OK (explicit object)

}

Static member functions have a class scope and they do not have access to

the pointer of the class. You could use a static member function to determine

whether some objects of the class have been created or not.

this Pointer

Every object in C++ has access to its own address through an important pointer called this pointer. The this pointer is an implicit parameter for all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.

Friend functions do not have a this pointer because friends are not members of a class. Only member function have this pointer.

Example:

#include <iostream>

using namespace std;

class Box {

public:

// Constructor definition

Box(double l = 2.0, double b = 2.0, double h = 2.0) {

```
cout <<"Constructor called." << endl;
length = l;
breadth = b;
height = h;
}
double Volume() {
return length * breadth * height;
}
int compare(Box box) {
return this->Volume() > box.Volume();
}
private:
double length; // Length of a box
double breadth; // Breadth of a box
double height; // Height of a box
};
int main(void) {
Box Box1(3.3, 1.2, 1.5); // Declare box1
Box Box2(8.5, 6.0, 2.0); // Declare box2
if(Box1.compare(Box2)) {
cout << "Box2 is smaller than Box1" <<endl;
} else {
cout << "Box2 is equal to or larger than Box1" <<endl;
}
return 0;
}
```

Return Calling Object's Reference Using this Pointer

To implement chain function calls, you need the reference of a calling object. You can use the "this" pointer to return the reference of the calling object.

Example:

```
#include <iostream>

using namespace std;

class Coordinates {

private:

int latitude;

int longitude;

public:

Coordinates(int lat = 0, int lon = 0) {

this->latitude = lat;

this->longitude = lon;

}

Coordinates& setLatitude(int lat) {

latitude = lat;

return *this;

}

Coordinates& setLongitude(int lon) {

longitude = lon;

return *this;

}

void display() const {

cout << "Latitude = " << latitude << ", Longitude = " << longitude << endl;

}

};

int main() {

Coordinates location(15, 30);
```

```
// Chained function calls modifying the same object

location.setLatitude(40).setLongitude(70);

location.display();

return 0;

}
```

Characteristics of the "this" Pointer

- The "this" pointer refers to the current instance of the object, where it allows the member function to access the object's attributes and methods.
- The "this" pointer is implicitly passed to all non-static member functions, where you don't need to explicitly write this in code.
- this points to the memory location of the current object.
- If there is a name conflict between a parameter and a member variable, this can be used to differentiate the member variable from the local parameter.
- the "this" pointer is constant (const), meaning it cannot be modified.
- Since this is a pointer, it can be dereferenced to access the current object. this pointer helps in preventing self-assignment in assignment operators making sure that an object doesn't assign itself to itself.
- this pointer makes method chaining possible by returning the current object (usually through *this), allowing you to call several methods on the same object in a single line of code.

this Pointer in Const Member Functions Vs Static Member Functions

In const member functions, this pointer is a pointer to a constant object (const MyClass*), where the object's members cannot be modified within the function, resulting in an object remaining unchanged when calling const functions.

Whereas static member functions don't have this pointer because they are not associated with any specific instance of the class, they belong to the class itself and can only access static members or methods, as they do not operate on object-specific data.

Example:

```
#include <iostream>

class MyClass {

public:

MyClass(int val) : data(val) {}

// Const member function (has 'this' pointer, but it's a const pointer)

void printData() const {

std::cout << "Data: " << data << std::endl; // 'this' points to a const object

// Uncommenting the next line will cause an error because 'this' is const
```

```
data = 10; // Error: cannot modify 'data' in a const member function

}

// Static member function (no 'this' pointer, operates on class-level data)

static void showMessage() {

std::cout << "This is a static function!" << std::endl;

// Uncommenting the next line will cause an error because static functions can't access instance members

std::cout << "Data: " << data << std::endl; // Error: 'data' is not accessible

}

private:

int data;

};

int main() {

MyClass obj(5);

// Calling const member function (can access 'this' as const)

obj.printData();

// Calling static member function (no 'this' pointer available)

MyClass::showMessage();

return 0;

}
```

C++ Friend Function

A friend function of a class is defined outside that class scope but it has the right to access all private and protected members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions.

A friend can be a function, function template, member function, or a class or class template, in which case the entire class and all of its members are friends.

Example:

```
#include <iostream>

using namespace std;

class ClassName {
```

```
private:

int privateData; // Private member

protected:

int protectedData; // Protected member

public:

ClassName() : privateData(0), protectedData(0) {}

// Declare a friend function

friend void friendFunction(ClassName& obj);

};

// Friend function definition

void friendFunction(ClassName& obj) {

obj.privateData = 42; // Access private member

obj.protectedData = 24; // Access protected member

cout << "Private Data: " << obj.privateData << endl;

cout << "Protected Data: " << obj.protectedData << endl;

}

int main() {

ClassName obj;

friendFunction(obj); // Call the friend function

return 0;

}
```

Friend Function vs Member Function

In C++, both friend functions and member functions are used to access and manipulate the data of a class, but still, they have significant differences in their scope and usage.

Friend Function

A friend Function is a non-member function that is declared inside a class using the "friend" keyword, it has special access to the class's private and protected members. Since it's not a member it is not bound to a specific object, can't overloaded based on objects, not use this pointer, and cannot be inherited by derived classes. They are defined outside the class but declared inside it.

Member function

Whereas the member function is defined within the class and operates using this pointer. It can access all members of the class (private, protected, and public), and as it is tied to class objects, it can be overloaded and inherited by derived classes.

Friend Classes

In C++, a friend class is a class that gives access to private and protected members of another class. When a class declares another class as a friend, the second class (the friend) can directly access the private and protected members of the first class.

Example:

```
#include <iostream>

using namespace std;

class ClassB; // Forward declaration of ClassB

class ClassA {

private:

int dataA;

public:

ClassA() : dataA(42) {} // Initialize dataA with a value

friend class ClassB;

};

class ClassB {

public:

void showDataFromA(const ClassA& objA) {

// Access private member of ClassA

cout << "Data from ClassA: " << objA.dataA << endl;

}

};

int main() {

ClassA objA;

ClassB objB;
```

```
// Use ClassB to access ClassA's private data

objB.showDataFromA(objA);

return 0;

}
```

Pointer to Classes

A pointer to a C++ class is done exactly the same way as a pointer to a structure and to access members of a pointer to a class you use the member access operator -> operator, just as you do with pointers to structures. Also as with all pointers, you must initialize the pointer before using it.

Example:

```
#include <iostream>

using namespace std;

class Box {

public:

// Constructor definition

Box(double l = 2.0, double b = 2.0, double h = 2.0) {

cout <<"Constructor called." << endl;

length = l;

breadth = b;

height = h;

}

double Volume() {

return length * breadth * height;

}

private:

double length; // Length of a box

double breadth; // Breadth of a box

double height; // Height of a box

};

int main(void) {
```

```
Box Box1(3.3, 1.2, 1.5); // Declare box1

Box Box2(8.5, 6.0, 2.0); // Declare box2

Box *ptrBox; // Declare pointer to a class.

// Save the address of first object

ptrBox = &Box1;

// Now try to access a member using member access operator

cout << "Volume of Box1: " << ptrBox->Volume() << endl;

// Save the address of second object

ptrBox = &Box2;

// Now try to access a member using member access operator

cout << "Volume of Box2: " << ptrBox->Volume() << endl;

return 0;

}
```

The Class Constructor

A class constructor is a special member function of a class that is executed whenever we create new objects of that class.

A constructor will have exact same name as the class and it does not have any return type at all, not even void. Constructors can be very useful for setting initial values for certain member variables.

Parameterized Constructor

Whereas a parameterized constructor is a constructor, which takes one or more parameters and is used to initialize an object with specific values passed when an object is created.

Example:

```
#include <iostream>

using namespace std;

class Line {

public:

void setLength( double len );

double getLength( void );

Line(double len); // This is the constructor
```

```
private:
double length;
};
// Member functions definitions including constructor
Line::Line( double len) {
cout << "Object is being created, length = " << len << endl;
length = len;
}
void Line::setLength( double len ) {
length = len;
}
double Line::getLength( void ) {
return length;
}
// Main function for the program
int main() {
Line line(10.0);
// get initially set length.
cout << "Length of line : " << line.getLength() <<endl;
// set line length again
line.setLength(6.0);
cout << "Length of line : " << line.getLength() <<endl;
return 0;
}
```

Using Initialization Lists to Initialize Fields

```
Line::Line( double len): length(len) {
cout << "Object is being created, length = " << len << endl;
}
```

Above syntax is equal to the following syntax −

```
Line::Line( double len) {

cout << "Object is being created, length = " << len << endl;

length = len;

}
```

If for a class C, you have multiple fields X, Y, Z, etc., to be initialized,

```
C::C( double a, double b, double c): X(a), Y(b), Z(c) {

....

}
```

What is Constructor Delegation?
Constructor delegation means **one constructor calls another constructor of the same class** to reuse initialization logic.

Example:

```
class Test {

   int x, y;

public:

   Test() : Test(10) {}        // delegates to Test(int)

   Test(int a) : x(a), y(20) {}

};
```

Default Constructors

C++ default constructors do not take any parameters. If a default constructor is not provided by the programmer explicitly, then the compiler provides a implicit default constructor. In that case, the default values of the variables are 0.

Implicit Default Constructor

An implicit default constructor is automatically generated by the compiler only if you do not define any constructor yourself.

```
class Student {

public:

int id;

};

int main() {
```

Student s; // compiler creates implicit default constructor }

Explicit Default Constructor

An explicit default constructor is written by the programmer.

class Student {

public:

int id;

Student() { // explicit default constructor

id = 0;

}

};

int main() {

Student s; // calls user-defined constructor

}

Example2:

class ClassName {

public:

explicit ClassName(int x = 10);

};

ClassName a = 5; // ✗ ERROR

ClassName b(5); // ✓ OK

ClassName c; // ✓ OK (uses default value x = 10)

✗ Without explicit

```
class ClassName {
public:
   ClassName(int x = 10);
};
```

ClassName a = 5; // ✓ allowed (implicit conversion)

Copy Constructor

A **copy constructor** is a special constructor that initializes a **new object** using an **existing object of the same class**.

- Compiler creates a default copy constructor if there is no user defined constructor. This compiler created constructor does a shallow copy and works well for basic types, but does not work for dynamic allocated memory.
- If a class has pointers or runtime resources (like dynamic memory, file handles, network connections), a custom copy constructor is needed. Otherwise, deletion of one object might cause deletion of an unrelated object members.

Default Copy Constructor

Example:

```
class A

{

  public:

    int x;

};

int main()

{

    // Creating an a1 object

    A a1;

    a1.x = 10;

    cout << "a1's x = " << a1.x << endl;

    // Creating another object using a1

    A a2(a1);

    cout << "a2's x = " << a2.x;

    return 0;

}
```

User Defined Copy Constructor

C++ also allows programmers to create their own version of copy constructor known as user defined or explicit copy constructor.

```
className (const ClassName &obj) {
   // Body of copy constructor
}
```

Here, the **const** is optional but is added so that we do not modify the **obj** by mistake. Copy constructor takes a reference to an object of the same class as an argument. Using & because when we pass obj without & it goes pass by value and because of that copy onstructor agan called. So it goes and infinite recursion.

If we define a copy constructor, then implicit definition of the default constructor will not be provided by the compiler. So, we would have to manually define it too.

Shallow Copy (Default Behavior)

A **shallow copy** copies **only the data members**, including **pointer addresses**, not the actual memory they point to. Both objects end up pointing to the same memory. shallow copy is ok when class has no dynamic memory.

Example:

```
#include <iostream>

using namespace std;

class Test {

public:

   int *ptr;

   Test(int x) {

       ptr = new int(x);

   }

   // Shallow copy constructor

   Test(const Test &t) {

       ptr = t.ptr;   // ✕ same address copied

   }

   ~Test() {

       delete ptr;

   }

};

int main() {

   Test t1(10);

   Test t2 = t1;  // shallow copy
```

} //*t2.ptr = 50;   // t1.ptr also becomes 50 😲

❌ Problem with Shallow Copy

- Both objects share **same heap memory**
- When destructors run:
  - First delete → OK
  - Second delete → ❌ **double free / crash**
- Changing one object affects the other

Deep Copy

A **deep copy** Allocates **new memory.** Copies **actual value**, not address. Each object owns its **own memory**

Example:

```
#include <iostream>

using namespace std;

class Test {

public:

   int *ptr;

   Test(int x) {

      ptr = new int(x);

   }

   // Deep copy constructor

   Test(const Test &t) {

      ptr = new int(*t.ptr);  // ✅ new memory

   }

   ~Test() {

      delete ptr;

   }

};

int main() {

   Test t1(10);

   Test t2 = t1;  // deep copy
```

```
    *t2.ptr = 50;

    cout << *t1.ptr << endl; // 10

    cout << *t2.ptr << endl; // 50

}
```

Need of User Defined Copy Constructor

- If no copy constructor is defined, C++ provides a default copy constructor.
- The default copy constructor performs a shallow copy, shallow copy copies pointer values, not the actual resources they point to.
- If a class has pointers or runtime resources (like dynamic memory, file handles, network connections), a custom copy constructor is needed.
- Without a custom copy constructor, shallow copy can lead to dangling pointers or resource conflicts.

When is the Copy Constructor Called?

In C++, a copy constructor may be called in the following cases:

- When an object of the class is returned by value.
- When an object of the class is passed (to a function) by value as an argument.
- When an object is constructed based on another object of the same class.
- When the compiler generates a temporary object.

Copy Constructor vs Assignment Operator

The main difference between Copy Constructor and Assignment Operator is that the Copy constructor makes a new memory storage every time it is called while the assignment operator does not make new memory storage.

Which of the following two statements calls the copy constructor and which one calls the assignment operator?

```
MyClass t1, t2;
MyClass t3 = t1;  // ----> (1)
t2 = t1;          // -----> (2)
```

A copy constructor is called when a new object is created from an existing object, as a copy of the existing object. The assignment operator is called when an already initialized object is assigned a new value from another existing object. In the above example (1) calls the copy constructor and (2) calls the assignment operator.

Move Constructor:

A **move constructor** is a special constructor that **transfers ownership of resources** (like heap memory) from a **temporary object (rvalue)** to a new object, **without copying**, to improve performance.

Example:

```
#include <iostream>

using namespace std;

class Test {

public:
```

```
    int *ptr;
    // Constructor
    Test(int x) {
        ptr = new int(x);
    }
    // Copy constructor
    Test(const Test &t) {
        ptr = new int(*t.ptr);
    }
    // 🔥 Move constructor
    Test(Test &&t) {
        ptr = t.ptr;       // steal resource
        t.ptr = nullptr;  // invalidate source
    }
    ~Test() {
        delete ptr;
    }
};
int main() {
    Test t1(10);
    Test t2 = std::move(t1);  // move constructor called
}
```

Const or Reference Members

Const variables and reference members cannot be reassigned, so they must be initialized in an initialization list,

```
class Config {
public:
Config(const std::string& product, const int & model)
```

: product (product), model(model) {}

private:

const std::string product;

const int & model;

};

Base Class Initialization

When a derived class inherits from a base class, you can use an initialization list to call the base class constructor,

class Base {

public:

Base(int value) : baseValue(value) {}

protected:

int baseValue;

};

class Derived : public Base {

public:

Derived(int value, int extra) : Base(value), extraValue(extra) {}

private:

int extraValue;

};

Dynamic Initialization Using Constructors

Dynamic initialization is the process of initializing variables or objects at runtime using constructors.

While creating an object, its constructor is called and if the constructor contains logic to initialize the data members with values, is known as dynamic initialization.

ClassName* objectName = new ClassName(constructor_arguments);

The Class Destructor

A destructor is a special member function of a class that is executed whenever an object of it's class goes out of scope or whenever the delete expression is applied to a pointer to the object of that class.

A destructor will have exact same name as the class prefixed with a tilde (~) and it can neither return a value nor can it take any parameters. Destructor can be very useful for releasing resources before coming out of the program like closing files, releasing memories etc.

C++ provides a default destructor for every class as the classes are created. The default destructor cleans up the statically allocated memory. But, if we have used a pointer for dynamic memory allocation using new, then we need to define a custom destructor with delete operator to clean up the memory. Destructors cannot be overloaded. Destructors cannot be inherited. You can not define more than one destructor. Destructors are called in **reverse order** of construction for multiple objects.

Virtual Destructor in C++

A virtual destructor is a destructor declared within the base class with the virtual keyword. When deleting a derived class object using a pointer to a base class, the base class should be defined with a virtual destructor.

Virtual destructors help prevent the problem of memory leaks and undefined behavior. If a virtual destructor is not used, then only the base class destructor is called, and the resources of derived class is not cleaned up.

Example:

```
#include <iostream>

using namespace std;

class Base {

public:

Base()

{

cout << "Base Constructor\n";

}

virtual ~Base()

{

cout << "Base Destructor\n";

}

};

class Derived : public Base {

public:

Derived()

{
```

```
cout << "Derived Constructor\n";

}

~Derived()

{

cout << "Derived Destructor\n";

}

};

int main() {

Base *basePtr = new Derived();

delete basePtr;

return 0;

}
```

When to Use Virtual Destructors?

You can use a virtual destructor in the following scenarios.

If a virtual function is used, then use virtual destructor. Since, virtual functions are used in polymorphism to override base class. Using a virtual destructor can free up memory of both, base and derived class.

In inheritance, make the destructor of base class a virtual destructor as the destructor of derived class is skipped and only the base class destructor is called, it may cause a memory leak.

If abstract base or pure virtual function is used, then use virtual destructor on base class.

Virtual Destructor Table (Vtable)

In C++, virtual table(vtable) is a lookup table which is used by the compiler to implement virtual functions or virtual destructors. For each class that has at least one virtual function or virtual destructor, the compiler will create a vtable for that class.

A vtable has pointers to all virtual functions and the virtual destructor of a class. In a class, each object has a hidden pointer known as vptr or virtual pointer. This vptr points to the vtable of the class, which is then used to call the respective destructor.

Virtual destructor ensures the correct destructor is called at runtime using the vtable when deleting objects through base class pointers.

Step-by-Step Working

When we declare a function as virtual in the base class, it enables dynamic binding instead of static binding.

For a class with virtual functions, the compiler creates a vtable, which stores pointers to those functions.

Here each object of a class with virtual functions has a vptr, which points to the class's vtable.

Then a base class pointer uses vptr to fetch the correct function from the vtable at runtime, ensuring the derived class's overridden function is called.

Inheritance (C++)

Inheritance is an OOP feature in C++ where a **new class (derived/child class)** acquires the **properties (data members) and behaviors (member functions)** of an **existing class (base/parent class)**, so code can be **reused, extended, and specialized**.

A derived class inherits all base class methods with the following exceptions −

- Constructors, destructors and copy constructors of the base class.
- Assignment operator is NOT inherited.
- The friend functions of the base class.

Public Inheritance − When deriving a class from a public base class, public members of the base class become public members of the derived class and protected members of the base class become protected members of the derived class. A base class's private members are never accessible directly from a derived class, but can be accessed through calls to the public and protected members of the base class.

Protected Inheritance − When deriving from a protected base class, public and protected members of the base class become protected members of the derived class.

Private Inheritance − When deriving from a private base class, public and protected members of the base class become private members of the derived class.

Types of Inheritance in C++

Single Inheritance

Single inheritance is a type of inheritance in which **one derived class inherits from exactly one base class**.

Multiple Inheritance

Multiple inheritance is a feature that allows a class to inherit from more than one base class, which means a derived class can have multiple parent classes and inherit attributes and behaviors from all the base classes. // class Rectangle : public Shape, public PaintCost{}

Multilevel Inheritance

Multilevel inheritance is a type of inheritance in which **a class is derived from another derived class**, forming a chain.

Hierarchical Inheritance

Hierarchical inheritance is a type of inheritance in which **multiple derived classes inherit from the same base class**.

Hybrid Inheritance

Hybrid inheritance is a type of inheritance that is formed by **combining two or more different types of inheritance**.☞ Combination of **multiple + hierarchical + multilevel**

Ambiguity Problem in C++

Ambiguity occurs when a derived class inherits from two or more base classes that contain members with the same name, and the compiler cannot decide which one to use.

Example:

```
#include <iostream>

using namespace std;

class A {

public:

   void show() {

      cout << "Show from A\n";

   }

};

class B {

public:

   void show() {

      cout << "Show from B\n";

   }

};

class C : public A, public B {

};

int main() {

   C obj;

   obj.show();   // ✕ ERROR: ambiguous

}
```

Solution:

```
class C : public A, public B {

public:

   void show() {

      A::show();   // or B::show()

   }

};
```

Diamond Problem in C++

The Diamond problem occurs when a class inherits from two classes that both inherit from the same base class, resulting in multiple copies of the base class and causing ambiguity and memory duplication. This is structural ambiguity + memory issue**.**

Example:

```
#include <iostream>

using namespace std;

class A {

public:

   int x;

};

class B : public A {

};

class C : public A {

};

class D : public B, public C {

};

int main() {

   D obj;

   obj.x = 10;   // ✕ ERROR: ambiguous

}
```

Solution:

```
class B : virtual public A {
```

```
};

class C : virtual public A {

};
```

Virtual Function

A virtual function is a member function of a class that is declared using the keyword virtual and is meant to be overridden in a derived class.
It ensures runtime polymorphism, meaning the function call is resolved at runtime based on the type of object pointed to, not the type of pointer.
Virtual function makes sure that the derived class's function is called even when using a base class pointer. It signals to the compiler that we don't want static linkage for this function.

Constructors, Static, and Friend Functions cannot be virtual.

Pure Virtual Function

A pure virtual function **is a** virtual function that has no implementation in the base class **and is declared using = 0.**
It forces derived classes to provide their own implementation**.** When you declare pure virual function so its must to be overridden in derived class.

Data Abstraction in C++

Data abstraction refers to providing only essential information to the outside world and hiding their background details, i.e., to represent the needed information in program without presenting the details.

Let's take one real life example of a TV, which you can turn on and off, change the channel, adjust the volume, and add external components such as speakers, VCRs, and DVD players, BUT you do not know its internal details, that is, you do not know how it receives signals over the air or through a cable, how it translates them, and finally displays them on the screen.

ADT — Abstract Data Type

An Abstract Data Type (ADT) **is a** logical description of a data type **that defines:**

- What operations can be performed
- What behavior those operations have
- It does NOT define how data is stored or how operations are implemented.

ADT = What you can do, not how you do it

Stack ADT

Operations:

- push()
- pop()
- peek()
- isEmpty()

✕ No info about:

- Array or linked list

- Memory allocation
- Index handling

noexcept: This function will NEVER throw an exception.

```
void func() noexcept {
// no exceptions thrown here
}
```

Abstraction Class

An abstract class is a class that contains **at least one pure virtual function**.

- Cannot create object
- Can have:
  - Data members
  - Constructors
  - Concrete functions
  - Virtual & pure virtual functions

Pure Abstract Class (Interface)

A **pure abstract class** is a class where:

- All functions are pure virtual
- No data members
- Only method declarations

Can a pure virtual function have a body?

But ☞ **it is NOT overridden automatically** and **cannot be called directly via base object**.

```
class Base {
public:
   virtual void show() = 0;   // pure virtual
};
```

Now give it a body **outside** the class:

```
void Base::show() {
   cout << "Base show implementation" << endl;
}
```

Can a pure abstract class have a constructor? = yes

- You **cannot create object** of abstract class
- Constructor runs **when derived object is created**

◆ **Example**

```
class Base {
public:
   Base() {
      cout << "Base constructor" << endl;
```

```
    }
    virtual void fun() = 0;
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived constructor" << endl;
    }
    void fun() {}
};
```

 Output

```
Base constructor
Derived constructor
```

☞ Abstract class constructor is used to:

- Initialize data members
- Set base state

Can a pure abstract class have a destructor? = yes

And it SHOULD be virtual

◆ **Example**

```
class Base {
public:
    virtual ~Base() {
        cout << "Base destructor" << endl;
    }
    virtual void fun() = 0;
};

class Derived : public Base {
public:
    ~Derived() {
        cout << "Derived destructor" << endl;
    }
    void fun() {}
};

int main() {
    Base* b = new Derived();
    delete b;
}
```

 Output

```
Derived destructor
Base destructor
```

☞ Without virtual destructor → **memory leak / undefined behavior**

Must constructors be overridden? = No

- Constructors are **not inherited**
- They **cannot be overridden**
- Each class has its own constructor

Must destructors be overridden? = No

- Base destructor **must be virtual**
- Derived destructor runs automatically

Encapsulation in C++

encapsulation is to bind the data members and methods into a single unit. and restricting direct access to the data from outside the class.

A class can hide the implementation part and discloses only the functionalities required by other classes. By making class data and methods private

How Encapsulation Is Achieved in C++

Encapsulation is implemented using:

1. **Classes**
2. **Access Specifiers**
    - private
    - protected
    - public

Rule of Zero, Five, Three

Rule of Zero, Three, and Five in C++
The Rule of Zero, Three, and Five is a set of guidelines for managing object resources in modern C++, related to structures and classes. These rules deal with the default behavior of constructors, destructors, and other special member functions that are necessary for proper resource management.

Rule of Zero
If a class does not manage resources directly, it should define NONE of the special member functions. The Rule of Zero states that if a class or structure does not explicitly manage resources, it should not define any of the special member functions, i.e., destructor, copy constructor, copy assignment operator, move constructor, and move assignment operator. The compiler will automatically generate these functions, and the behavior will be correct for managing resources like memory and file handles.

Example:
```
struct MyResource {
   std::string name;
   int value;
};
```
In this example, MyResource is a simple structure that does not manage any resources, so it does not define any special member functions. The compiler will generate them automatically, and the behavior will be correct.

Example (Rule of Zero)

```
#include <iostream>
#include <memory>
using namespace std;

class Test {
   unique_ptr<int> ptr;

public:
   Test(int val) : ptr(make_unique<int>(val)) {}
};
```

✓ No destructor
✓ No copy constructor
✓ No assignment operator
✓ No move functions

Compiler generates everything safely.

Rule of Three

The Rule of Three states that a class or structure that manages resources should define the following three special member functions:

- Destructor
- Copy constructor
- Copy assignment operator

These functions are necessary for proper resource management, such as releasing memory or correctly handling deep copies.

Example*:*

```
class MyResource {
public:
   // Constructor and destructor
   MyResource() : data(new int[100]) {}
   ~MyResource() { delete[] data; }

   // Copy constructor
   MyResource(const MyResource& other) : data(new int[100]) {
      std::copy(other.data, other.data + 100, data);
   }

   // Copy assignment operator
   MyResource& operator=(const MyResource& other) {
      if (&other == this) { return *this; }
      std::copy(other.data, other.data + 100, data);
      return *this;
   }

private:
   int* data;
};
```

In this example, MyResource is a class that manages a resource (an array of integers), so it defines the destructor, copy constructor, and copy assignment operator.

Rule of Five
The Rule of Five extends the Rule of Three to include two additional special member functions:

- Move constructor
- Move assignment operator

Modern C++ introduces move semantics, which allows for more efficient handling of resources by transferring ownership without necessarily copying all the data.

Example:

```
class MyResource {
public:
    // Constructors and destructor
    MyResource() : data(new int[100]) {}
    ~MyResource() { delete[] data; }

    // Copy constructor
    MyResource(const MyResource& other) : data(new int[100]) {
        std::copy(other.data, other.data + 100, data);
    }

    // Copy assignment operator
    MyResource& operator=(const MyResource& other) {
        if (&other == this) { return *this; }
        std::copy(other.data, other.data + 100, data);
        return *this;
    }

    // Move constructor
    MyResource(MyResource&& other) noexcept : data(other.data) {
        other.data = nullptr;
    }

    // Move assignment operator
    MyResource& operator=(MyResource&& other) noexcept {
        if (&other == this) { return *this; }
        delete[] data;
        data = other.data;
        other.data = nullptr;
        return *this;
    }

private:
    int* data;
};
```

auto (Automatic Type Deduction)

Auto
auto is a keyword in C++ language introduced in C++11, which is used for automatic type deduction. It automatically deduces the type of a variable from the type of its initializer expression at compile time. The **auto** keyword is useful when you are dealing with complex types or when the type of a variable is hard to predict. It can help in writing cleaner and less error-prone code.
Here's a simple example of using **auto** for type deduction:

```
#include <iostream>
#include <vector>
```

```
int main() {
    // Traditional way of declaring a variable:
    int myInt = 5;

    // Using auto for type deduction:
    auto myAutoInt = 5; // Automatically deduces the type as 'int'

    // Example with more complex types:
    std::vector<int> myVector = {1, 2, 3, 4, 5};

    // Without auto, iterating the vector would look like this:
    for (std::vector<int>::iterator it = myVector.begin(); it != myVector.end(); ++it) {
        std::cout << *it << '\n';
    }

    // With auto, the iterator declaration becomes simpler:
    for (auto it = myVector.begin(); it != myVector.end(); ++it) {
        std::cout << *it << '\n';
    }
}
```

Keep in mind that **auto** deduces the type based on the initializer expression, so if you don't provide an initial value, you will get a compile-time error:

```
auto myVar; // Error: Cannot deduce the type without initializer
```

In C++14, you can also use **auto** with function return types to let the compiler automatically deduce the return type based on the returned expression:

```
auto add(int x, int y) {
    return x + y; // The compiler deduces the return type as 'int'
}
```

Dynamic Typing in C++

C++ is known as a statically-typed language, which means the data types of its variables are determined at compile time. However, C++ also provides concepts to have certain level of *dynamic typing*, which means determining the data types of variables at runtime.

void* Pointers

A **void*** pointer is a generic pointer that can point to objects of any data type. They can be used to store a reference to any type of object without knowing the specific type of the object.

Example:

```
#include <iostream>


int main() {

    int x = 42;

    float y = 3.14f;

    std::string z = "Hello, world!";


    void* void_ptr;


    void_ptr = &x;
```

```
    std::cout << "int value: " << *(static_cast<int*>(void_ptr)) << '\n';

    void_ptr = &y;
    std::cout << "float value: " << *(static_cast<float*>(void_ptr)) << '\n';

    void_ptr = &z;
    std::cout << "string value: " << *(static_cast<std::string*>(void_ptr)) << '\n';

    return 0;
}
```

std::any (C++17)

C++17 introduced the **std::any** class which represents a generalized type-safe container for single values of any type.

Example:

```
#include <iostream>
#include <any>

int main() {
    std::any any_value;

    any_value = 42;
    std::cout << "int value: " << std::any_cast<int>(any_value) << '\n';

    any_value = 3.14;
    std::cout << "double value: " << std::any_cast<double>(any_value) << '\n';

    any_value = std::string("Hello, world!");
    std::cout << "string value: " << std::any_cast<std::string>(any_value) << '\n';

    return 0;
}
```

dynamic_cast operator

dynamic_cast is a type-casting operator that performs a runtime type check and safely downcasts a

base pointer or reference to a derived pointer or reference. It returns null or throws a bad_cast exception (if casting references) when the casting fails.

```
#include <iostream>


class Base { virtual void dummy() {} };

class Derived1 : public Base { /* ... */ };

class Derived2 : public Base { /* ... */ };


int main() {

    Base* base_ptr = new Derived1;


    // Using dynamic_cast to safely downcast the pointer

    Derived1* derived1_ptr = dynamic_cast<Derived1*>(base_ptr);

    if (derived1_ptr) {

        std::cout << "Downcast to Derived1 successful\n";

    }

    else {

        std::cout << "Downcast to Derived1 failed\n";

    }


    Derived2* derived2_ptr = dynamic_cast<Derived2*>(base_ptr);

    if (derived2_ptr) {

        std::cout << "Downcast to Derived2 successful\n";

    }

    else {

        std::cout << "Downcast to Derived2 failed\n";

    }


    delete base_ptr;

    return 0;
```

If the conversion is invalid:

- For **pointers** → returns nullptr
- For **references** → throws std::bad_cast

```
Derived* d = dynamic_cast<Derived*>(basePtr);
Derived& r = dynamic_cast<Derived&>(baseRef);
```

Memory Model in C++

The memory model in C++ defines how the program stores and accesses data in computer memory. It consists of different segments, such as the Stack, Heap, Data and Code segments. Each of these segments is used to store different types of data and has specific characteristics.

Stack Memory

Stack memory is used for automatic storage duration variables, such as local variables and function call data. Stack memory is managed by the compiler, and it's allocation and deallocation are done automatically. The stack memory is also a LIFO (Last In First Out) data structure, meaning that the most recent data allocated is the first to be deallocated.

```
void functionExample() {

   int x = 10; // x is stored in the stack memory

}
```

Heap Memory

Heap memory is used for dynamic storage duration variables, such as objects created using the **new** keyword. The programmer has control over the allocation and deallocation of heap memory using **new** and **delete** operators. Heap memory is a larger pool of memory than the stack, but has a slower access time.

```
void functionExample() {

   int* p = new int; // dynamically allocated int in heap memory

   *p = 10;

   // more code

   delete p; // deallocate memory

}
```

Data Segment

The Data segment is composed of two parts: the initialized data segment and the uninitialized data segment. The initialized data segment stores global, static, and constant variables with initial values, whereas the uninitialized segment stores uninitialized global and static variables.

```
// Initialized data segment

int globalVar = 10; // global variables

static int staticVar = 10; // static local variables

const int constVar = 10; // constant variables with value



// Uninitialized data segment

int globalVar; // uninitialized global variables
```

Code Segment

The Code segment (also known as the Text segment) stores the executable code (machine code) of the program. It's usually located in a read-only area of memory to prevent accidental modification.

```
void functionExample() {
  // The machine code for this function is stored in the code segment.
}
```

In summary, understanding the memory model in C++ helps to optimize the usage of memory resources and improves overall program performance.

Object Lifetime in C++

Object lifetime refers to the time during which an object exists, from the moment it is created until it is destroyed. In C++, an object's lifetime can be classified into four categories:

•Static Storage Duration: Objects with static storage duration exist for the entire run of the program. These objects are allocated at the beginning of the program's run and deallocated when the program terminates. Global variables, static data members, and static local variables fall into this category.

```
int global_var;           // Static storage duration

class MyClass {
  static int static_var;  // Static storage duration
};

void myFunction() {
  static int local_var;   // Static storage duration
}
```

•Thread Storage Duration: Objects with thread storage duration exist for the lifetime of the thread they belong to. They are created when a thread starts and destroyed when the thread exits. Thread storage duration can be specified using the **thread_local** keyword.

```
thread_local int my_var;  // Thread storage duration
```

•Automatic Storage Duration: Objects with automatic storage duration are created at the point of definition and destroyed when the scope in which they are declared is exited. These objects are also known as "local" or "stack" objects. Function parameters and local non-static variables fall into this category.

```
void myFunction() {
  int local_var;          // Automatic storage duration
}
```

•Dynamic Storage Duration: Objects with dynamic storage duration are created at runtime, using memory allocation functions such as **new** or **malloc**. The lifetime of these objects must be managed manually, as they are not automatically deallocated when the scope is exited. Instead, it is the programmer's responsibility to destroy the objects using the **delete** or **free** functions when they are no longer needed, to avoid memory leaks.

```
int* ptr = new int;       // Dynamic storage duration

delete ptr;
```

Standard Template Library (STL) in C++

STL is a collection of pre-built classes and functions that make it easy to manage data using common data structures like vectors, stacks, and maps. It saves time and effort by providing ready-to-use, efficient algorithms and containers.

Containers in C++ STL

Standard Template Library (STL) provides the built-in implementation of commonly used data structures known as containers.

Each container is implemented as a template class that also contains the methods to perform basic operations on it. Every STL container is defined inside its own header file.

1. Sequence Containers

Sequence containers implement linear data structures in which the elements can be accessed sequentially.

Array class in C++

- Array classes knows its own size, whereas C-style arrays lack this property. So when passing to functions, we don't need to pass size of Array as a separate parameter.
- With C-style array there is more risk of array being decayed into a pointer. Array classes don't decay into pointers
- Array classes are generally more efficient, light-weight and reliable than C-style arrays.

Example:

```
#include <iostream>
#include <array> // including header file to use stl array and there function
using namespace std;
template<size_t N>
void printArr(array<int,N>& arr){
    for(array<int,6>::iterator it =arr.begin();it!=arr.end();it++)
  {
    cout<<*it<<" ";
  }
  cout<<endl;
}
int main()
{
    //we cannot do that because array size is constexpr means evaluate at compile
time.
//     int n =5;
//  array<int,n> arr={1,2,3,4,5};

 array<int,5> arr{1,2,3,4,5};
 for(int i =0;i<5;i++){
    //cout<<arr.at(i)<<" ";
     cout<<arr[i]<<" ";
 }
  cout<<endl;

  cout<<get<0>(arr)<<endl;
  for(array<int,6>::iterator it =arr.begin();it!=arr.end();it++)
  {
    cout<<*it<<" ";
  }
```

```
 cout<<endl;
 cout<<"size: "<<arr.size()<<endl;
 cout<<"size: "<<arr.max_size()<<endl;

 std::array<int,3> a1 = {1,2,3};
std::array<int,6> a2 = {1,2,3,4,5,6};
//a1.swap(a2); this give error because size is not same
printArr(a1);
printArr(a2);


int *ptr= arr.data();
cout<<"fist element using ptr "<<*ptr<<endl;

// rbegin() → points to the last element
// rend() → points to one before the first element (reverse sentinel)

for(auto it = arr.rbegin(); it != arr.rend(); ++it)
    std::cout << *it << " ";
    cout<<endl;

int f = arr.front();
int l = arr.back();
cout<<"fist element "<<f<<endl;
cout<<"last element "<<l<<endl;
cout<<"is arr empty "<<arr.empty()<<endl;
//arr.fill(9);
//printArr(arr);
//below are some non member function
// arr1 == arr2
// arr1 != arr2
// arr1 < arr2
// swap(arr1,arr2);
// std::get<2>(arr);
    return 0;
}
```

size() :- It returns the number of elements in array. This is a property that C-style arrays lack. **max_size()** :- It returns the maximum number of elements array can hold i.e, the size with which array is declared. The size() and max_size() return the same value. **swap()** :- The swap() swaps all elements of one array with other. **empty()** :- This function returns true when the array size is zero else returns false. **fill()** :- This function is used to fill the entire array with a particular value. **get()** :- This function is also used to access the elements of array. This function is not the member of array class but overloaded function from class tuple.

Vector in C++ STL

A vector represents a dynamic sized array in the Standard Template Library(STL) that automatically grows when elements are added beyond current capacity. Certain insertions and removals can become slightly costly in terms of time as it internally resizes itself beyond certain predefined limits.

- push_back(value) : Inserts element at the end of the vector efficiently.
  Takes constant time O(1) on average, so it's very fast.
- insert(position, value) : Inserts element at any position (beginning, middle, or end).
  Takes linear time O(n), as it shifts elements to make space.

- You can access elements in a vector using v[i] for direct access or v.at(i) for bounds-checked access.
- Both return the element at index i, but at(i) throws an exception if i is out of range.
- It takes constant time O(1) on average, so it's very fast.

```cpp
#include<iostream>
#include <vector>
#include<array>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v={1,2,3,45};

    vector<int> v1(v);
    std::vector<int> v3 = v1;
    for(auto i : v1){
        cout<<i<<" ";
    }
    vector<vector<int>> matrix1(3,vector<int>(4,0));
    cout<<endl;
    std::vector<int> v4(5, 10); //10 10 10 10 10
    std::vector<int> v5(5);
    std::vector<int> v6(std::move(v1));
    std::vector<int> v7 = std::move(v1);
    array<int,5>arr1;
    std::vector<int> v8(arr1.begin(), arr1.end());
    int arr[] = {1,2,3,4};
    std::vector<int> v9(arr, arr + 4);

    v.push_back(10);
        v.pop_back();
        v.insert(v.begin(),56);
      //  v.erase(v.begin());
      //v.erase(v.begin(),v.begin()+2);
      v.erase(find(v.begin(),v.end(),2));
      for(auto i : v){
        cout<<i<<" ";
    }
    cout<<endl;
    cout<<v.front()<<endl;
    cout<<v.back()<<endl;
    cout<<*(v.data())<<endl;
    cout<<v.capacity()<<endl;

//     vector<vector<int>> matrix = {
//  {1, 2, 3},
//  {4, 5, 6},
//  {7, 8, 9}
// };
// // 3 rows , 4 columns all initialized to 0
// vector<vector<int>>matrix(3,vector<int>(4,0));
// v1 == v2
// v1 != v2
// v1 <  v2
```

```
    // v1 <= v2
    // v1 >  v2
    // v1 >= v2
        return 0;
    }
```

Deque in C++ STL

Deque stands for Double-Ended Queue. It's a sequence container that allows you to add or remove elements efficiently from both the front and the back. Deque can act as both stack and Queue.

```
#include <iostream>
#include <algorithm>
#include <deque>
using namespace std;

int main()
{
    // deque<int> d(5);
    // std::deque<int> d(5, 10);
    //     std::vector<int> v{1,2,3};
    // std::deque<int> d(v.begin(), v.end());
    // std::deque<int> d2(d1);
    // d.assign(5, 100);      // five 100s
    // d.assign(std::begin(arr), std::end(arr));
    std::deque<int> d;
    d = {1, 2, 3, 4};
    // Adding elements at the back
 d.push_back(10);
// Adding elements at the front
 d.push_front(30);

d.pop_front();
d.pop_back();
d. front(); //Returns the first element. Does not remove it.
d. back(); //Returns the last element. Useful to check the end value.

d.insert(d.begin() + 2, 25);
d.erase(d.begin() + 2);
d.insert(d.begin() + 1, 3, 100); //Insert 100 three times.
cout<<"deque elements\n";
for(int i=0;i<d.size();i++){
    cout<<d[i]<<" ";
}
cout<<endl;
d.clear(); //Removes all elements from deque. Deque becomes empty after this.
    return 0;
}
```

List in C++ STL

A list in C++ is a sequence container that allows you to store elements one after another.

- Implemented as a doubly linked list and maintains both front and back for fast operations on both the ends.
- Data is stored in non-contiguous memory, allowing fast insertion and deletion anywhere in the list (beginning, middle, or end).

```cpp
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

int main()
{
    list<int> l = {1, 2, 3, 4, 5};
    // list<char> l1(5);
    // list<int> l2(5,6);
    // list<int> l3(l);
    // l=l2;
    auto it = l.begin();
    advance(it, 2);
    l.insert(it, 4);
    // updating element
    *it = 20;
    auto it1 = find(l.begin(), l.end(), 4); // print 4
    l.erase(it1);
    l.push_back(10);
    l.push_front(50);

    l.pop_back();
    l.pop_front();
    cout << l.front() << endl;
    cout << l.back() << endl;
    cout << *next(l.begin(), 2) << endl;
    l.remove(20);
    l.sort();
    l.push_back(1);
    l.unique();//emoves only consecutive duplicates
    for (list<int>::iterator it = l.begin(); it != l.end(); it++)
    {
        cout << *it << " ";
    }
    //     auto it = d.cbegin();
    // *it = 100;   // ❌ ERROR const

    cout << endl;
    return 0;
}
```

- **Forward List in C++ STL**

Forward_list container provides the implementation of the singly linked list data structure. It stores data in non-contiguous memory where each element points to the next element in the sequence. This makes insertion and deletion faster once the position of the element is known.

Accessing Elements

forward_list does not support index access like arrays or vectors. Elements must be accessed sequentially using iterators (with next() or advance()), except the first element which can be accessed directly using front().

```
forward_list<int> fl = {1, 5, 3, 4};

fl.front();

auto it = next(fl.begin(), 2); //3
```

Inserting Elements

```
 // Inserting Element at front

   fl.push_front(1);

   // Insert 3 after the second element

auto it = fl.begin();

   advance(it, 1);

   fl.insert_after(it, 3); // inset() does not exist
```

Updating Elements

```
// Updating third element

   auto it = next(fl.begin(), 2);

   *it = 333;
```

Deleting Elements

```
// Delete first element

   fl.pop_front();

  // Delete third element

   auto it = fl.begin();

   advance(it, 1);

   fl.erase_after(it); //erase does not exist
```

Size of Forward List

```
int size = distance(flist.begin(),flist.end()); //does not have bult in size fuction.
```

Reverse iteration not possible so rbegin() not present

2. Associative Containers

Associative containers store data in some sorted order. It provides fast search, insert and delete in O(log n) time by using balanced trees like Red Black Trees.

Set in C++ STL

A Set is a container which stores unique elements in some sorted order. It is an implementation of a Self-Balancing Binary Search Tree, specifically a Red-Black Tree which ensures,

- Search, insert, and delete in O(log n) time.
- Does not allow duplicates.
- Elements are always sorted in ascending order by default. You can also choose your own way of ordering them using a custom rule (comparator).

A **comparator** is a rule that tells the container: "When comparing two elements, which one should come first?"

If you want descending order, you must change the comparator.

By default it set to set<int, less<int>> s;

```cpp
#include <iostream>
#include <set>
#include <algorithm>
using namespace std;

int main()
{
    set<int, greater<int>> s1 = {1, 2, 3, 4, 5};
    set<int, greater<int>> s2{1, 2, 3, 4, 5};
    //  set<int> s3(5); we cannot do this
    // set<int> s4(s2); //we canot do this
    set<int, greater<int>> s4(s2);
    s2 = s1;
    set<int> s5{1, 2, 3};
    set<int, greater<int>> s6(s5.begin(), s5.end());
    for (auto i : s1)
    {
        cout << i << " ";
    }
    cout << endl;
    set<int> s;
    s.insert(10);
    s.insert(5);
    s.insert(20);
    s.insert(15);
    auto it1 = s.lower_bound(20);
    auto it2 = s.upper_bound(5);

    cout << "lower bound " << *it1 << endl;
    cout << "upper bound " << *it2 << endl;
    cout << *next(s.begin(), 2) << endl;
    if (s.count(15))
        cout << "15 exists in the set" << endl;
```

```
    //     s[0]; // ❌ not allowed
    // s.at(0); // ❌ not allowed
    s.erase(5);
    auto it = s.find(20);
    if (it != s.end())
        cout << "Element found: " << *it << endl;

    for (auto i : s)
    {
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

you cannot modify an element of set

Why?

Because in std::set, elements are treated as **const**.

Why?

Because the set is internally a balanced tree (Red-Black Tree).
If you change a value directly, the ordering may break.

Map in C++ STL

Maps are associative containers that store key–value pairs in sorted order using a self-balancing Red-Black Tree. They ensure O(log n) time for search, insertion, and deletion, do not allow duplicate keys, and provide ordered traversal along with functions like upper_bound() and lower_bound().

// Initialze map with list

map<int, string> m2 = {{1, "Geeks"},

{2, "For"}, {3, "Geeks"}};

// Inserting a key value pair

m.insert({1, "Geeks"});

// Accessing elements

cout << m[1] << endl;

cout << m.at(2);

// Updating value

m[0] = "Tweaks";

m.at(1) = "By";

// Finding element with key 2

```
    auto it = m.find(2);

    if (it != m.end())

        cout << it->first << " " << it->second;

   // Deleting by key

    m.erase(2);

    m.erase(m.begin());
```

```
vector<pair<int,string>> v = {
{1,"one"},
{2,"two"}
};

map<int,string> m(v.begin(), v.end());
map<int,string> m1{{1,"one"}};
map<int,string> m2(m1);
m2 = m1;
map<int,string> m2(move(m1));
map<int,string, greater<int>> m;
```

Multiset in C++ STL

Multiset is an associative container similar to a set, but it can store multiple elements with the same value. It is sorted in increasing order by default, but it can be changed to any desired order using a custom comparator.

- Allows duplicate elements, unlike a set.
- Supports standard set operations like insert(), erase(), count(), and find().
- It also internally seems to be using a Self-Balancing Binary Search Tree (like a set), handling duplicates by making a rule to either insert in the left or right subtree to ensure O(Log n) time complexity of operations.

```
// Inserting elements (duplicates allowed)

    ms.insert(5);

// Access third element

    auto it2 = next(it1, 2);

// Finding an element (3)

    auto it = ms.find(3);

    // Delete first element

    ms.erase(ms.begin());
```

Multimap in C++ STL

Multimap is an associative container similar to map, but it can have multiple elements with same keys. It stores all the elements in increasing order based on their keys by default but can be changed if required.

```
// Inserting key-value pairs

    mm.insert({1, "Geeks"});
```

Elements are accessed only through iterators, using ->first for keys and ->second for values, since [] is not available. You can move iterators with next() or advance(), and quickly get the first and last elements using begin() and end().

```
// Accessing 2nd element

    auto it = next(f, 1);

    cout << it->first << ": " << it->second;

    // Updating 2nd element value

    auto it = next(mm.begin(), 1);

    it->second = "Java";

    // Find element with key 2

    auto it = mm.find(2);

    cout << it->first << ": " << it->second;

    // Delete element using key

    mm.erase(2);

    // Delete element using iterator

    mm.erase(mm.find(1));
```

3. Unordered Associative Containers

Unordered associative containers implement unsorted hashed data structures that can be quickly searched (O(1) amortized, O(n) worst-case complexity).

Hashing in Data Structure

Hashing is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access.

- Hashing involves mapping data to a specific index in a hash table (an array of items) using a hash function**.** It enables fast retrieval of information based on its key.
- The great thing about hashing is, we can achieve all three operations (search, insert and delete) in O(1) time on average.

Unordered Sets in C++ STL

In C++, unordered_set is an unordered associative container that stores unique elements.

- Internally uses a hashing to store elements which provides average constant time O(1) for insert, search and delete operations.
- Elements have no particular order and are best used when fast access is more important than sorted order. On other hand set is used for sorted order and less access time.

```
// Accessing third element

    auto it = next(us.begin(), 2);
```

In unordered set, the value of the elements cannot be changed after it is inserted.

```
auto it = us.find(4);

    if (it != us.end())

        cout << *it;

// Delete element by position

    us.erase(us.begin());
```

insert → “Take this object.”
emplace → “Build it here using these arguments.”

Unordered Map in C++ STL

In C++, unordered_map is an associative container that stores unique key–value pairs using hashing, offering fast access. Unlike map, it does not maintain elements in sorted order.

- Uses hashing for data storage
- Elements are not stored in any specific order
- Faster than **map** when ordering is not required

```
// Insert elements using square brackets

    um[1] = "Geeks";

    // Insert elements using insert() method

    um.insert({2, "For"});

    um.insert({3, "C++"});

  // using [] operator

    cout << um[2] << endl;

    cout << um.at(1);

//updating

um[2] = "By"
```

```
um.at(1) = "Tips";

// Finding element with key 2

auto it = um.find(2);

// Delete element which have key 2

um.erase(2);

// Delete first element

um.erase(um.begin());
```

Unordered Multiset in C++ STL

In C++, **unordered multiset** is an unordered associative container that works similarly to an unordered set, but it can store multiple copies of the same value. It provides fast insert, delete and search operations using hashing, but the elements are not in any particular order.

Updating Elements

```
auto it = s.find(20);

*it = 50; // ✕ compile error
```

Unordered Multimap in C++ STL

In C++, the **unordered_multimap** is an unordered associative container that stores data in the form of key-value pairs. It is similar to unordered map, but it allows multiple elements with the same key. It provides fast insertion, deletion and search operations in O(1) time by using **hashing**.

```
// Find 2nd element

auto it = next(umm.begin(), 1);

// Searching for key '2' using find()

auto it = umm.find(2);



// Updating value

it->second = "Java";
```

Container	Ordered?	Duplicate Keys?	Can Modify Key?	Can Modify Value?
map	Yes	No	✕	✓
multimap	Yes	Yes	✕	✓
unordered_map	No (hash)	No	✕	✓
unordered_multimap	No (hash)	Yes	✕	✓
set	Yes	No	✕	✕

Container	Ordered?	Duplicate Keys?	Can Modify Key?	Can Modify Value?
multiset	Yes	Yes	✗	✗
unordered_set	No	No	✗	✗
unordered_multiset	No	Yes	✗	✗

4.Container Adapters

Container adapters in C++ are not real containers.

They are wrappers built **on top of existing containers** that restrict the interface to enforce a specific behavior.

A **container adapter** is a class template that:

- Uses another container internally
- Exposes a limited interface
- Enforces a specific access pattern

Container adapters are special containers that change or limit how we use existing containers.
They are specialized interfaces created on top of other sequential containers like deque, vector, or list by limiting functionality in a pre-existing container and providing more specific functionalities. It is done so as to avoid defining a completely new interface for containers that can be built on top of the already existing containers.

Stack in C++ STL

Stack container follows LIFO (Last In First Out) order of insertion and deletion. It means that most recently inserted element is removed first and the first inserted element will be removed last. This is done by inserting and deleting elements at only one end of the stack which is generally called the top of the stack.

```
// Inserting element top of the stack
  st.push(10);

// Only the top element of the stack can be accessed using top() method.

  cout << st.top();

// Deleting top element only top can be deleted

  st.pop();

  cout << "Size of stack: " << st.size() << endl;
```

Pseudo Traversal
A stack cannot be directly traversed, but by creating a copy and repeatedly accessing and popping the top element, we can traverse it without modifying the original stack.

```
// Create a copy

  stack<int> temp(st);
```

```
while (!temp.empty())

{

    cout << temp.top() << " ";

    temp.pop();

}
```

Queue in C++ STL

A queue is a container adapter that stores elements in FIFO (First In, First Out) order.

- The elements that are inserted first should be removed first.
- This is possible by inserting elements at one end (called back) and deleting them from the other end (called front) of the data structure.

```
  // Pushing elements into the queue
    q.push(3);

// Accessing the front and back elements

    cout << q.front() << endl;

    cout << q.back();

   // Deleting elements from the front of the queue

    q.pop();
```

- Since only the front and back element can be accessed in a queue, we cannot directly traverse it.

```
  // Create a copy

    queue<int> temp(q);

    while (!temp.empty())

    {

        cout << temp.front() << " ";

        temp.pop();

    }
```

Priority Queue in C++ STL

A Priority Queue adds and removes elements according to priority.

- Internally uses heap data structure.
- Uses a max heap by default, higher its value, higher its priority. But this can be changed to any desired priority scheme by providing a custom comparison.

A **Heap** is a complete binary tree data structure that satisfies the heap property: for every node, the value of its children is greater than or equal to its own value. Heaps are usually used to implement priority queues, where the smallest (or largest) element is always at the root of the tree

```
#include <iostream>

#include <queue>

using namespace std;

int main(){


    // Create a max-heap priority queue (default)

    priority_queue<int> pq;

    pq.push(30);

    pq.push(10);

    pq.push(20);

    pq.push(40);

    cout << "Elements removed from priority queue in order:\n";

    while (!pq.empty()){

        cout << pq.top() << " ";

        pq.pop();

    }

    return 0;

}//output<<40 30 20 10
```

Algorithms

Algorithms are ready-made functions that help you perform common operations like searching, sorting, counting, and comparing on containers (like arrays, vectors, sets, etc.). These algorithms are defined in the **<algorithm>** and **<numeric>** header files.

Type of Algorithms

Searching algorithms are methods used to find a specific element or information in a data structure like an array, vector, or list.They help locate an element efficiently without manually checking each item.

std::find() is a standard algorithm provided by the C++ Standard Template Library (STL). It is used to find the first occurrence of a given value in a specified range. std::find() does not use binary search; it still scans elements one by one. function defined inside **<algorithm>** header file.

```
vector<int> v = {1, 3, 6, 2, 9};

auto it = find(v.begin(), v.end(), 6);
```

std :: find_if

This function returns an iterator to the first element in the range [first, last) for which pred(function) returns true. If no such element is found, the function returns last.

```
#include <bits/stdc++.h>

using namespace std;

// Returns true if argument is odd

bool IsOdd(int i) { return i % 2; }

// Driver code

int main()

{

    vector<int> vec{ 10, 25, 40, 55 };

    // Iterator to store the position of element found

    vector<int>::iterator it;

    // std::find_if

    it = find_if(vec.begin(), vec.end(), IsOdd);

    cout << "The first odd value is " << *it << '\n'; //output is 25

    return 0;

}
```

std :: find_if_not

This function returns an iterator to the first element in the range [first, last) for which pred(Unary Function) returns **false**. If no such element is found, the function returns last. Means output of above code when we use this function is 10.

Binary Search is a popular searching algorithm which is used for finding the position of any given element in a **sorted array.**

```
#include <bits/stdc++.h>

using namespace std;

int main() {

    int arr[] = {1, 4, 5, 7, 9};

    int n = sizeof(arr) / sizeof(arr[0]);

    int k = 7;
```

```
    // Check if the element exists

    if (binary_search(arr, arr + n, k))

        cout << k << " is Present";

    else

        cout << k << " is NOT Present";

    return 0;

}
```

std::lower_bound is a Standard Template Library (STL) algorithm used to find the first position where a given value can be inserted in a sorted range without violating the order. In simple terms, it returns an iterator pointing to the first element that is greater than or equal to (>=) the given value.

```
  vector<int> v = {10, 20, 30, 40, 50};

    auto it = lower_bound(v.begin(), v.end(), 30);

    cout << *it; // print 30
```

Example:

```
#include <algorithm>

#include <iostream>

#include <vector>

using namespace std;

int main()

{

    vector<int> v = {10, 20, 30, 40, 50};

    int val = 35;

    // Finding lower and upper bound of val in v

    auto lb = lower_bound(v.begin(), v.end(), val);

    // Finding the number of smaller elements

    cout << "No. of Smaller Elements: " << lb - v.begin() << endl;//3

    // Finding the number of larger elements

    cout << "No. of Larger Elements: " << v.size() - (lb - v.begin());//2

    return 0;

}
```

lower_bound() finds the position where 35 would be inserted in the sorted vector; elements before it are smaller, and the remaining elements are larger.

```
vector<int> v = {10, 20, 30, 40, 50};

    int val = 35;

    // Finding lower and upper bound of val in v

    auto lb = lower_bound(v.begin(), v.end(), val);

    // Inserting 35 to its correct position

    v.insert(lb, val); //10 20 30 35 40 50
```

Both lower_bound and upper_bound use **binary search** internally. Use O(log n) and space complexity is O(1)

upper_bound is a Standard Template Library (STL) function used to find the first element that is strictly greater than a given value in a sorted range. It is commonly used for efficient searching and range queries in sorted containers.

```
  vector<int> v = {10, 20, 30, 40, 50};

    // Finding upper bound for value 30 in vector v

    cout << *upper_bound(v.begin(), v.end(), 30);
```

Example: //Count Elements Smaller and Larger than a Value

```
#include <algorithm>

#include <iostream>

#include <vector>

using namespace std;

int main()

{

    vector<int> v = {10, 20, 30, 40, 50};

    int val = 30;

    // Finding the upper bound of val in v

    auto ub = upper_bound(v.begin(), v.end(), val);

    // Finding the number of smaller elements

    cout << "No. of Smaller Elements: " << ub - v.begin() << endl;

    // Finding the number of larger elements
```

```
    cout << "No. of Larger Elements: " << v.end() - ub;

    return 0;

}
```

Finding Upper Bound in a Set

It is recommended to use set::upper_bound() for set as sets do not have random access to its elements. So, the upper_bound() function have to increment it sequentially to find the middle element (part of binary search algorithm) each time leading to increased time complexity.

```
#include<iostream>
#include <set>
#include <algorithm>
using namespace std;

int main(){

    set<int> s = {1, 2, 4, 4, 6, 8};

    // Recommended: O(log n)
    auto it1 = s.upper_bound(4);

    if (it1 != s.end())
        cout << "set::upper_bound -> " << *it1 << endl;
    else
        cout << "set::upper_bound -> Not found" << endl;

    // Not recommended: O(n) for set
    auto it2 = upper_bound(s.begin(), s.end(), 4);

    if (it2 != s.end())
        cout << "std::upper_bound -> " << *it2 << endl;
    else
        cout << "std::upper_bound -> Not found" << endl;

    return 0;
}
```

count() is a built-in function used to find the number of occurrences of an element in the given range. This range can be any STL container or an array.

```
vector<int> v = {2, 3, 2, 1, 5, 4, 2};

    // Count the occurrence of 2

    cout << count(v.begin(), v.end(), 2); // 3
```

count_if() function returns the number of elements in a range that satisfy the condition.

```
Input: 0 1 2 3 4 5 6 7 8 9

Output: Total no of even numbers is: 5
```

2. Sorting and Rearranging Algorithms

sort() function is used to sort elements in a container or array. Default sort is ascending order. The algorithm initially uses Quick Sort because it is fast and efficient for average cases.

```
sort(v.begin(), v.end());

 // Sort array in descending order

   sort(arr, arr + n, greater<int>());
```

is_sorted() : Checks if the given range is already sorted in ascending order (or by custom comparator).
is_sorted_until() : Returns an iterator to the first element where the sorted order breaks.
partial_sort() : Sorts only the first N elements in order, leaving the rest unsorted.

3.Manipulation Algorithms

copy(strt_iter1, end_iter1, strt_iter2) : The generic copy function used to **copy** a **range** of elements from one container to another. It takes 3 arguments:

copy_n(strt_iter1, num, strt_iter2) : This version of copy gives the freedom to choose how many elements have to be copied in the destination container. IT also takes 3 arguments:

```
#include<iostream>
#include<algorithm> // for copy() and copy_n()
#include<vector>
using namespace std;
int main()
{
  // initializing source vector
  vector<int> v1 = { 1, 5, 7, 3, 8, 3 };
  // declaring destination vectors
  vector<int> v2(6);
  vector<int> v3(6);
  // using copy() to copy 1st 3 elements
  copy(v1.begin(), v1.begin()+3, v2.begin());
  // printing new vector
  cout << "The new vector elements entered using copy() : ";
  for(int i=0; i<v2.size(); i++)
  cout << v2[i] << " ";
```

```
    cout << endl;


    // using copy_n() to copy 1st 4 elements
    copy_n(v1.begin(), 4, v3.begin());
    // printing new vector
    cout << "The new vector elements entered using copy_n() : ";
    for(int i=0; i<v3.size(); i++)
    cout << v3[i] << " ";
}
```

OUTPUT:

```
The new vector elements entered using copy() : 1 5 7 0 0 0
The new vector elements entered using copy_n() : 1 5 7 3 0 0
```

copy_if(): As the name suggests, this function copies according to the result of a "**condition**".This is provided with the help of a **4th argument**, a **function returning a boolean value**

```
// using copy_if() to copy odd elements
    copy_if(v1.begin(), v1.end(), v2.begin(), [](int i){return i%2!=0;}); //1 5 3 3 0 0
```

copy_backward(): This function starts copying elements into the destination container **from backward** and keeps on copying till all numbers are not copied. The copying starts from the "**strt_iter2**" but in the backward direction. It also takes similar arguments as copy().

```
#include<iostream>
#include<algorithm> // for copy_if() and copy_backward()
#include<vector>
using namespace std;
int main()
{
    // initializing source vector
    vector<int> v1 = { 1, 5, 6, 3, 8, 3 };
    // declaring destination vectors
    vector<int> v2(6);
    vector<int> v3(6);
    // using copy_if() to copy odd elements
    copy_if(v1.begin(), v1.end(), v2.begin(), [](int i){return i%2!=0;});
```

```
    // printing new vector
    cout << "The new vector elements entered using copy_if() : ";
    for(int i=0; i<v2.size(); i++)
    cout << v2[i] << " ";
    cout << endl;
    // using copy_backward() to copy 1st 4 elements
    // ending at second last position
    copy_backward(v1.begin(), v1.begin() + 4, v3.begin()+ 5);
    // printing new vector
    cout << "The new vector elements entered using copy_backward() : ";
    for(int i=0; i<v3.size(); i++)
    cout << v3[i] << " ";
}
```

Output:

```
The new vector elements entered using copy_if() : 1 5 3 3 0 0
The new vector elements entered using copy_backward() : 0 1 5 6 3 0
```

std :: move

Moves the elements in the range [first,last] into the range beginning at the result. The value of the elements in the [first,last] is transferred to the elements pointed out by the result. After the call, the elements in the range [first,last] are left in an unspecified but valid state.

```
std :: vector <int> vec1 {1, 2, 3, 4, 5};
    std :: vector <int> vec2 {7, 7, 7, 7, 7};
// move first 4 element from vec1 to starting position of vec2
    std :: move (vec1.begin(), vec1.begin() + 4, vec2.begin() + 1);
```

output:

```
Vector2 contains after std::move function: 7 1 2 3 4
Vector1 contains : 1 2 3 4 5
```

swap() is a built-in function in the C++ STL which swaps the value of two variables. This function supports almost every data type available in C++, whether it is a primitive type such as int, char, etc. or an STL containers such as vector, map, etc.

```
 int a = 1, b = 55;
```

```
  swap(a, b);
```

The swap function just swaps the internal address of the allocated memory in the vector, so, the process only requires constant time and space.

```
vector<int> v1 = {1, 2, 3, 4};

    vector<int> v2 = {6, 7, 8, 9};

swap(v1, v2);    // Swapping the vectors

int arr1[] = {1, 2, 3, 4};

    int arr2[] = {6, 7, 8, 9};

    // Swapping the arr ///size must be same

    swap(arr1, arr2);
```

Both the arrays should be of the same size and type for **swap()** to work. Also, arrays' address cannot be swapped, so each element is swapped one by one leading to the linear time complexity.

swap() for Custom Class
Your custom class can also have a specialized swap function which in turn uses the inbuilt **std::swap()**. It helps your class in working seamlessly with STL algorithms that uses swap operation.

```
#include <iostream>

using namespace std;

class GfG {

    int a, b;

public:

    GfG(int x, int y) : a(x), b(y) {}

    // Custom swap function for GfG class

    friend void swap(GfG& first, GfG& second) noexcept {

        // for ADL

        using std::swap;

        swap(first.a, second.a);

        swap(first.b, second.b);

    }

    void print() {

        cout << a << " " << b << endl;
```

```
    }
};
int main() {
    GfG gfg1(1 ,22);
    GfG gfg2(99, 8);
    // Swapping values
    swap(gfg1, gfg2);
    gfg1.print();
    gfg2.print();
    return 0;
}
```

std::replace

Assigns new_value to all the elements in the range [first, last) that compare to old_value. The function use operator == to compare the individual elements to old_value

```
#include <bits/stdc++.h>
using namespace std;
// Driver code
int main()
{
    int arr[] = { 10, 20, 30, 30, 20, 10, 10, 20 };
    int n = sizeof(arr) / sizeof(arr[0]);
    // variable containing the old and new values
    int old_val = 20, new_val = 99;
    // print old array
    cout << "Original Array:";
    for (int i = 0; i < n; i++)
        cout << ' ' << arr[i];
    cout << '\n';
```

```
    // Function used to replace the values
    replace(arr, arr + n, old_val, new_val);
    // new array after using std::replace
    cout << "New Array:";
    for (int i = 0; i < n; i++)
        cout << ' ' << arr[i];
    cout << '\n';
    return 0;
}
```

Output:

```
Original Array: 10 20 30 30 20 10 10 20
New Array: 10 99 30 30 99 10 10 99
```

std::replace_if

Assigns new_value to all the elements in range [first, last) for which **pred** returns true.

```
bool IsOdd(int i)
{
  return ((i % 2) == 1);
}
    int arr[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
replace_if(arr, arr + n, IsOdd, new_val); // 0 2 0 4 0 6 0 8 0 10
```

remove() : moves all elements not equal to a given value to the front of the range and returns an iterator to the new logical end (does not actually reduce container size).

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    vector<int> v = {1, 2, 3, 4, 3, 5};
    // Remove all occurrences of 3 from vector v
    auto ne = remove(v.begin(), v.end(), 3);
  cout<<*ne<<endl;
```

```
        v.erase(ne, v.end());


    for (auto i: v)
        cout << i << " ";
    return 0;
} //3 // it prits whatever at index 3.
1 2 4 5 3 5
```

std::fill is a C++ STL algorithm defined in the <algorithm> header that assigns a specified value to every element in a given range [first, last). It works with arrays and STL containers that provide at least forward iterators (such as vector, list, and deque).

```
vector<int> v(8);
    // Fill elements from index 2 to index 6 with value 4
    fill(v.begin() + 2, v.end() - 1, 4); // 0 0 4 4 4 4 4 0
```

transform() is a built-in STL function used to apply the given operation to a range of elements and store the result in another range.

```
vector<int> v1 = {1, 5, 6, 8};
    vector<int> v2(v1.size());
    // Incrementing all elements of vector by 1
    transform(v1.begin(), v1.end(), v2.begin(),
            [](int a) {
              return a + 1;
            });// 2 6 7 9
```

transform*(first, last, res, op1); // For Unary Operation*
transform*(first, last, first1, res, op2); // For Binary Operation*

- **first:** Iterator to the first element of the first input range.
- **last:** Iterator to the element just after the last element of the first input range.
- **first1**: Iterator to the first element of second input range.
- **res**: Iterator to the beginning of the output range.
- **op1**: A unary operation to be performed.
- **op2**: A binary operation to be performed.
- Returns an iterator to element just after the last element of output range.

```
#include <bits/stdc++.h>

using namespace std;
```

```
int main() {
    vector<int> v1 = {6, 7, 8, 9, 10};
    vector<int> v2 = {1, 4, 8, 9, 43};
    vector<int> v(v1.size());
    // Adding all elements of vector v1 and v2
    transform(v1.begin(), v1.end(), v2.begin(),
              v.begin(),
              [](int a, int b) {
                return a + b;
              });
    for (auto i : v)
        cout << i << " ";
    return 0;
}// 7 11 16 18 53
```

std::generate, as the name suggests is an STL algorithm, which is used to generate numbers based upon a generator function, and then, it assigns those values to the elements in the container in the range [first, last). The generator function has to be defined by the user, and it is called successively for assigning the numbers.
void generate (ForwardIterator first, ForwardIterator last, Generator gen);

```
#include <iostream>
#include <vector>
#include <algorithm>
// Defining the generator function
int gen()
{
static int i = 0;
return ++i;
}
using namespace std;
int main()
{
int i;
```

```
// Declaring a vector of size 10

vector<int> v1(10);

// using std::generate

std::generate(v1.begin(), v1.end(), gen);

vector<int>::iterator i1;

for (i1 = v1.begin(); i1 != v1.end(); ++i1) {

cout << *i1 << " ";

}

return 0;

} //1 2 3 4 5 6 7 8 9 10
```

lexicographical_compare() : Checks if one range is lexicographically less than another (like dictionary order).

```
template
  bool lexicographical_compare(iter1 beg1, iter1 end1,
                               iter2 beg2, iter2 end2)
{
  while (beg1!=end1)
  {
    if (beg2==end2 || *beg2<*beg1) return false;
    else if (*beg1<*beg2) return true;
    ++beg1; ++beg2;
  }
  return (beg2!=end2);
}
```

Returns a boolean true, if range1 is strictly lexicographically smaller than range2 else returns a false.

std::is_permutation in C++ STL

std::is_permutation checks whether one sequence is a permutation of another, i.e., whether both sequences contain the same elements in any order. It uses the == operator (or a custom predicate) for comparison. This function was introduced in C++11.

```
#include <iostream>

#include <algorithm>

using namespace std;

int main() {

    int A[] = {1, 7, 0, 2};

    int B[] = {0, 7, 2, 1};

    if (is_permutation(A, A+4, B))

        cout << "B is a permutation of A";
```

```
    else
        cout << "B is not a permutation of A";
    return 0;
}//B is a permutation of A
```

All elements in A exist in B in any order, so the function returns true.

Iterators in C++

Iterators are the pointer like objects that are used to point to the memory addresses of STL containers.

We can use iterators to move through the contents of the container.

```
vector<int>::iterator it = v.begin();
```

File Handling in C

File handling in C is the process in which we create, open, read, write, and close operations on a file. C language provides different functions such as **fopen()**, **fwrite()**, **fread()**, **fseek()**, **fprintf()**, etc. to perform input, output, and many different C file operations in our program.

Open a File in C

The function returns a file pointer (FILE *) which is used to perform further operations on the file, such as reading from or writing to it.

```
FILE* fopen(*file_name, *access_mode);
```

Example:

```
#include <stdio.h>
#include <stdlib.h>

int main(){
    FILE* fptr;

    // Creates a file "demo_file"
    // with file access as write mode
    fptr = fopen("demo_file.txt", "w+");

            //
    fprintf(fptr, "%s", "GeeksforGeeks");
    fclose(fptr);

    return 0;
}
```

Opening Modes	Description
r	Searches file. ***Opens the file for reading only***. If the file is opened successfully fopen() loads it into memory and sets up a pointer that points to the first character in it. If the file cannot be opened fopen() returns NULL.
w	Searches file. If the file exists already, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file. ***It creates a new file for writing only(no reading).***
a	Searches file. If the file is opened successfully fopen() loads it into memory and sets up a pointer that points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file. ***The file is opened only for appending(writing at the end of the file).***
r+	Searches file. ***Opens the file for both reading and writing***. If opened successfully, fopen() loads it into memory and sets up a pointer that points to the first character in it. Returns NULL, if unable to open the file.
w+	Searches file. If the file exists, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file. ***The difference between w and w+ is that we can also read the file created using w+.***
a+	Searches file. If the file is opened successfully fopen( ) loads it into memory and sets up a pointer that points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open the file. **The file is opened for reading and appending(writing at the end of the file).**
rb	Open the **binary file in read mode.** If the file does not exist, the open() function returns NULL.
wb	Open the **binary file in write mode.** As the pointer is set to the start of the file, the **contents are overwritten.** If the file does not exist, a **new file is created.**
ab	Open the **binary file in append mode.** The file pointer is set **after the last character in the file**. A **new file is created** if no file exists with the name.
rb+	Open the **binary file in read and write mode.** If the file does not exist, the open() function returns NULL.
wb+	Open the **binary file in read and write mode.** Contents are overwritten if the file exists. It will be created if the file does not exist.
ab+	Open the **binary file in read and append mode.** A file will be created if the file does not exist.

Function	Output goes to	Formatting?	Auto newline?
puts	stdout	✗ No	✓ Yes
fputs	file/stream	✗ No	✗ No
printf	stdout	✓ Yes	✗ No
fprintf	file/stream	✓ Yes	✗ No

fputs(str, fp);

fputc()

fputc() is used to write a single character at a time to a given file. It writes the given character at the position denoted by the file pointer and then advances the file pointer.
This function returns the character that is written in case of successful write operation else in case of error EOF is returned.

```
int fputc(int char, FILE *pointer)
char:  character to be written.
This is passed as its int promotion.
pointer: pointer to a FILE object that identifies the
stream where the character is to be written.
```

```
#include<stdio.h>

int main()

{

   int i = 0;

   FILE *fp = fopen("output.txt","w");

   // Return if could not open file

   if (fp == NULL)

     return 0;

   char string[] = "good bye", received_string[20];

   for (i = 0; string[i]!='\0'; i++)

      // Input string into the file

      // single character at a time

      fputc(string[i], fp);

   fclose(fp);

   fp = fopen("output.txt","r");

   // Reading the string from file
```

```
    fgets(received_string,20,fp);

    printf("%s", received_string);

    fclose(fp);

    return 0;

}
```

scanf() and fscanf() in C

scanf() function is used to read formatted input from stdin. It returns the whole number of characters written in it otherwise, returns a negative value.

```
#include <stdio.h>
int main()
{
    int a;
    scanf("This is the value %d", &a);
    printf("Input value read : a = %d", a);
    return 0;
}
// Input  : This is the value 100
    scanf("%*s %d", &a); //The %*s in scanf is used to ignore some input as required. In this case, it ignores the input until the next space or newline
scanf("%d", &a); //General purpose of use
```

fscanf Function in C

```
int fscanf(FILE *ptr, const char *format, ...)
```

Return Value: It returns zero or EOF, if unsuccessful. Otherwise, it returns the number of items successfully assigned.
Time Complexity: O(n)

Auxiliary Space: O(n) where n is the length of the input.

```
#include <stdio.h>

// Driver Code

int main()

{

    FILE* ptr = fopen("abc.txt", "r");

    if (ptr == NULL) {

        printf("no such file.");

        return 0;

    }

    /* Assuming that abc.txt has content in below

       format

       NAME    AGE   CITY

       abc     12    hyderabad
```

```
    bef    25    delhi
    cce    65    bangalore */
  char buf[100];
  while (fscanf(ptr, "%*s %*s %s ", buf) == 1)
     printf("%s\n", buf);
  return 0;
}
```

fgets() in C

It stores the input into a character array and stops reading when it reaches a newline character, the specified number of characters, or end-of-file (EOF).

It is safer than gets() because it prevents buffer overflow by limiting the number of characters read.
We can use it to read a full line (including whitespace) from input, rather than stopping at the first space like scanf("%s", …) would.

gets() reads a line from stdin into a buffer until it sees a newline or EOF.

✕ The big problem
gets() does not know the buffer size.

gets is removed from the C standard and should never be used.

☞ buffer overflow occurs
☞ memory corruption
☞ crashes or security vulnerabilities

fgets() reads at most n-1 characters from a stream and adds a null terminator.

```
#include<stdio.h>
#include <string.h>

int main() {
  char name[100];
  FILE* fp = fopen("abc.txt","r");
  if(fp){
       while(fgets(name, sizeof(name),fp)!=NULL){
            name[strcspn(name,"\n")] = '\0';
          printf("Hello %s\n", name);
       }
  fclose(fp);
  }else{
```

```
   fgets(name, sizeof(name), stdin);
   }
  //  printf("Hello %s\n", name);
}
//name[strcspn(name, "\n")] = '\0';//we replace \n with \0.//if(name[i]=='\n') name[i]='\0';
```

fgetc()

fgetc() is used to obtain input from a file single character at a time. This function returns the ASCII code of the character read by the function. It returns the character present at position indicated by file pointer. After reading the character, the file pointer is advanced to next character. If pointer is at end of file or if an error occurs EOF file is returned by this function.

```
#include <stdio.h>
int main ()
{
   // open the file
   FILE *fp = fopen("test.txt","r");
   // Return if could not open file
   if (fp == NULL)
     return 0;
   do
   {
      // Taking input single character at a time
      char c = fgetc(fp);
      // Checking for end of file
      if (feof(fp))
         break ;
      printf("%c", c);
   }  while(1);
   fclose(fp);
   return(0);
}
```

C fread() Function

The C fread() is a standard library function used to read the given amount of data from a file stream. Defined inside <stdio.h>, the fread() function reads the given number of elements of specific size from the file stream and stores it in the buffer memory. The total number of bytes read by fread() function is the number of elements read multiplied by the size of each element in bytes.

size_t fread(void * buffer, size_t size, size_t count, FILE * stream);

buffer: It refers to the pointer to the buffer memory block where the data read will be stored.

size: It refers to the size of each element in bytes.
count: It refers to the count of elements to be read.
stream: It refers to the pointer to the file stream.

Return Value

The function returns the number of elements that are read successfully from the file.
If the return value is less than the count, it means that an error occurred or it has reached the end of the file.
If the value of size or count is zero, fread() returns zero and performs no other action.

```
#include <stdio.h>

int main(){

   // File pointer

   FILE *filePointer;

   // Buffer to store the read data

   char buffer[10];

   size_t bytesRead;

   // "Gfg.txt" file is opened in read mode

   filePointer = fopen("test.txt", "r");

   // Ensure the file was opened successfully

   if (!filePointer){

      printf("Error opening file.\n");

      return 1;

   }

   // Data is read from the file into the buffer

   while ((bytesRead = fread(buffer, 1,sizeof(buffer) - 1, filePointer)) > 0){

     // Null-terminate the buffer

      buffer[bytesRead] = '\0';

      // Print the read data

      printf("%s", buffer);
```

```
    }

    fclose(filePointer);

    return 0;

}
```

fwrite() in C

fwrite() is a built-in function used to write a block of data from the program into a file. It can write arrays, structs, or other data to files but is especially designed to write binary data in binary files.

```
#include<stdio.h>

#include <string.h>

int main()

{

  char word[]="omkar sir";

  word[sizeof(word)]='\0';

  FILE* fp= fopen("demo1.txt","w");

  if(fp)

  {

        printf("sizeof(word) =%ld\n",sizeof(word));

        printf("len= %ld\n",strlen(word));

        printf("%c\n",word[sizeof(word)]);

        fwrite(word,1,strlen(word),fp);

        fclose(fp);

  }

}
```

Move File Pointer

File pointer generally points to the position according to the mode or last read/write operation. We can manually move this pointer to any position in the file using fseek() function.

fseek() in C

The fseek() function is used to move the file pointer associated with a given file to a specific location in the file. It provides random access capabilities, which allow the programmer to randomly move the file pointer to any location in the file to read or write data.

int fseek(FILE *pointer, long int offset, int position);
pointer: It is the pointer to a FILE object that identifies the stream.

- offset: It is the number of bytes to offset from the position
- position: It is the position from where the offset is added. Position defines the point with respect to which the file pointer needs to be moved. It has three values:
    - SEEK_END: It denotes the end of the file.
    - SEEK_SET: It denotes starting of the file.
    - SEEK_CUR: It denotes the file pointer's current position.

It returns zero if successful, or else it returns a non-zero value.

Ftell - It is used to return the current position of a file pointer.

```
#include <stdio.h>

int main()

{

   FILE* fp;

   fp = fopen("test.txt", "r");

   // Moving pointer to end

   fseek(fp, 0, SEEK_END);

   // Printing position of pointer

   printf("%ld", ftell(fp));

   return 0;

}
```

Now, imagine you want to read this file after writing. We can use fseek() here too, but these is one more function specifically for this purpose which is rewind().

```
#include <stdio.h>

int main() {

   FILE *fptr = fopen("file.txt", "w+");

   // Write data to file

   fprintf(fptr, "Hello, GFG!");

   // Rewind fptr to the beginning

   rewind(fptr);

   // Read data from file
```

```
    char ch;
    while ((ch = fgetc(fptr)) != EOF) {
        printf("%c", ch);
    }
    fclose(fptr);
    return 0;
}//rewind() only affects the read position
```

Read and Write in a Binary File

We use fwrite() function to write data to a binary file. The data is written to the binary file in the form of bits (0's and 1's).
To open a file in binary mode, we use the rb, rb+, ab, ab+, wb, and wb+ access mode in the fopen() function

```
#include <stdio.h>
#include <stdlib.h>
struct threeNum {
    int n1, n2, n3;
};
int main() {
    int n =1 ;
    // Structure variable declared.
    struct threeNum num;
    FILE* fptr;
    fptr = fopen("binaryFile.bin", "wb");
    int flag = 0;
    num.n1 = n;
    num.n2 = 5 * n;
    num.n3 = 5 * n + 1;
    // Write the Structure data
    // to binary file.
    flag = fwrite(&num, sizeof(struct threeNum), 1,
                  fptr);
    // Checking if the data is written.
    if (!flag)
```

```
        printf("Write Operation Failure");
    else
        printf("Write Operation Successful");
    fclose(fptr);
    return 0;
}
```

Reading from Binary File

```
#include <stdio.h>
#include <stdlib.h>

// Structure that store
// binary file data
struct threeNum {
    int n1, n2, n3;
};

int main() {
    int n;
    struct threeNum num;
    FILE* fptr;
    fptr = fopen("binaryFile.bin", "rb");

    // Read the data from binary
    // file and print that data
    fread(&num, sizeof(struct threeNum), 1, fptr);
    printf("n1: %d\tn2: %d\tn3: %d\n", num.n1, num.n2,
           num.n3);
    fclose(fptr);
    return 0;
}// n1: 1   n2: 5   n3: 6
```

File Handling in C++

Programs run in RAM, meaning data exists only while the program is running, when a program ends, all data in RAM is lost automatically.

File handling allows storing data in secondary memory (like HDD or SSD), so it can be preserved permanently and can be saved and accessed even after the program terminates.

Before reading from or writing to a file, we first need to open it. Opening a file loads that file in the RAM. In C++, we open a file by creating a stream to it using the fstream class that represent the file stream i.e. stream for input and output to the file.

In C++, a stream refers to a sequence of characters that are transferred between the program and input/output (I/O) devices.

```
fstream str("filename.ext", mode);
```

File Opening Modes

Mode	Description
ios::in	File open for reading. If file does not exists, the open operation fails.
ios::out	File open for writing: the internal stream buffer supports output operations.
ios::binary	Operations are performed in binary mode rather than text.
ios::ate	The output position starts at the end of the file.
ios::app	All output operations happen at the end of the file, appending to its existing contents.
ios::trunc	Any contents that existed in the file before it is open are discarded.

If the file opened in write mode does not exists, a new file is created. But if the file opened in read mode doesn't exists, then no new file is created, and an exception is thrown.

These modes can also be combined using OR operator (|). For example, you can open the file stream in both read and write mode as shown:

```
fstream str("file.txt", ios::in | ios::out);
```

1. ios::in This mode opens the file for reading. The file must exist.

```
#include<iostream>
#include<fstream>
using namespace std;
int main()
{
fstream file("example.txt",ios::in);
if(file.is_open())
{
        std::string line;
        while(getline(file,line))
        {
        cout<<line<<endl;
        }`
}
else
{
cout<<"file is not open\n";
}
return 0;
}
```

2.ios_out This mode opens the file for writing. If the file exists, it is truncated (erased) to zero length. If it doesn't exist, it is created.

```
#include<iostream>
#include<fstream>
using namespace std;
int main()
{
        fstream file("example.txt",ios::out);
        if(file.is_open())
        {
                file << "Hii i am Omkar. and i am developer";
                file.close();
```

```
	}
	else
	{
		cout<<"File is open\n";
	}
	return 0;
}
```

3.ios_app This mode opens the file and appends new content to the end of the file without altering the existing data.

```
#include<iostream>
#include<fstream>
using namespace std;

int main()
{
	fstream file("example.txt",ios::app);
	if(file.is_open())
	{
		file << "\nI am Good boy.";
		file.close();
	}
	else
	{
		cout<<"File is open\n";
	}
	return 0;
}
```

4.ios_ate This mode opens the file for reading or writing, and the file pointer is moved to the end of the file immediately after opening. but you can still move the pointer back to any position and write at that position (you have the ability to seek and overwrite). Without ios::in or ios::out this do nothing.

```
#include<iostream>
#include<fstream>
using namespace std;

int main()
{
fstream file("example.txt", ios::ate | ios::out );
if(file.is_open())
{
	file<<"Thank you god for everything";
	file.seekp(0);
	file<<"You are amazing";
	file.close();
}
}
```

5.ios_trunc This mode opens the file for writing. If the file exists, its contents are deleted (truncated). If it doesn't exist, the file is created. Without ios::in or ios::out this do nothing. And give unable to open file.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
```

```
    fstream file("example.txt", ios::trunc | ios::out   );

    if (file.is_open()) {
        file << "This will overwrite the entire file.";
        file.close();
        cout << "File content is overwritten." << endl;
    } else {
        cout << "Unable to open file." << endl;
    }

    return 0;
}
```

6. ios_binary This mode opens the file in binary mode. It is commonly used when working with binary data, like reading and writing structs, images, or raw data files. Without ios::in or ios::out this do nothing. And give unable to open file.

```
#include <iostream>
#include <fstream>
using namespace std;

struct Person {
    int age;
    char name[50];
};

int main() {
    Person p = {25, "John Doe"};

    // Writing binary data to a file
    fstream file("person.dat", ios::out |  ios::binary);
    if (file.is_open()) {
        file.write((char*)&p, sizeof(p));
        file.close();
        cout << "Binary data written to file." << endl;
    } else {
        cout << "Unable to open file for binary writing." << endl;
    }

    // Reading binary data from the file
    Person readPerson;
    file.open("person.dat", ios::in | ios::binary);
    if (file.is_open()) {
        file.read((char*)&readPerson, sizeof(readPerson));
        file.close();
        cout << "Binary data read from file:\n";
        cout << "Name: " << readPerson.name << ", Age: " << readPerson.age << endl;
    } else {
        cout << "Unable to open file for binary reading." << endl;
    }

    return 0;
}
```

Other File Streams

fstream is not the only file stream provided by C++. There are two more specialized streams:

- ifstream: Stands for input file stream. It is equivalent to open fstream in ios::in mode.
- ofstream: Stands for output file stream. It is equivalent to opening fstream in ios::out mode.

The above modes are default modes for these streams. These modes cannot be changed but can be clubbed together with other modes.
ifstream filein("file.txt");
ofstream fileout("file.txt");

my_file.is_open()

my_file.fail()

while (!my_file.eof()) { }

my_file.close();

file.flush();

Binary File Handling in C++

A **binary file** stores **raw bytes exactly as they exist in memory**. No formatting, no conversion to text.

```
#include<iostream>
#include<fstream>
using namespace std;

int main() {
    ofstream file;
    file.open("data.bin", ios::binary);
    if(file.fail())
    {
        cout<<"could not open file\n";
        return 1;
    }
    int data= 100;
    file.write((char*)&data,sizeof(data));
    file.close();
    return 0;
}
```

```
#include<iostream>
#include <fstream>
using namespace std;

int main() {
        ifstream in;
    in.open("data.bin", ios::binary);
    if(!in)
    {
        cout<<"could not open file\n";
        return -1;
    }
    int num;
    in.read((char*)&num,sizeof(num));
    cout<<num<<endl;
    in.close();
```

```
    return 0;
}
```

File Pointer Functions

Function	Meaning
`seekg()`	move read pointer
`seekp()`	move write pointer
`tellg()`	get read position
`tellp()`	get write position

1.tellg

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream file("file.txt");
    cout << file.tellg() << endl;
    char ch;
    file.get(ch);
    cout << ch << endl;
    cout << file.tellg() << endl;
}
```

2.tellp

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ofstream file("data.txt");
    file << "HELLO";
    cout << file.tellp();
}
```

3.seekg

file.seekg(position);

```
file.seekg(offset, reference);
```

`ios::beg`	beginning
`ios::cur`	current
`ios::end`	end

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main()
{
    ifstream file("data.txt");
    file.seekg(3);
    char ch;
    file.get(ch);
    cout << ch;
    file.seekg(-4,ios::end);
     char ch1;
    file.get(ch1);
    cout << ch1;
}
```

4.seekp

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    fstream file;
    file.open("data.txt",ios::in | ios::out);
    if (file.is_open())
    {
        file.seekp(3, ios::beg);
        file << "X";
        file.close();
    }
    return 0;
}
```

Finding File Size Using File Pointer

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream file("data.txt");
    file.seekg(0, ios::end);
    int size = file.tellg();
    cout << "File size: " << size << " bytes";
}
```

Move Semantics in C++

Move semantics are used in C++ to transfer the ownership of resources from one object to another instead of copying them. It improves

the performance as it avoid unnecessary copying of objects, reducing memory usage, improves efficiency, and efficiently handles the

temporary objects like rvalue.

Why Move Semantics are Needed?

Move semantics avoid copying of resources. It helps in avoiding duplicates and cause less memory overhead.

It handles temporary objects like rvalues.

It improves the performance by moving the resources instead of copying.

lvalue Expression

An lvalue is a type of expression in which object has a memory address and can be modified if it is not const. Example of lvalue can be a

variable, array elements, and many more.

An lvalue reference is used to create a reference to an lvalue. It is denoted by (&) and is used for implementing copy semantics.

rvalue Expression

An rvalue is a type of expression that does not have a memory address and it represents a value that generally appears on the right side. It

is a temporary expression that is about to get destroyed. Example of rvalue can be a constant, temporary object, and many more.

An rvalue reference is used to reference rvalues or temporary objects. It is denoted by (&&) and is used for implementing move

semantics.

```
#include <iostream>

using namespace std;

int main() {

// x is an lvalue

int x = 10;

// 'x + 5' is an rvalue

int y = x + 5;
```

```
cout << "Old x: " << x << endl;

cout << "Old y: " << y << endl;

// Demonstrating assignments

x = 20; // correct, as x is an lvalue

// (x + 5) = 15; // wrong, as x + 5 is an rvalue

cout << "New x: " << x << endl;

return 0;

}
```

example2 :

```
#include <iostream>

using namespace std;

void printValue(int &x){

cout << "Calling with Lvalue reference: " << x << endl; // lvalue reference

}

void printValue(int &&x){

cout << "Calling with Rvalue reference: " << x << endl; // rvalue reference

}

int main(){

int a = 10;

printValue(a); // a is an lvalue

printValue(20); // 20 is an rvalue

return 0;

}
```

Why do Move Semantics Apply to Rvalues Only?

The move semantics is applied only to rvalues because, the rvalues are temporary objects, that are about to get destroyed. The rvalue will not affect the program in future as they are temporary and can be changed. So, applying move semantics on rvalue to transfer the ownership won't affect the program.

Move Constructor

A move constructor is a special constructor used to transfer the ownership of resources from a temporary object or rvalue to a new

object using rvalue reference. The move constructor is automatically called when initializing an object with an rvalue.

Move Assignment Operator

The move assignment operator uses '(=)' and rvalue reference for move semantic. First, it releases the current resource of object and then

take ownership of the source object's resources.

```
#include <iostream>

#include <cstring>

using namespace std;

class MyString {

private:

char* data;

size_t length;

public:

// Regular constructor

MyString(const char* str) {

length = strlen(str);

data = new char[length + 1];

strcpy(data, str);

cout << "Constructor called\n";

}

// Move constructor

MyString(MyString&& other) noexcept {

data = other.data;

length = other.length;

other.data = nullptr;
```

```
other.length = 0;
cout << "Move constructor called\n";
}
// Move assignment operator
MyString& operator=(MyString&& other) noexcept {
if (this != &other) {
delete[] data;
data = other.data; // transferring ownership
length = other.length;
other.data = nullptr;
other.length = 0;
cout << "Move assignment operator called\n";
}
return *this;
}
// Destructor
~MyString() {
delete[] data;
cout << "Destructor called\n";
}

void print() const {
if (data)
cout << data << endl;
}
};
int main() {
MyString s1("Hello");
```

```
MyString s2("World");
cout << "Before move assignment:\n";
s1.print();
s2.print();
s1 = std::move(s2); // Move assignment operator called
cout << "After move assignment:\n";
s1.print();
s2.print(); // s2 is now empty
return 0;
}
#include <iostream>
#include <string>
#include <utility> // for move
using namespace std;
int main(){
cout << "Before move() function:";
string str1 = "Hello";
cout << "\nstring 1: " << str1 << endl;
cout << "\nAfter move() function:";
string str2 = move(str1);
cout << "\nstr2: " << str2;
cout << "\nstr1: " << str1 << "\n";
return 0;
}
```

exception handling in c

The C programming language does not have built-in support for exception

handling mechanism

Exception Handling in C++

Exception handling in C++ is a mechanism to detect and manage runtime errors (errors

that occur during program execution) in a structured way. Examples of runtime errors

are.

Division by zero

Accessing invalid memory

File I/O failures

1. try-catch Block

C++ provides an inbuilt feature for handling exceptions using try and catch block. It is an exception handling mechanism where the code that may cause an exception is placed inside the try block and the code that handles the exception is placed inside the catch block.

2. Throwing Exceptions

Throwing exception means returning some kind of value that represent the exception from the try block.

The matching catch block is found using the type of the thrown value. The throw keyword is used to

throw the exception.

There are three types of values that can be thrown as an exception:

Built-in Types

Standard Exceptions

Custom Exceptions

Throwing Built-in Types

```
#include <bits/stdc++.h>

using namespace std;

int main() {

int x = 7;
```

```
try {
if (x % 2 != 0) {
// Throwing int
throw -1;
}
}
// Catching int
catch (int e) {
cout << "Exception Caught: " << e;
}
return 0;
}
```

Throwing Standard Exceptions

Standard exceptions are the set of classes that represent different types of common exceptions. All these classes are defined inside <stdexcept> header file and mainly derived from std::exception class which act as the base class for inbuilt exceptions.

The what() method is present in every standard exception to provide information about the exception itself.

Throwing Custom Exceptions

When the standard exceptions cannot satisfy our requirement, we can create a custom exception class. It is recommended to inherit standard exception in this class to provide seamless integrity with library components though, it is not compulsory.

```
#include <iostream>
#include <exception>
using namespace std;

// Custom exception class
class NegativeValueException : public exception {
```

```
private:
int value;
public:
// Constructor
NegativeValueException(int val) : value(val) {}
// Override what() method
const char* what() const noexcept override {
return "Negative value error occurred!";
}
// Optional: method to get the invalid value
int getValue() const {
return value;
}
};
// Function that throws the custom exception
void checkValue(int x) {
if (x < 0) {
throw NegativeValueException(x);
}
else {
cout << "Value is: " << x << endl;
}
}
int main() {
int numbers[] = {10, -5, 20};
for (int n : numbers) {
try {
checkValue(n);
```

```
}
catch (NegativeValueException &e) {
cout << "Exception caught: " << e.what()
<< " Value = " << e.getValue() << endl;
}
}
return 0;
}
```

3. Catching Exceptions

As already told, the catch block is used to catch the exceptions thrown in the try block. The catch block takes one argument, which should be of the same type as the exception.

```
try {
// Code that might throw an exception
}
catch (type1 e) {
// executed when exception is of type1
}
catch (type2 e) {
// executed when exception is of type2
}
catch (...) {
// executed when no matching catch is found
}
```

```
#include<iostream>
using namespace std;
```

```
int main()
{
try{
int choice;
cout<<"Enter No "<<endl;
cin>>choice;

if(choice == 1)
{
throw invalid_argument("Invalid Argument.");
}
else if(choice == 2)
{
throw out_of_range("Out of Range.");
}
else
{
cout<<"Unknow Error "<<endl;
}
}
catch(invalid_argument e)
{
cout<<"caught Exception "<<e.what()<<endl;
}
catch (out_of_range e) {
cout << "Caught exception: " << e.what() << endl;
}
```

```
// executed when no matching catch is found
catch (...) {
cout << "Caught an unknown exception." << endl;
}
return 0;
}
```

How to Catch All Exceptions in C++?

To catch all kinds of exceptions in our catch block in C++, we can define the catch block using the catch-all clause - catch (...)

Catch by Value or Reference

1. Catch by Value: Catching exceptions by value creates a new copy of the thrown object in the catch block. Generally, the exceptions objects are not very large so there is not much overhead of creating copies.

2. Catch by Reference: Catch by reference method just pass the reference to the exception thrown instead of creating a copy. Although it reduces the copy overhead, it is not the primary advantage of this method. The main advantage of this method is in catching polymorphic exception types.

```
#include<iostream>
using namespace std;
int main()
{
int a=5, b=0;
try
{
if (b == 0)
throw runtime_error("Divide by zero exception");
int c= a/b;
cout<<c<<endl;
```

```
}
//catch(runtime_error e) //catch by value
catch(exception &e) //catch by reference
{
cout<<"exception Caught : "<< e.what() <<endl;
}
return 0;
}
```

Handle Uncaught Exceptions

If the exception is not caught by any of the catch block (in case catch all is not present), then by default, the exception handling subsystem aborts the execution of the program. But this behaviour can be changed by creating a custom terminate handler and setting it as default using set_terminate() function.

```
void myHandler()
{
cout<<"my handler\n";
abort();
}
int main()
{
set_terminate(myHandler);
try
{
cout<<"insdide try"<<endl;
throw 100;
}catch(char a)
{
```

```
cout<<"inside catch block\n";
}
}
```

Exception Propagation

Exception propagation tells us how the exception travels in the call stack.

When an exception is thrown, execution of the current block is immediately terminated and all the resources allocated are automatically deallocated (except dynamic resources allocated using new).

The stack unwinding occurs as the thrown exception propagates through the call stack to look for the matching catch block.

When the corresponding catch block is found, exception is caught and handled. If not caught, the program terminates

Stack Unwinding in C++

Stack unwinding is the process of removing function call frames from the call stack at runtime. During this process, local objects are destroyed in the reverse order of their creation.

Stack unwinding happens:

Normally, when a function returns.

During exception handling, when an exception is thrown and control transfers to a matching catch block.

When an exception is not handled in the current function, the runtime:

1. Exits the function.
2. Destroys all local objects.
3. Moves up the call stack searching for a matching catch block.
4. Terminates the program if no handler is found.

```
#include <iostream>
using namespace std;
```

```
// Function f1() that throws an exception
void f1() {
cout << "f1() Start\n";
throw 100;
cout << "f1() End\n"; // Not executed
}

// Function f2() that calls f1()
void f2() {
cout << "f2() Start\n";
f1();
cout << "f2() End\n"; // Not executed
}

// Function f3() that handles the exception
void f3() {
cout << "f3() Start\n";
try {
f2();
}
catch (int i) {
cout << "Caught Exception: " << i << "\n";
}
cout << "f3() End\n";
}

int main() {
f3();
```

return 0;

}

output:

f3() Start

f2() Start

f1() Start

Caught Exception: 100

f3() End

Destructors are called in reverse order of construction

Stack unwinding does not automatically free heap memory allocated using new

unless it is managed by an object whose destructor handles cleanup.

```
#include <bits/stdc++.h>
using namespace std;
void func() {
// Create a unique_ptr that manages dynamic memory
unique_ptr<int> data(new int(10));
cout << "Resource allocated: " << *data << endl;
// Throw the exception
throw runtime_error("An error occurred");
}

int main() {
try {
// Call the function that may throw an exception
func();
}
```

```
catch (const exception& e) {
cout << e.what();
}
return 0;
}
```

How to Safely Handle Resources During Stack Unwinding

Use RAII (Resource Acquisition Is Initialization).

Prefer smart pointers (unique_ptr, shared_ptr) over raw pointers.

Use standard containers (vector, map, set) instead of manual memory management.

Avoid new and delete in exception-prone code.

Let destructors handle cleanup automatically.

Nested Try Catch Blocks

```
#include <iostream>
using namespace std;
// function throwing exceptions
void func(int n)
{
if (n < 10) {
throw 22;
}
else {
throw 'c';
}
}

// driver code
int main()
```

```
{

try {

try {

cout << "Throwing exception from inner try "

"block\n";

func(2);

}

catch (int n) {

cout << "Inner Catch Block caught the exception"

<< endl;

}

}

catch (char c) {

cout << "Outer catch block caught the exception"

<< endl;

}

cout << "Out of the block";

return 0;

}
```

Rethrowing an Exception

Rethrowing an exception in C++ involves catching an exception within a try block and instead of dealing with it locally throwing it again to be caught by an outer catch block. By doing this. we preserve the type and details of the exception ensuring that it can be handled at the appropriate level within our program.

This approach becomes particularly valuable when managing exceptions at multiple levels or when

additional actions need to be performed before resolving the exception.

```
#include<iostream>

using namespace std;

int devide(int a, int b)
```

```
{
try{
if(b==0)
throw runtime_error("Division by zero");

return a/b;
}
catch(exception& e)
{
cout<<"inner exception caught "<<e.what()<<endl;
throw;
}
}
int Addition_positive(int a, int b)
{

if(a<0 || b<0)
{
throw runtime_error("Negative Number. ");
}
return a+b;
}
int main()
{
try{
Addition_positive(10, devide(20,0));
}
catch(exception& e)
```

```
{
cout<<"outer catch caught exception "<<e.what()<<endl;
}
return 0;
}
```

Exception Specification

C++ provides exception specifications to specify that the given function may or may not throw an exception. In modern C++, there are two types of exception specifications:

1. noexcept: Tells that the given function is guaranteed to not throw an exception.

2. noexcept(false): Tells that the given function may throw an exception. It is default behaviour in case no exception specification is used.

1. runtime_error

Error detected during program execution.

```
throw runtime_error("Runtime error occurred");
```

2. logic_error

Error in program logic (programmer mistake).

```
throw logic_error("Logic error");
```

invalid_argument

Used when a function argument is invalid.

```
throw invalid_argument("Invalid argument"); //stoi("abc");
```

domain_error

Mathematical domain error.

```
throw domain_error("Domain error");//sqrt(-1)
```

length_error

Object exceeds maximum allowed size.

```
throw length_error("Length error"); //very large string/vector.
```

out_of_range

Accessing element outside container bounds.

```
vector<int> v1={1,2,3};
try
{
v1.at(10);
}catch(out_of_range e)
{
cout<<"caught exception "<<e.what()<<endl;
}
```

std::exception

```
│
├── logic_error
│   ├── invalid_argument
│   ├── domain_error
│   ├── length_error
│   └── out_of_range
│
├── runtime_error
│   ├── range_error
│   ├── overflow_error
│   └── underflow_error
│
├── bad_alloc
├── bad_cast
├── bad_typeid
└── bad_exception
```

Dynamic Memory Allocation in C

Dynamic memory allocation allows a programmer to allocate, resize, and free memory at runtime. Memory is allocated on the heap area instead of stack.

malloc()

The **malloc()** (stands for **m**emory **alloc**ation) function is used to allocate a single block of contiguous memory on the heap at runtime. The memory allocated by malloc() is uninitialized, meaning it contains garbage values.
Assume that we want to create an array to store 5 integers. Since the size of int is 4 bytes, we need 5 * 4 bytes = 20 bytes of memory. This can be done as shown:

```
int *ptr = (int *)malloc(20);

int *ptr = (int *)malloc(sizeof(int) * 5);

void* malloc(size_t size);
```

calloc()

The **calloc()** (stands for **c**ontiguous **alloc**ation) function is similar to malloc(), but it initializes the allocated memory to zero. It is used when you need memory with default zero values.
This function also returns a void pointer to the allocated memory that is converted to the pointer of required type to be usable. If allocation fails, it returns NULL pointer.

```
void* calloc(size_t n, size_t size);
   int *ptr = (int *)calloc(5, sizeof(int));
```

free()

The memory allocated using functions malloc() and calloc() is not de-allocated on their own.
The **free()** function is used to release dynamically allocated memory back to the operating system. It is essential to free memory that is no longer needed to avoid memory leaks.

```
   free(ptr);
```

realloc()

realloc() function is used to resize a previously allocated memory block. It allows you to change the size of an existing memory allocation without needing to free the old memory and allocate a new block.
Suppose we initially allocate memory for 5 integers but later need to expand the array to hold 10 integers. We can use realloc() to resize the memory block:

```
  int *ptr = (int *)malloc(5 * sizeof(int));

   // Resize the memory block to hold 10 integers

   ptr = (int *)realloc(ptr, 10 * sizeof(int));
void *realloc(void *ptr, size_t size);
```

It is important to note that if realloc() fails and returns NULL, the original memory block is not freed, so you should not overwrite the original pointer until you've successfully allocated a new block

When you increase memory using realloc, **the existing data is preserved**. New part contains **garbage values**.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
```

```
int *ptr = calloc(3, sizeof(int));

ptr[0] = 10;
ptr[1] = 20;
ptr[2] = 30;

ptr = realloc(ptr, 6 * sizeof(int));

for(int i=0;i<6;i++)
printf("%d ", ptr[i]);
} //10 20 30 0 4196320 0
```

Namespaces in C++

A namespace is designed to overcome naming conflict difficulty and is used as additional information to differentiate similar functions, classes, variables etc.

```
namespace first_space {

void func() {

cout << "Inside first_space" << endl;

}

}

first_space::func();
```

The using directive

```
using namespace std;
```

The using directive can also be used to refer to a particular item within a namespace. For example, if the only part of the std namespace that you intend to use is cout, you can refer to it as follows –

```
#include <iostream>

using std::cout;

int main () {

cout << "std::endl is used with std!" << std::endl;

return 0;

}
```

Nested Namespaces

```
namespace namespace_name1 {

// code declarations

namespace namespace_name2 {
```

```
// code declarations

}

}

// to access members of namespace_name2

using namespace namespace_name1::namespace_name2;

// to access members of namespace:name1

using namespace namespace_name1;
```

C++ Templates

A generic class in C++ simply means a class written using templates so it can work with different data types without rewriting the class.

Templates are the foundation of generic programming, a programming style that allows writing functions, classes, algorithms, and different code snippets that work with different data types.

A template is a blueprint or formula for creating a generic class or a function. The library containers like iterators and algorithms are examples of generic programming and have been developed using template concepts.

There is a single definition of each container, such as vector, but we can define many different kinds of vectors, for example, vector <int> or vector <string>.The two keywords that are used to define a template are template and typename.

Function Template

Function templates define a blueprint for functions that enables a function to operate on different data types without rewriting the same logic.

```
template <typename identifier> function_declaration;

#include <iostream>

#include <string>

using namespace std;

template <typename T>

inline T const& Max (T const& a, T const& b) {

return a < b ? b:a;

}

int main () {

int i = 39;
```

int j = 20;

cout << "Max(i, j): " << Max(i, j) << endl;

double f1 = 13.5;

double f2 = 20.7;

cout << "Max(f1, f2): " << Max(f1, f2) << endl;

string s1 = "Hello";

string s2 = "World";

cout << "Max(s1, s2): " << Max(s1, s2) << endl;

return 0;

}

Class Template

Similarly, class templates also define a blueprint for creating classes that can work with any data type.

template <class type> class class-name { .};

```
#include<iostream>
#include<vector>
#include<stdexcept>
using namespace std;

template <class T>
class stack{
    private:
        vector<T> v;
    public:
        void push(T const& ele);
        void pop();
        T top () const;
        bool empty() const{
            return v.empty();
        }
};

template <class T>
void stack<T>::push(T const& ele){
    v.push_back(ele);

}
template <class T>
void stack<T>::pop()
{
```

```
    if(v.empty())
    {
        throw out_of_range("stack is empty.");
    }
    v.pop_back();
}

template <class T>
T stack<T>::top() const{
    if(v.empty())
    {
        throw out_of_range("stack is empty. out of range.");
    }
    return v.back();
}
int main() {
    try{
        //stack<int> s1;
                stack<string> s1;
        s1.push("omkar");
        s1.push("swappy");
        s1.push("Ankya");
        cout<<s1.top()<<endl;
        s1.pop();
        cout<<s1.top()<<endl;
        s1.pop();
        s1.pop();
        if(s1.empty())
        {
            cout<<"stack is empty"<<endl;
        }
        cout<<s1.top()<<endl;

    }catch(exception& e)
    {
        cout<<"exception caught: "<<e.what()<<endl;
    }

    return 0;
}
```

Template Argument Deduction

Template argument deduction is a feature that automatically deduces (understands) the data type of the argument passed to a function or class template. Instead of explicitly specifying template arguments, the compiler figures them out for you.

template<typename T>

T add(T a, T b) {

return a + b;

}

int main() {

// The compiler deduces T as int

auto result1 = add(5, 3);

// The compiler deduces T as double

auto result2 = add(3.14, 2.86);

}

C++ Signal Handling

A signal is a software interrupt sent to a process to notify it that some event occurred.

Signals are the interrupts delivered to a process by the operating system which can terminate a program prematurely.

You can generate interrupts by pressing Ctrl+C on a UNIX, LINUX, Mac OS X or Windows system.

These signals are defined in C++ header file <csignal>.

The raise() Function

You can generate signals by function raise().

```
#include<iostream>
#include <csignal>
using namespace std;
void signalHandler(int sig)
{
    cout<<"signal Handler "<<sig<<endl;
    exit(sig);

}
int main() {
    signal(SIGINT, signalHandler);
    int i =0;
    while(1)
    {
        printf("Hello omkar\n");
        if(i==3)
        {
            raise(SIGINT);
        }
        i++;
    }
    return 0;
}
```

Most Important Signals

1. SIGINT

Signal: SIGINT

Meaning:

- Interrupt from keyboard
- Triggered by **Ctrl + C**

Default action:

- Terminate process

2. SIGTERM

Signal: SIGTERM

Meaning:

- Termination request
- Sent by default when you run:

kill PID

Default action:

- Terminate process

Programs should **handle SIGTERM** to shutdown cleanly.

3. SIGKILL

Signal: SIGKILL

Meaning:

- Immediate process termination.

Example:

kill -9 PID

Important facts:

- **Cannot be caught**
- **Cannot be ignored**
- **Cannot be handled**

Kernel kills process instantly.

4. SIGSTOP

Signal: SIGSTOP

Meaning:

- Pause process execution.

Example:

kill -STOP PID

Properties:

- Cannot be caught
- Cannot be ignored

Resume using:

kill -CONT PID

5. SIGCONT

Signal: `SIGCONT`

Meaning:

- Resume stopped process.

Example:

```
kill -CONT PID
```

6. SIGSEGV

Signal: SIGSEGV

Meaning:

- Segmentation fault.

Occurs when:

- Invalid memory access
- NULL pointer dereference
- Buffer overflow

Example:

```
int *p = NULL;
*p = 5;
```

7. SIGABRT

Signal: SIGABRT

Meaning:

- Abort signal.

Triggered by:

abort();

Default:

- Terminate + core dump

8. SIGFPE

Signal: SIGFPE

Meaning:
Floating point exception.

Examples:

- Divide by zero
- Arithmetic overflow

9. SIGILL

Signal: SIGILL

Meaning:
Illegal CPU instruction.

Occurs when:

- Corrupted executable
- CPU unsupported instruction

Default:

Terminate + core dump.

10. SIGALRM

Signal: SIGALRM

Meaning:
Timer expired.

Example:

#include <unistd.h>

alarm(5);

After 5 seconds:

SIGALRM

Used for:

- timeouts
- watchdog timers.

11. SIGCHLD

SIGCHLD is sent to a **parent process** when the **state of its child process changes**.

Meaning:
Child process changed state.

Occurs when:

- Child exits
- Child stops

Used with:

wait()
waitpid()

12. SIGHUP

Signal: SIGHUP

Original meaning:

- Terminal disconnected.

Modern usage:

- **Reload configuration**

Example:

kill -HUP nginx_pid

sigaction

`sigaction()` is a **POSIX system call** used to **define how a process handles signals**.

It allows you to:

- Install a **signal handler**
- Control **which signals are blocked during handler execution**
- Set **advanced flags** that change signal behavior
- Get **extra information about the signal**

int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);

`signum`	Signal number (SIGINT, SIGTERM, etc)
`act`	New action for signal
`oldact`	Old signal action (optional backup)

Return value:

```
0  -> success
-1 -> error
```

```
struct sigaction {
void (*sa_handler)(int);
void (*sa_sigaction)(int, siginfo_t *, void *);
sigset_t sa_mask;
int sa_flags;
};

void handler(int sig)
{
printf("Signal received: %d\n", sig);
}

struct sigaction sa;
sa.sa_handler = handler;
```

sa_sigaction

If you want **more information about the signal**, use this.

```
void handler(int sig, siginfo_t *info, void *context)
{
printf("Signal: %d\n", sig);
printf("Sender PID: %d\n", info->si_pid);
}

sa.sa_flags = SA_SIGINFO;
sa.sa_sigaction = handler;
```

You **cannot use both `sa_handler` and `sa_sigaction` simultaneously**.

`sa_mask`

Controls **which signals are blocked while handler runs**.

```
sigemptyset(&sa.sa_mask);
sigaddset(&sa.sa_mask, SIGINT);
```

Meaning:

While handling signal **X**, block **SIGINT**.

Why?

To avoid **signal re-entry bugs**.

```cpp
#include <iostream>
// #include <csignal>
#include <signal.h>
#include <unistd.h>

using namespace std;
```

```
void handler(int sig)
{
     const char msg[] = "SIGINT received\n";
    write(STDOUT_FILENO, msg, sizeof(msg)-1);
    _exit(sig);
}

// void sa_sigaction_handler(int sig, siginfo_t* info, void* context)
// {
//         cout<<"handler : "<<sig<<endl;
//     cout<<"User pid : "<<info->si_pid<<endl;
// }
int main()
{
    struct sigaction sa;
    sa.sa_handler = handler;
    sa.sa_flags = 0;
    sigemptyset(&sa.sa_mask);
        //sa.sa_flags = SA_SIGINFO;
    //  sigaddset(&sa.sa_mask,SIGINT);
     sigaction(SIGINT, &sa, NULL);
    while (1)
    {
        printf("Running...\n");
        sleep(1);
    }
    return 0;
}
```

sa_flags

`SA_SIGINFO`	Use `sa_sigaction` instead of `sa_handler`
`SA_RESTART`	Restart interrupted system calls
`SA_NOCLDWAIT`	Don't create zombie children
`SA_NODEFER`	Do not block signal during handler
`SA_RESETHAND`	Reset handler to default after execution

C++ Multithreading

Multithreading is a specialized form of multitasking and a multitasking is the feature that allows your computer to run two or more programs concurrently. A **thread** is the smallest unit of execution inside a process.

A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread, and each thread defines a separate path of execution. Before C++ 11, there is no built-in support for multithreaded applications. Instead, it relies entirely upon the operating system to provide this feature.

```
#include<thread>
#include<iostream>
using namespace std;
```

```
void task(int x)
{
    cout<<"task execute "<<x<<endl;
     cout << "Thread 1 is running. ID: " << this_thread::get_id() << "\n";
}
int main() {
    thread t(task, 10);
    cout<<t.get_id()<<endl;
    if(t.joinable())
        t.join();
    return 0;
}
```

Thread Using Lambda

std::thread t([](){
std::cout<<"Hello thread\n";
});

Joining a Thread
Before joining a thread it is preferred to check if the thread can be joined using the joinable() method. The joinable method checks whether the thread is in a valid state for those operations or not.

```
thread_name.joinable()
```

The joinable() method returns true if the thread is joinable else returns false.

Joining a Thread: Joining two threads C++ blocks the current thread until the thread associated with the std::thread object finishes execution. To join two threads ini C++ we can use join() function. Which is called inside the bidy of the thread to which the specified thread is to be joined.

```
thread_name.join();
```

The thread.join function throws std::system_error if the thread is not joinable.
t.join();

If you forget join() or detach() → **program terminates**.

join()

Main thread **waits**.

main ----wait----> thread finished

detach()

Thread runs independently.

main exits → detached thread may still run

Detached threads easily create **bugs and resource leaks**.

Getting Thread ID
In Multithreading in C++ each thread has a unique ID which can be obtained by using the get_id() function.

Mutex (Mutual Exclusion)

To prevent race conditions.

#include <mutex>

std::mutex m;
int counter = 0;

void inc()
{
for(int i=0;i<100000;i++)
{
m.lock();
counter++;
m.unlock();
}
}

try_lock

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;
mutex m;
void task(int id)
{
    if (m.try_lock())
    {
        cout << "Thread " << id << " got lock\n";
        this_thread::sleep_for(chrono::seconds(2));
        m.unlock();
    }
    else
    {
        cout << "Thread " << id << " could not get lock\n";
    }
}
int main()
{
    thread t1(task, 1);
    thread t2(task, 2);
    t1.join();
    t2.join();
}
```

Better Way: lock_guard

Manual lock() / unlock() is risky.

std::mutex m;

void inc()
{
for(int i=0;i<100000;i++)
{
std::lock_guard<std::mutex> lock(m);
counter++;

}
} // Lock automatically releases.

unique_lock

unique_lock offers more flexibility than **lock_guard**. It provides features like manual locking and unlocking, deferred locking, and ownership transfer. Unlike lock_guard, which automatically locks and unlocks the mutex, unique_lock requires explicit calls to lock and unlock.
std::unique_lock<std::mutex> lock(m);

What happens here:

1. lock object is created
2. m.lock() is called
3. When lock goes out of scope → m.unlock() automatically happens
4. So it provides **RAII-based locking**, just like lock_guard.

void task(int id)
{
unique_lock<mutex> lock(m);

cout << "Thread " << id << " running\n";

}

unique_lock allows **unlocking before scope ends**.

void task()
{
unique_lock<mutex> lock(m);

cout<<"critical section\n";

lock.unlock();

cout<<"non critical work\n";
}

std::shared_lock

std::shared_lock is a **RAII wrapper** used with **std::shared_mutex** to acquire a **shared (reader) lock**.

Meaning:

- Multiple threads can hold this lock **at the same time**
- Used for **read-only operations**

Feature	shared_lock	unique_lock
Lock type	Shared	Exclusive
Readers allowed	Multiple	No
Writers allowed	No	Only one
Used for	Reading	Writing
Works with	shared_mutex	mutex, shared_mutex

```
#include <iostream>
```

```cpp
#include <thread>
#include <shared_mutex>
#include <mutex>
using namespace std;

shared_mutex smtx;
mutex cout_mtx;
int data1 = 0;
void reader(int id)
{
    int value;
    {
        shared_lock<shared_mutex> lock(smtx);
        value = data1;
    } // shared lock released here
    {
        lock_guard<mutex> coutLock(cout_mtx);
        cout << "data " << value << " id " << id << endl;
    }
}

void writer()
{
    {
        unique_lock<shared_mutex> lock(smtx);
        data1++;
    } // release lock early
    {
        lock_guard<mutex> coutLock(cout_mtx);
        cout << "Writer updated data\n";
    }
}
int main()
{
    thread t1(reader, 1);
    thread t2(reader, 2);
    thread t3(writer);
    t1.join();
    t2.join();
    t3.join();
    return 0;
}
```

Deferred locking

Sometimes you **create the lock but don't lock immediately**.

unique_lock<mutex> lock(m, defer_lock);

cout<<"do something first\n";

lock.lock();

condition_variable **requires unique_lock**.

unique_lock with try_to_lock

unique_lock<mutex> lock(m, try_to_lock);

if(lock.owns_lock)

{
cout<<"lock acquired\n";
}
else
{
cout<<"lock failed\n";
}

std::timed_mutex

Functions of timed_mutex

Function	Meaning
lock()	waits forever
try_lock()	tries once
try_lock_for()	waits for specific duration
try_lock_until()	waits until specific time

```cpp
#include <iostream>
#include <mutex>
#include <thread>
using namespace std;

timed_mutex tmtx;

void task(int id)
{
    if (tmtx.try_lock_for(chrono::seconds(2)))
    // if (tmtx.try_lock_until(chrono::steady_clock::now() + chrono::seconds(2)))
    {
        cout << "lock got for id " << id << endl;
        this_thread::sleep_for(chrono::seconds(4));
        tmtx.unlock();
    }
    else
    {
        // cout << "Dont lock gor for id " << this_thread::get_id() << endl;
        cout << "Dont lock got for id " << id << endl;
    }
}
int main()
{
    thread t1(task, 1);
    thread t2(task, 2);
    t1.join();
    t2.join();
    return 0;
}
```

shared_mutex

multiple readers allowed

Lock types

Function	Meaning
lock()	exclusive lock (writer)
lock_shared()	shared lock (reader)
unlock()	release writer
unlock_shared()	release reader

```
#include <iostream>
#include <thread>
#include <shared_mutex>
using namespace std;

shared_mutex smtx;
int data1 = 0;
void reader(int id)
{
    smtx.lock_shared();
    cout << "data " << data1 << " id " << id << endl;
    smtx.unlock_shared();
}

void writer()
{
    smtx.lock();
    data1++;
    cout << "Writer updated data\n";
    smtx.unlock();
}
int main()
{
    thread t1(reader,1);
    thread t2(reader,2);
    thread t3(writer);
    t1.join();
    t2.join();
    t3.join();
    return 0;
}
```

This protect only data1 not cout.

Multithreading is a technique where a program is divided into smaller units of execution called **threads**. Each thread runs independently but shares resources like memory, allowing tasks to be performed simultaneously.

Callables in Multithreading

A callable (such as a function, lambda, or function object) is passed to a thread. The callable is executed in parallel by the thread when it starts. like, **thread t(func)**; creates a thread that runs the **func** function. We can also pass parameters along with callable, like this thread **t(func, param1, param2)**;

Function Objects

Function Objects or **Functors** can also be used for a thread as callable. To make functors callable, we need to overload the operator parentheses operator **()**.

```
#include <iostream>

#include <thread>

using namespace std;

// Define a function object (functor)

class SumFunctor {

public:

    int n;

    SumFunctor(int a) : n(a) {}

    // Overload the operator() to  make it callable

    void operator()() const {

        cout << n;

    }

};

int main() {

    // Create a thread using  the functor object

    thread t(SumFunctor(3));

    // Wait for the thread to complete

    t.join();

    return 0;

}
```

Non-Static and Static Member Function

We can also use thread using the non-static or static member functions of a class. For non-static member function, we need to create an object of a class but it's not necessary with static member functions.

```
#include <iostream>

#include <thread>

using namespace std;
```

```
class MyClass {
public:
    // Non-static member function
    void f1(int num) {
        cout << num << endl;
    }
    // Static member function that takes one parameter
    static void f2(int num) {
        cout << num;
    }
};
int main() {
    // Member functions
    // requires an object
    MyClass obj;
    // Passing object and parameter
    thread t1(&MyClass::f1, &obj, 3);
    t1.join();
    // Static member function can
    // be called without an object
    thread t2(&MyClass::f2, 7);

    // Wait for the thread to finish
    t2.join();
    return 0;
}
```

Condition Variable

A **condition variable lets a thread sleep until another thread notifies it.**

condition variable is a synchronization primitive that is used to notify the other threads in a multithreading environment that the shared resource is free to access. It is defined as the std::condition_variable class inside the <condition_variable> header file.

Condition variable is especially needed in cases where one thread has to wait for another thread execution to continue the work.

S. No.	Function	Description
1	**wait()**	This function tells the current thread to wait till the condition variable is notified.
2	**wait_for()**	This function tells the current thread to wait for some specific time duration. If the condition variable is notified earlier than the time duration, the thread awakes. The time is specified as relative time.
3	**wait_until()**	This function is similar to the wait_for() but here, the time duration is defined as the absolute time.
4	**notify_one()**	This function notifies one of the waiting threads that the shared resource is free to access. The thread selection is random.
5	**notify_all()**	This function notifies all of the threads.

```
#include<iostream>
#include <mutex>
#include <condition_variable>
#include <thread>
using namespace std;

mutex mtx;
condition_variable cv;
bool is_ready = false;
void producer()
{
    this_thread::sleep_for(chrono::seconds(3));
    cout<<"producer running\n";
    lock_guard<mutex> lock(mtx);
    is_ready = true;
    cv.notify_one();
```

```
}

void consumer()
{
    cout<<"consumer start\n";
    unique_lock<mutex> lock(mtx);
  //  cv.wait(lock);
    cv.wait(lock, []{return is_ready;});
    cout<<"consumer end\n";
}
int main() {
    thread t1(producer);
    thread t2(consumer);

    t1.join();
    t2.join();
    return 0;
}
```

Using lock_guard instead of unique_lock . wrong, because Because wait() must **unlock and relock** mutex. lock_guard cannot unlock.

if(cv.**wait_for**(lock, chrono::seconds(3), []{ return ready; }))
cout << "Data received\n";
else
cout << "Timeout occurred\n";

auto deadline = chrono::steady_clock::now() + chrono::seconds(3);

if(cv.**wait_until**(lock, deadline, []{ return ready; }))
cout<<"Data received\n";
else
cout<<"Timeout\n";

atomic

multiple threads often access and modify shared data simultaneously, which can lead to data races and unpredictable behavior. <atomic> solves this problem by providing atomic operations allowing threads to access and modify variables safely, without explicit locks or synchronization mechanisms. std::atomic type and the <atomic> header.

Manages shared variables between threads in a thread-safe manner without using locks.

For simple operations, mutex is slow.

Use atomic.

#include <atomic>

std::atomic<int> counter = 0;

void inc()
{
   for(int i=0;i<100000;i++)

```
    counter++;
}
```

No mutex needed

load()

Read value.

int x = counter.load();

Equivalent to:

atomic read

store()

Write value.

counter.store(5);

fetch_add()

Add and return old value.

counter.fetch_add(1);

Example:

counter = 5

fetch_add(1)

returns 5
counter becomes 6

exchange()

Replace value atomically.

counter.exchange(10);

Example:

counter = 3

exchange(10)

returns 3
counter becomes 10

```cpp
#include<iostream>
#include<thread>
#include <atomic>
using namespace std;

atomic<int> counter(0);
```

```
void func()
{
    for(int i =0;i<100000;i++)
    {
      //  counter++;
      counter.fetch_add(1);
    }
}
int main() {
    thread t1(func);
    thread t2(func);
    t1.join();
    t2.join();
    cout<<counter.load()<<endl;
    return 0;
}
```

atomic_flag is the **simplest atomic boolean type** in C++.

- Only two operations:
  - test_and_set()
  - clear()

It is guaranteed **lock-free** on all platforms.

It is mostly used for **spinlocks**.

How it works

test_and_set()

- Sets the flag to true
- Returns **previous value**

So if it was already true → another thread owns it.

```
#include <iostream>
#include <atomic>
#include <thread>

using namespace std;

atomic_flag lock = ATOMIC_FLAG_INIT;

void work(int id)
{
    cout<<"thread start "<<id<<endl;
    while(lock.test_and_set()) ;   // spin until false
    this_thread::sleep_for(chrono::seconds(3));
    cout << "Thread " << id << " entered critical section\n";

    lock.clear();
    cout<<"thread end "<<id<<endl;
```

```
}

int main()
{
    thread t1(work,1);
    thread t2(work,2);

    t1.join();
    t2.join();
}
```

What happens

Thread 1

test_and_set → false
sets true
enters critical section

Thread 2

test_and_set → true
spins until clear()

This is called **spinlock**.

compare_exchange_weak and compare_exchange_strong

```
#include <iostream>
#include <thread>
#include <atomic>
using namespace std;
atomic<int> at = 10;
int expected = 10;
// int expected= 15;
bool flag = at.compare_exchange_strong(expected, 20);
atomic<int> x = 0;

void increment()
{
    int expected1;

    do
    {
        expected1 = x.load();
        cout << "expected " << x.load() << endl;
    } while (!x.compare_exchange_weak(expected1, expected1 + 1));
    cout << "expected1 " << x << endl;
}
int main()
{
    cout << expected << endl;
    cout << at << endl;
    cout << flag << endl;
    increment();
```

```
    return 0;
}
```

compare_exchange_weak

May **fail even if values match**.

This is called **spurious failure**.

So it is usually used inside loops.

Example:

while(!x.compare_exchange_weak(expected,20))
{
}

Why?

Because some CPU architectures (ARM) allow spurious failure.

---

compare_exchange_strong

- Does **not fail spuriously**
- Slower sometimes

Types of Memory Ordering

1. memory_order_relaxed

Definition

Only guarantees:

operation is atomic

But **NO ordering guarantees**.

CPU may reorder everything.

This only ensures:

no data race

But no ordering between other variables.

---

2. memory_order_release

Definition

Ensures:

All previous writes become visible
before this store.

Used when **publishing data**.

---

3. memory_order_acquire

Definition

Ensures:

All reads after acquire
see writes before release.

Why acquire?

Because it ensures **data = 100 is visible**.

Acquire + Release together

producer: release
consumer: acquire

Guarantees:

producer writes → visible to consumer

```
#include <atomic>
#include <thread>
#include <iostream>
using namespace std;
int data1;
atomic<bool> ready(false);
void producer()
{
    data1 = 42;

    ready.store(true, memory_order_release);
}
void consumer()
{
    while (!ready.load(memory_order_acquire));
    cout << data1 << endl;
}
atomic<int> x = 0;

void write1()
{
    x.store(10, memory_order_relaxed);
}

void read1()
{
```

```
    cout << x.load(memory_order_relaxed);
}
int main()
{
    thread t1(producer);
    thread t2(consumer);
    t1.join();
    t2.join();
    thread t3(write1);
    thread t4(read1);
    t3.join();
    t4.join();
}
```

---

4. memory_order_seq_cst

Definition

Strongest ordering.

Guarantees **global sequential consistency**.

Meaning:

all threads observe operations
in same order

This is **default ordering in C++ atomics**.

Example

atomic<int> x=0;

x.store(10); // seq_cst by default

Ordering	Guarantee	Performance
relaxed	only atomic	fastest
acquire	read barrier	fast
release	write barrier	fast
seq_cst	full ordering	slowest

Thread Pool

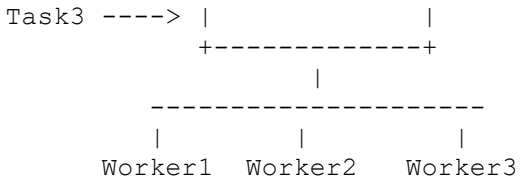
Create a **fixed set of worker threads once**.

Then reuse them for tasks.

```
Tasks → Queue → Worker Threads
```

Workers repeatedly take tasks and execute them.

```
            +--------------+
Task1 ----> |              |
Task2 ----> |  Work Queue  |
```



Workers run forever waiting for tasks.

Advantages

1 Avoid thread creation overhead
2 Limit number of threads
3 Better CPU utilization
4 Prevent system overload

Bad approach

Imagine a web server doing this:

```
request arrives
create thread
process request
destroy thread
```

If 10,000 requests arrive → **10,000 thread creations**.

Your CPU spends time **managing threads instead of doing work**.

Work Queue

A **work queue** is a thread-safe queue where tasks are stored until workers execute them.

1. Deadlock

Definition

A deadlock happens when **two or more threads wait forever for each other's lock**, so none of them can continue.

Example

mutex m1, m2;

Thread1:
lock(m1);
lock(m2);

Thread2:
lock(m2);
lock(m1);

Situation:

Thread1 holds m1 → waiting for m2
Thread2 holds m2 → waiting for m1

Both wait forever → **deadlock**.

---

2. Livelock

Definition

Threads are **not blocked**, but they keep reacting to each other and **never make progress**.

Example idea

Two threads repeatedly release and retry locks:

Thread1: can't get lock → release → retry
Thread2: can't get lock → release → retry

Both keep moving but **work never finishes**.

---

3. Lock Ordering

Definition

A technique to **prevent deadlock** by always acquiring locks in the **same order**.

Example

Correct order:

```
lock(m1);
lock(m2);
```

All threads must follow:

m1 → then m2

Never:

m2 → then m1

This prevents circular waiting.

---

4. Starvation

Definition

A thread **never gets CPU or resources** because other threads keep taking them.

Example

Many readers using shared_mutex
Writer keeps waiting forever

Writer **starves**.

---

5. False Sharing

Definition

Two threads modify **different variables**, but those variables are on the **same CPU cache line**, causing performance slowdown.

Example

```
struct Data {
   int a;
   int b;
};
```

Thread1 updates a
Thread2 updates b

They share same cache line → **constant cache invalidation** → slow.

---

6. Atomic vs Mutex

Atomic

Definition

Lock-free variable operations that are **guaranteed to be thread-safe**.

Example:

```
atomic<int> counter;

counter++;
```

Mutex

Definition

A lock that allows **only one thread** to access a critical section.

Example:

```
mutex m;

m.lock();
counter++;
m.unlock();
```

7. Thread Lifecycle

Definition

The stages a thread goes through during execution.

Typical stages:

Created → Runnable → Running → Waiting → Terminated

Example in C++:

```
thread t(task);
t.join();
```

Lifecycle:

created → running → finished

8. Race Condition

Definition

A situation where **program result depends on the order of thread execution**.

Example:

```
int x = 0;
```

```
Thread1: x++
Thread2: x++
```

Result could be:

1 or 2

depending on timing.

---

9. Critical Section

Definition

A part of code where **shared data is accessed** and must be **protected from concurrent access**.

Example:

```
mutex m;
```

```
m.lock();
balance += 100;
m.unlock();
```

This block is the **critical section**.

Reference to Array

This preserves the **array type and size**.

No decay happens.

Example

```
#include <iostream>
using namespace std;

template<size_t N>
void func(int (&arr)[N])
{
   cout << N << endl;
}

int main()
{
   int arr[5];
   func(arr);
}
```

Multithreading in C

A **thread** is the **smallest unit of execution inside a process** that the operating system can schedule.

A thread has its own:

- program counter
- registers
- stack

But **shares with other threads in the same process**:

- memory (heap)
- global variables
- code segment
- file descriptors

Multithreading is a technique where **multiple threads run concurrently within a single process to perform tasks simultaneously.**

Creating a Thread

```
pthread_create(thread, attr, routine, arg);
```

where,

- **thread** : Pointer to a **pthread_t** variable where the system stores the ID of the new thread.
- **attr** : Pointer to a thread attributes object that defines thread properties. Use NULL for default attributes.
- **routine**: Pointer to the function that the thread will execute. It must return void* and accept a void* argument.
- **arg**: A single argument passed to the thread function. Use NULL if no argument is needed. You can pass a struct or pointer to pass multiple values.

```
#include <stdio.h>
```

```c
#include <unistd.h>
#include <pthread.h>

void* func(void* arg)
{
        printf("Created a new thread\n");
    int num = *(int*)arg;
    printf("arg is %d\n",num);
    return NULL;
}
int main(int argc, char const *argv[])
{
    pthread_t tid;
    int x=10;
    pthread_create(&tid, NULL, func, &x);
    sleep(3); // sleep not add then it does not print anything
    return 0;
}
```

Use default attributes

Which typically means:

- stack size ≈ 8MB
- thread is **joinable**
- normal scheduling policy

What attributes control

The attribute object can control things like:

1. **Thread detached or joinable**
2. **Stack size**
3. **Stack address**
4. **Scheduling policy**
5. **Thread priority**

pthread_attr_t attr;

pthread_attr_init(&attr);

pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);

pthread_create(&tid, &attr, worker, NULL);

pthread_attr_destroy(&attr);

if it is **detached**, the OS automatically cleans it.

Then you **cannot join that thread**. A **detached thread** cleans itself automatically after finishing.

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```c
void* task(void* arg)
{
printf("Thread started\n");
sleep(2);
printf("Thread finished\n");
return NULL;
}

int main()
{
pthread_t tid;
pthread_attr_t attr;

// 1. Initialize attribute object
pthread_attr_init(&attr);

// 2. Change attribute: make thread detached
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);

// 3. Create thread with this attribute
pthread_create(&tid, &attr, task, NULL);

// 4. Destroy attribute object (not needed anymore)
pthread_attr_destroy(&attr);

printf("Main thread exiting\n");

sleep(3); // keep process alive to see thread output

return 0;
}
```

pthread_join() function allows one thread to wait for the termination of another thread. It is used to synchronize the execution of threads.

int pthread_join(pthread_t thread, void **retval);

void **retval

This is used to **receive the return value from the thread function**.

void* function(void* arg)

So the return value is a **pointer**.

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* func(void* arg)
{

   static int result = 42;
   return &result;
}
```

```c
int main(int argc, char const *argv[])
{
    pthread_t tid;
    void* ret;
    pthread_create(&tid, NULL, func, NULL);
    pthread_join(tid,&ret);
     printf("Thread returned: %d\n", *(int*)ret);
    return 0;
}
```

pthread_exit() function allows a thread to terminate its execution explicitly. **pthread_exit()** is called when a thread needs to terminate its execution and optionally return a value to threads that are waiting for it.

```
    //Explicity terminate thread

    pthread_exit(NULL);
```

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* task(void* arg)
{
    sleep(2);
    printf("Thread finished\n");
    return NULL;
}

int main()
{
    pthread_t tid;
    pthread_create(&tid, NULL, task, NULL);
    pthread_exit(NULL); // main exits but thread continues
}
```

main thread exits
other threads keep running

pthread_cancel() function is used to request the cancellation of a thread. It sends a cancellation request to the target thread, but the actual termination depends on whether the thread is in a cancellable state and if it handles cancellation.

```
// To exit the current thread

    // pthread_self() return the particular thread id

    pthread_cancel(pthread_self());
```

pthread_self() function returns the thread ID of the calling thread. This is useful when you need to identify or store the ID of the current thread in multi-threaded programs.

```
// Get current thread ID
```

```
pthread_t thisThread = pthread_self();

printf("Current thread ID: %lu\n",

    (unsigned long)thisThread);
```

Race Condition

The race condition issue occurs when multiple threads try to access a shared resource at the same time and the output depends on the order of the execution of those threads. This issue can lead to unpredictable behavior of the program and cause the program to generate different results each time when it is executed.

- **Critical Section**: A code part where the shared resources are accessed. It is critical as multiple processes enter this section at same time leading to data corruption and errors.
- **Synchronization:** It is the process of controlling how and when multiple processes or threads access the shared resources ensuring that only one can enter the critical section at same time.
- **Mutual Exclusion (Mutex):** A mutex is like a lock that ensures only one process can access a resource at a time. If a process holds a lock, others must wait their turn preventing race conditions.

Deadlocks

A deadlock condition arises when multiple threads are blocked forever as they are waiting for each other to release the occupied resource. It generally arises in situations where thread acquire some resources initially and request for more resources midway during their execution. Which leads to a cycle of dependencies.
Example: Thread A has occupied Resource 1 and 2 arewaiting for resource 3 while thread B has occupied resource 3 and is waiting for resource 2 for its completion.

Deadlock is a state in an operating system where two or more processes are stuck forever because each is waiting for a resource held by another

Thread Block

Thread blocking happens when a thread is unable to continue its work because it's waiting for a resource that's currently unavailable. It pauses until the resource is free.

Example: A cashier waiting for the customer ahead to finish paying before attending to the next person in line.

Starvation

The starvation condition arises when a thread is denied access for the requested resource for indefinite amount of time. This situation commonly occurs with priority-based scheduling algorithms when they are biased toward certain threads.
Example: A thread with low priority will have to wait indefinitely as higher priority threads are continuously available.

Starvation (or indefinite blocking) occurs in **priority scheduling** when a low-priority process keeps waiting indefinitely because higher-priority processes continuously get the CPU.

Aging - Solution to Starvation

Aging is a scheduling technique used to prevent starvation by gradually increasing the priority of processes waiting too long in the system.

- Ensures fairness, as long-waiting processes eventually get CPU time.
- Often combined with algorithms like priority scheduling or round-robin to balance short-term efficiency with long-term fairness.

For example, if priorities range from 127 (low) to 0 (high), a waiting process can move up one level every 15 minutes, ensuring even the lowest-priority process eventually gets executed.

Threading Issues in OS

fork() and exec() System Calls

fork() system call, therefore. Let us assume that one of the threads belonging to a multi-threaded program has instigated a fork() call. Therefore, the new process is a duplication of fork(). Here, the question is as follows; will the new duplicate process made by fork() be multi-threaded like all threads of the old process or it will be a unique thread?
Child → exact copy (memory, stack, heap)

- POSIX `fork()` → **copies only the calling thread**
- There is **no standard fork that copies all threads**

"What happens if fork is called in multithreaded program?"

Answer:

Only the calling thread is duplicated in the child process.
Other threads are not copied, which may lead to inconsistent states such as locked mutexes, causing deadlocks.
Therefore, after fork in a multithreaded program, only async-signal-safe functions should be called, typically followed by exec.

When the next system call, namely exec() system call is issued, it replaces the whole programming with all its threads by the program specified in the exec() system call's parameters. Ordinarily, the exec() system call goes into queue after the fork() system call.

Thread Cancellation

A **thread cancellation issue** happens when you try to **stop (cancel/kill) a thread**, but:

- it **doesn't stop when expected**, or
- it **stops in a dangerous state**, causing bugs like:
  - memory leaks
  - deadlocks
  - corrupted data

Two main types of cancellation

1. Deferred cancellation (safe way)

- Thread checks at specific points: "should I stop?"
- Cancellation happens only at **safe points**

☞ Example safe points:

- pthread_testcancel()

- blocking calls like sleep(), wait()

✓ Controlled
✓ Safer

2. Asynchronous cancellation (dangerous)

- Thread can be killed **anytime**
- Even while holding a lock or modifying data

```
pthread_mutex_lock(&mtx);

// critical section
// thread gets cancelled here ✗

pthread_mutex_unlock(&mtx);
```

Thread Synchronization

Thread synchronization is a process that is used to ensure that multiple threads can work with shared resources without causing any issues like race conditions, deadlocks, or data corruption. Thread synchronisation techniques control how threads interact or modify shared data or resources and also ensures that certain critical sections of code are executed in a controlled manner.

Mutex-Based Synchronization

A mutex is the most basic synchronization mechanism. It ensures that only one thread can access a shared resource at a time, preventing race conditions

```c
#include <pthread.h>
#include <stdio.h>
//pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock;
int counter = 0;
void *increament(void *arg)
{
    pthread_mutex_init(&lock, NULL);
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
}
int main(int argc, char const *argv[])
{
    pthread_t tid[2];
    for (int i = 0; i < 2; i++)
    {
        pthread_create(&tid[i], NULL, increament, NULL);
    }
    for (int i = 0; i < 2; i++)
    {
        pthread_join(tid[i], NULL);
    }
    printf("ct = %d\n", counter);
```

```
    return 0;
}
```

Recursive mutex:

```
#include <pthread.h>
#include <stdio.h>

pthread_mutex_t lock;

void inner()
{
    pthread_mutex_lock(&lock);
    printf("Inner\n");
    pthread_mutex_unlock(&lock);
}

void outer()
{
    pthread_mutex_lock(&lock);
    printf("Outer\n");
    inner(); // 🔥 tries to lock again
    pthread_mutex_unlock(&lock);
}

int main()
{
    pthread_mutexattr_t attr;
    pthread_mutexattr_init(&attr);
    pthread_mutexattr_settype(&attr, PTHREAD_MUTEX_RECURSIVE);

    pthread_mutex_init(&lock, &attr);

    outer();

    pthread_mutex_destroy(&lock);
}
```

Semaphores in Process Synchronization

A **semaphore** is a synchronization tool that controls access to a shared resource using a **counter**.

A semaphore is a synchronization tool used in operating systems to manage access to shared resources in a multi-process or multi-threaded environment. It is an integer variable that controls process execution using atomic operations like wait() and signal(). Semaphores help prevent race conditions and ensure proper coordination between processes.

- Controls entry into the critical section.
- Maintains a counter representing available resources.
- Ensures mutual exclusion among processes.
- Can block and wake up processes during execution.
- Widely used in concurrent programming.

Example: Let's consider two processes P1 and P2 sharing a semaphore S, initialized to 1:

- **State 1**: Both processes are in their non-critical sections, and S = 1.
- **State 2**: P1 enters the critical section. It performs wait(S), so S = 0. P2 continues in the non-critical section.
- **State 3**: If P2 now wants to enter, it cannot proceed since S = 0. It must wait until S > 0.

- **State 4**: When P1 finishes, it performs signal(S), making S = 1. Now P2 can enter its critical section and again sets S = 0.

Features of Semaphores
- **Mutual Exclusion**: Semaphore ensures that only one process accesses a shared resource at a time.
- **Process Synchronization**: Semaphore coordinates the execution order of multiple processes.
- **Resource Management**: Limits access to a finite set of resources, like printers, devices, etc.
- **Reader-Writer Problem**: Allows multiple readers but restricts the writers until no reader is present.
- **Avoiding Deadlocks**: Prevents deadlocks by controlling the order of allocation of resources.

Types of Semaphores
Semaphores are mainly of two Types:

1. Counting Semaphore

A counting semaphore can have values ranging from 0 to any positive integer. It is used when multiple instances of a resource are available and need to be managed.

- Value ranges from 0 to n.
- Manages multiple resource instances.
- Controls access to limited resources.
- Example: Managing access to 5 printers or 3 database connections.

2. Binary Semaphore

A binary semaphore has only two possible values: 0 and 1. It is mainly used for mutual exclusion, ensuring that only one process enters the critical section at a time.

- Value is either 0 or 1.
- Used for mutual exclusion.
- Similar to a mutex lock.
- Managing access to a single critical section

Limitations of Semaphores
- Priority Inversion: A low-priority process holding a semaphore can block a high-priority one.
- Deadlock: Processes may wait on each other's semaphores in a cycle, causing indefinite blocking.
- Complex to Manage: The OS must carefully track wait and signal calls; misuse can cause errors.
- Busy Waiting: In basic implementations, processes may keep checking the semaphore value, wasting CPU time.

POSIX Semaphore Functions (C)

```
#include <semaphore.h>

sem_t sem;

sem_init(&sem, 0, value);   // initialize
sem_wait(&sem);             // decrement (block if 0)
sem_post(&sem);             // increment
sem_destroy(&sem);          // cleanup
```

Binary Semaphore (0 or 1)

Key difference:

- Mutex → ownership (same thread must unlock)
- Semaphore → no ownership (any thread can post)

```
#include <stdio.h>
```

```c
#include <pthread.h>
#include <semaphore.h>

sem_t sem;

void* task(void* arg)
{
sem_wait(&sem);   // enter critical section

printf("Thread %ld in critical section\n", (long)arg);

sem_post(&sem);   // exit

return NULL;
}

int main()
{
pthread_t t1, t2;

sem_init(&sem, 0, 1); // binary semaphore

pthread_create(&t1, NULL, task, (void*)1);
pthread_create(&t2, NULL, task, (void*)2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

sem_destroy(&sem);
}
```

Counting Semaphore

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t sem;

void* task(void* arg)
{
   sem_wait(&sem);  // take a resource

   printf("Thread %ld using resource\n", (long)arg);
   sleep(2);

   printf("Thread %ld releasing resource\n", (long)arg);

   sem_post(&sem);  // release resource

   return NULL;
```

```
}

int main()
{
   pthread_t threads[5];

   sem_init(&sem, 0, 3); // 3 resources

   for (long i = 0; i < 5; i++)
      pthread_create(&threads[i], NULL, task, (void*)i);

   for (int i = 0; i < 5; i++)
      pthread_join(threads[i], NULL);

   sem_destroy(&sem);
}
```

What's happening:

- Only **3 threads run simultaneously**
- Others wait
- Perfect for:
  - DB connections
  - Thread pools
  - Rate limiting

Conditional Variables

Condition variables allow threads to wait for certain conditions to be met before proceeding. A condition variable is always used along with a mutex lock. A thread will wait on the condition variable and release the mutex until it is notified by another thread.

What are conditional wait and signal in multi-threading?
Explanation: When you want to sleep a thread, condition variable can be used. In C under Linux, there is a function pthread_cond_wait() to wait or sleep.
On the other hand, there is a function pthread_cond_signal() to wake up sleeping or waiting thread. Threads can wait on a condition variable.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_cond_t cond1 = PTHREAD_COND_INITIALIZER;

pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;

int done = 1;

void *func(void *arg)
{
   pthread_mutex_lock(&mtx);
   if (done == 1)
   {

      // let's wait on condition variable cond1
      done = 2;
      printf("Waiting on condition variable cond1 : %d\n", (long)arg);
```

```
        pthread_cond_wait(&cond1, &mtx);
    }
    else
    {

        // Let's signal condition variable cond1
        printf("Signaling condition variable cond1 : %d\n", (long)arg);
        pthread_cond_signal(&cond1);
    }
    printf("thread exit : %d\n", (long)arg);
    pthread_mutex_unlock(&mtx);
}
int main()
{
    pthread_t tid1, tid2;

    // Create thread 1
    pthread_create(&tid1, NULL, func, (void *)1);

    // sleep for 1 sec so that thread 1
    // would get a chance to run first
    sleep(1);

    // Create thread 2
    pthread_create(&tid2, NULL, func, (void *)2);

    // wait for the completion of thread 2
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    return 0;
}
```

Barrier Synchronization

Every thread hits a checkpoint → **no one moves forward until ALL arrive**.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define NUM_THREADS 3

pthread_barrier_t barrier;

void *task(void *arg)
{
    int id = *(int *)arg;

    sleep(id); // simulate different speeds
    printf("Thread %d before barrier\n", id);

    pthread_barrier_wait(&barrier);

    printf("Thread %d after barrier\n", id);
```

```
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];

    pthread_barrier_init(&barrier, NULL, NUM_THREADS);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, task, &ids[i]);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    pthread_barrier_destroy(&barrier);
    return 0;
}
```

Read-Write Locks (pthread_rwlock)

- Writers block everyone (including readers)
- Readers can starve writers if not careful
- This is about **performance optimization**, not correctness

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_rwlock_t rwlock;
int shared_data = 0;

void* reader(void* arg) {
    int id = *(int*)arg;

    pthread_rwlock_rdlock(&rwlock);
    printf("Reader %d reads: %d\n", id, shared_data);
    sleep(1);
    pthread_rwlock_unlock(&rwlock);

    return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;

    pthread_rwlock_wrlock(&rwlock);
    shared_data++;
    printf("Writer %d writes: %d\n", id, shared_data);
    sleep(2);
```

```
    pthread_rwlock_unlock(&rwlock);

    return NULL;
}

int main() {
    pthread_t r1, r2, w1;
    int id1 = 1, id2 = 2, id3 = 3;

    pthread_rwlock_init(&rwlock, NULL);

    pthread_create(&r1, NULL, reader, &id1);
    pthread_create(&r2, NULL, reader, &id2);
    pthread_create(&w1, NULL, writer, &id3);

    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    pthread_join(w1, NULL);

    pthread_rwlock_destroy(&rwlock);
    return 0;
}
```

Design pattern

Singleton Pattern in C++

A Singleton Pattern is a design pattern which allows to create only one object of a class. Instead of allowing the class to create multiple instances everytime it is called, the singleton pattern ensures that only one instance of the class is created and it also provides it a global access point so that it can be accessed from anywhere in the code.

Key Points of Singleton Pattern

Following are the key points of Singleton Pattern −

• It restricts the instantiation of a class to one single instance.

• It provides a global access point to that instance.

• It is often used for managing shared resources such as database connections, configuration settings, logging, etc.

• It can be implemented using lazy initialization, where the instance is created only when it is needed for the first time.

Implementation of Singleton Pattern in C++

Lazy Initialization Singleton

In this approach, the instance of the class is created only when it is needed for the first time. This can help us in creating only one instance of the class and it also saves memory.

Step to implement Singleton Pattern in C++

Following are the steps to implement Singleton Pattern in C++ −

• Make the constructor of the class private so that it cannot be instantiated from outside the class.

• Then, create a static variable that holds the single instance of the class.

• Create a static method that returns the instance of the class. This method checks if the instance is already created, if not it creates one and returns it. Otherwise, it simply returns the existing instance.

Rule (clean version):

Inside a class, you can initialize static members **only if** they are:

- constexpr (C++11+)
- or inline (C++17+)
- or **integral const** (special old exception like int, char, etc.)

```
#include <iostream>
using namespace std;

class logger
{
private:
   static logger *lg_obj;
   logger() {}

public:
   static logger *getInstance()
   {
      if (lg_obj == nullptr)
      {
         lg_obj = new logger();
      }
      return lg_obj;
   }
   void log(string message)
   {
      cout << "Log: " << message << endl;
   }
};

logger *logger::lg_obj = nullptr;
int main()
{
   logger *lg1 = logger::getInstance();
   lg1->log("thi is log");
   return 0;
}
```

Eager Initialization Singleton(Hungry Singleton) Implementation

In this approach, we create the instance of the class at the time of class loading. This approach is simpler and it is also thread-safe. However, there is a drawback of this approach is that the instance is created even if it is not used, which can lead to resource wastage. So if you are sure that the instance will be used, then this approach is a good choice

```
#include <iostream>
using namespace std;

class Logger
{
private:
   static Logger *lg_obj;
   Logger() {}

public:
   static Logger *getInstace()
   {
      return lg_obj;
   }

   void log(string message)
   {
      cout << message << endl;
   }
};

Logger *Logger::lg_obj = new Logger();
int main()
{
   Logger *lg1 = Logger::getInstace();
   lg1->log("log message");

   return 0;
}
```

Mayers' Singleton Implementation

The Mayers' Singleton implementation is a very good modern approach for creating singleton classes in C++. This approach uses a static local variable inside the getInstance() method to hold the single instance of the class. The instance is created when the method is called for the first time and it is destroyed automatically when the program exits. This approach is thread-safe and it also ensures that only one instance of the class is created.

```
#include <iostream>
using namespace std;

class Logger
{
private:
   Logger() {}

public:
   static Logger &getInstace()
```

```
   {
      static Logger lg_obj;
      return lg_obj;
   }

   void log(string message)
   {
      cout << message << endl;
   }
};

int main()
{
   Logger &lg1 = Logger::getInstace();
   lg1.log("log message");
   Logger &lg2 = Logger::getInstace();
   lg2.log("log message1");

   return 0;
}
```

Real-world use cases of Singleton Pattern

Following are some real-world use cases of Singleton Pattern −

Logger class − As we have seen in the examples, the logger class uses the singleton pattern, so it can be accessed anywhere but creates only one instance throughout the application.

Configuration class − A configuration class can be implemented as a singleton to ensure that there is only one instance of the configuration settings throughout the application.

Database connection class − Another usecase is to implement a database connection class, so that you don't need to create multiple connections to the database.

Printer Spooler − A printer spooler can be implemented as a singleton to manage print jobs in a centralized manner. So, your whole system has only one instance of the printer spooler instead of making multiple instances for every application on the system.

Factory Method Pattern in C++

Factory Method is a design pattern used to **create objects without exposing the exact creation logic to the user**.

The Factory Method Pattern is a creational design pattern that defines an interface for creating an object. It allows its subclasses to change the type of object that will be created

For example, imagine you are creating an employee management system for your comapany, and there are different types of employees such as full-time employees, part-time employees, and contract employees. Each type of employee has different attributes and behaviors. Instead of creating a separate class for each type of employee, you can use the factory method pattern to create a single interface for creating employees and then let the subclasses decide which type of employee to create.

Implementation of Factory Method by the Official GoF Way in C++

This is an official implementation of the factory method pattern as described in the Gang of Four book. In this approach, we create an abstract class that defines the factory method and then create concrete subclasses that implement the factory method to create specific types of objects.

Let's take an example of a car factory. We have an abstract class CarFactory that defines the factory method createCar(). Then, we create concrete subclasses such as SUVFactory and SedanFactory that implement the factory method to create specific types of cars.

```
#include <iostream>
using namespace std;

class Car
{
public:
   virtual void drive() = 0;
   virtual ~Car() {}
};

class mustang : public Car
{
public:
   void drive()
   {
      cout << "Driving mustang\n";
   }
};

class porsche : public Car
{
public:
   void drive()
   {
      cout << "Driving porsche\n";
   }
};

class carFactory
{
public:
   virtual Car *createCar() = 0;
   virtual ~carFactory() {}
};

class mustangFactory : public carFactory
{
public:
   Car *createCar()
   {
      return new mustang();
   }
};
```

```
class porscheFactory : public carFactory
{
public:
    Car *createCar()
    {
        return new porsche();
    }
};
int main()
{
    carFactory *factory = new mustangFactory();
    Car *car = factory->createCar();
    car->drive();
    delete car;
    delete factory;

    factory = new porscheFactory();
    car = factory->createCar();
    car->drive();
    delete car;
    delete factory;
    return 0;
}
```

Simple Factory Pattern Implementation in C++

This is a simple way to implement the factory pattern also known as Static Factory Method. In this approach, we create a single class that contains a static method to create objects of different types based on the input parameters. Then, in the factory method, we use a switch statement or if-else statements to determine which type of object to create based on the input parameters.

```
#include <iostream>
using namespace std;

class Shape
{
public:
    virtual void draw() = 0;
    virtual ~Shape() {}
};

class Circle : public Shape
{
public:
    void draw() override
    {
        cout << "Drawing Circle" << endl;
    }
};

class Triangle : public Shape
{
```

```
public:
    void draw() override
    {
        cout << "Drawing Triangle" << endl;
    }
};

class ShapeFactory
{
public:
    static Shape *createShape(const string &type)
    {
        if (type == "Circle")
        {
            return new Circle();
        }
        else if (type == "Triangle")
        {
            return new Triangle();
        }
        else
        {
            return nullptr;
        }
    }
};

class client
{
public:
    void drawShape(const string &type)
    {
        Shape *shape = ShapeFactory::createShape(type);
        if (shape)
        {
            shape->draw();
            delete shape;
        }
        else
        {
            cout << "Invalid shape\n";
        }
    }
};
int main()
{
    // Shape* shape = ShapeFactory::createShape("Circle");
    // shape->draw();
    client c1;
    c1.drawShape("Circle");
    c1.drawShape("Triangle");
    c1.drawShape("square");
    return 0;
}
```

Registry-based Factory Pattern Implementation in C++

This is a more advanced way to implement the factory pattern. In this approach, we create a registry of factory methods that can be used to create objects of different types. The registry is typically implemented as a map or dictionary that maps a string or an enum value to a factory method. This allows us to easily add new types of objects without modifying the existing code.

In the previous approaches, we have used block of if-else or used small factory classes for each type. But in this approach, we can centralize the creation logic in a single registry class.

Think of it as a map where −

The key is the type of thing we want (like "Cat" or "Dog").

The value is a function that knows how to make that thing we want.

It is like a notebook where you write down how to make different things. When you want to make something, you look it up in your notebook and follow the instructions. And when you want to make something new, you just add a new entry to your notebook without changing the existing ones.

This way, we can easily extend the factory to create new types of objects without modifying the existing code.

```
#include <iostream>
#include <functional>
#include <map>
using namespace std;

class Shape
{
public:
   virtual void draw() = 0;
   virtual ~Shape() {}
};

class Circle : public Shape
{
public:
   void draw() override
   {
      cout << "Drawing circle" << endl;
   }
};

class Square : public Shape
{
public:
   void draw() override
   {
      cout << "Drawing square" << endl;
```

```
    }
};

class shapeFactory
{
public:
    using shapecreator = function<Shape *()>;

    static Shape *createShape(string type)
    {
        auto it = registory().find(type);
        if (it != registory().end())
        {
            return it->second();
        }
        return nullptr;
    }
    static void registerShape(string type, shapecreator creator)
    {
        registory()[type] = creator;
    }

private:
    static map<string, shapecreator> &registory()
    {
        static map<string, shapecreator> instance;
        return instance;
    }
};

class client : public shapeFactory
{
public:
    void drawing(string type)
    {
        Shape *shape = shapeFactory::createShape(type);
        if (shape)
        {
            shape->draw();
            delete shape;
        }
        else
        {
            cout << "Invalid Shape" << endl;
        }
    }
};
int main()
{
    shapeFactory::registerShape("Circle", []()
                                { return new Circle; });
    shapeFactory::registerShape("Square", []()
                                { return new Square; });
```

```
    client c1;
    c1.drawing("Square");
    c1.drawing("Circle");
    c1.drawing("Triangle");
    c1.drawing("square");

    return 0;
}
```

The Registry-based Factory Pattern has several important properties that make it very useful for beginners to understand.

First, it provides extensibility, which means you can easily add new types of shapes to your program without having to change any of the code you already wrote. This is like being able to add new recipes to your cookbook without having to rewrite the existing ones.

Second, it offers centralized control, meaning all the logic for creating different shapes is kept in one place (the factory), making it much easier to manage and find when you need to make changes. Think of it like having all your tools organized in one toolbox instead of scattered around your garage.

Finally, it provides decoupling, which is a fancy way of saying that the parts of your code that use the shapes don't need to know exactly how those shapes are created - they just ask the factory for what they need, similar to how you order food at a restaurant without needing to know how the kitchen prepares it.

Pros and Cons of Factory Method Pattern

Following are some of the pros and cons of using the Factory Method Pattern.

Pros	Cons
Encapsulates object creation logic.	Can cause unnecessary complexity if not used properly.
Code become easy to reuse and maintain.	Multiple factory classes can make the codebase complex.
Supports the Open/Closed Principle that allows new types of objects to be added without modifying existing code.	Can make the code harder to understand for developers who does not know the pattern.
Decouples the client code from the concrete classes.	It may affect performance due to the additional layer of abstraction.

Real-World Applications of Factory Method Pattern

Following are some real-world applications of the Factory Method Pattern:

GUI Frameworks − Many GUI frameworks use factory methods to create UI components like buttons, text fields, etc.

Game Development − In game development, factory methods are used to create different types of game objects (e.g., enemies, power-ups) without specifying their concrete classes.

Database Connections − Factory methods can be used to create database connection objects, and allows them to be used interchangeably for different types of databases.

Abstract Factory Design Pattern in C++

The Abstract Factory is a creational design pattern that provides an interface with methods for a particular type of object, but it allows subclasses to alter the type of objects that will be created. The Abstract Factory pattern is used when we have multiple families of related or dependent objects and we want to create them without specifying their concrete classes.

In other words, it is a factory of factories. In the last chapter, we discussed the Factory Method pattern which is used to create objects of a single family. But in Abstract Factory pattern, we can create objects of multiple families.

Imagine you are building your dream house, and you have two choices for furniture: Modern or Victorian. Each type has its own matching set of items like a chair, a sofa, and a coffee table. Instead of picking each item one by one and possibly making a mix-match, you use a furniture factory that gives you the full set in one style. If you want Modern, you get all Modern items. If you want Victorian, you get all Victorian items. The Abstract Factory pattern works the same way: it gives you a whole set of related objects in one style, so everything fits together perfectly.

Implementation of Abstract Factory Pattern in C++

Following are the some important ways to implement Abstract Factory Pattern in C++:

Conventional Abstract Factory − It uses classic interface and concrete factories.

Registry-based Abstract Factory − It uses a registry of factories to create objects of different families.

Concrete Factory means a class which implements the Abstract Factory interface and is responsible for creating the objects of a particular family. The client code uses the abstract factory interface to create the objects without knowing the concrete classes.

Conventional Abstract Factory Implementation in C++

In this approach, we create an abstract factory interface which is used for defining the methods for creating family of related objects. Then, we create concrete factory classes that implement the abstract factory interface and provide the implementation for creating the objects of a particular family.

Example of Abstract Factory Pattern in C++

In the below example, we have an abstract factory interface FurnitureFactory that defines the methods for creating family of related objects. We have two concrete factory classes ModernFurnitureFactory and VictorianFurnitureFactory that implement the FurnitureFactory interface and provide the implementation for creating the objects of a particular family. The client code uses the abstract factory interface to create the objects without knowing the concrete classes.

```
#include <iostream>
using namespace std;

class chair
{
```

```
public:
    virtual string getType() = 0;
    virtual ~chair() {}
};

class sofa
{
public:
    virtual string getType() = 0;
    virtual ~sofa() {}
};

class modernChair : public chair
{
public:
    string getType() override
    {
        return "Modern chair";
    }
};

class victorianChair : public chair
{
public:
    string getType() override
    {
        return "Victorian chair";
    }
};

class modernSofa : public sofa
{
public:
    string getType() override
    {
        return "Modern sofa";
    }
};

class victorianSofa : public sofa
{
public:
    string getType() override
    {
        return "Victorian sofa";
    }
};

class Furniture
{
public:
    virtual chair *createChair() = 0;
    virtual sofa *createSofa() = 0;
};
```

```
class ModernFurniture : public Furniture
{
public:
    chair *createChair()
    {
        return new modernChair();
    }
    sofa *createSofa()
    {
        return new modernSofa();
    }
};

class VictorianFurniture : public Furniture
{
public:
    chair *createChair()
    {
        return new victorianChair();
    }
    sofa *createSofa()
    {
        return new victorianSofa();
    }
};

int main()
{

    Furniture *furniture = new ModernFurniture();
    chair *c1 = furniture->createChair();
    sofa *s1 = furniture->createSofa();
    cout << c1->getType() << endl;
    cout << s1->getType() << endl;
    delete c1;
    delete s1;
    return 0;
}
```

Registry-based Abstract Factory Implementation in C++

This is a similar approach we have discussed in the Factory Method pattern. In this approach, we create a registry of factories that can create objects of different families. The registry is a map that maps the family name to the corresponding factory. The client code uses the registry to get the factory for a particular family and then uses the factory to create the objects.

Example of Registry-based Abstract Factory Pattern in C++

Let's take an example of a car factory. Where, we have an abstract class CarFactory that defines the factory method createCar(). Then, we create concrete subclasses such as SUVFactory and SedanFactory that implement the factory method to create specific types of cars. We also create a registry of factories that maps the family name to the corresponding factory.

```cpp
#include <iostream>
#include <map>
using namespace std;

class chair
{
public:
    virtual string getType() = 0;
    virtual ~chair() {}
};

class sofa
{
public:
    virtual string getType() = 0;
    virtual ~sofa() {}
};

class modernChair : public chair
{
public:
    string getType() override
    {
        return "Modern chair";
    }
};

class victorianChair : public chair
{
public:
    string getType() override
    {
        return "Victorian chair";
    }
};

class modernSofa : public sofa
{
public:
    string getType() override
    {
        return "Modern sofa";
    }
};

class victorianSofa : public sofa
{
public:
    string getType() override
    {
        return "Victorian sofa";
    }
};
```

```
class Furniture
{
public:
    virtual chair *createChair() = 0;
    virtual sofa *createSofa() = 0;
};

class ModernFurniture : public Furniture
{
public:
    chair *createChair()
    {
        return new modernChair();
    }
    sofa *createSofa()
    {
        return new modernSofa();
    }
};

class VictorianFurniture : public Furniture
{
public:
    chair *createChair()
    {
        return new victorianChair();
    }
    sofa *createSofa()
    {
        return new victorianSofa();
    }
};

class furntureFactory
{
private:
    map<string, Furniture *> registory;

public:
    void registorFactory(string type, Furniture *f1)
    {
        registory[type] = f1;
    }

    Furniture *getFactory(string type)
    {
        return registory[type];
    }
};
int main()
{
    furntureFactory fc;
    fc.registorFactory("Modern", new ModernFurniture());
    fc.registorFactory("Victorian", new VictorianFurniture());
```

```
    Furniture *furniture = fc.getFactory("Victorian");
    chair *c1 = furniture->createChair();
    sofa *s1 = furniture->createSofa();
    cout << c1->getType() << endl;
    cout << s1->getType() << endl;
    delete c1;
    delete s1;
    return 0;
}
```

Prototype Design Pattern in C++

The Prototype Pattern is a part of the Creational Design Patterns family.

Consider a scenario where you have an object and your friend asks you to create its replica. How would you do that? You are probably thinking of creating a new object and then copying the values from the existing object to the new object. That is some good thinking! But, what if the existing objects have some fields that are private and not visible outside the class. And also, you would have to know the class of the existing object to create a new object of the same class, which makes your code dependent on the class of the object. In such cases, we need to use the Prototype Pattern.

The Prototype Pattern allows us to make copies from an actual object. It declares a common interface for all objects and those objects support cloning themselves. This means the object itself is responsible for creating a copy of itself. The client code can use the common interface to clone the object without knowing the class of the object. This interface has only one method, which is clone(). The clone() method is responsible for creating a copy of the object and returning it to the client code.

Whichever object that supports cloning is called a prototype. If your objects have too many feilds and many different types of configurations, then you can just clone them.

1. Basic Prototype Pattern in C++

```
#include <iostream>
#include <memory>
using namespace std;

class virtualMachine
{
public:
    virtual unique_ptr<virtualMachine> clone() const = 0;
    virtual void showConfig() const = 0;
    virtual ~virtualMachine() {}
};

class linuxVm : public virtualMachine
{
    string os;
    int ram;
    int cpu;

public:
    linuxVm(string os, int ram, int cpu) : os(move(os)), ram(ram), cpu(cpu) {}
```

```
    unique_ptr<virtualMachine> clone() const override
    {
        cout << "cloneing linux vm..." << endl;
        return make_unique<linuxVm>(*this);
    }

    void showConfig() const override
    {
        cout << "os: " << os << endl;
        cout << "ram: " << ram << endl;
        cout << "cpu: " << cpu << endl;
    }
};
int main()
{
    unique_ptr<virtualMachine> linuxv1 = make_unique<linuxVm>("ubuntu 25", 8, 4);
    linuxv1->showConfig();
    unique_ptr<virtualMachine> cloneLinux = linuxv1->clone();
    cloneLinux->showConfig();
    return 0;
}
```

2. Prototype Pattern with Registry in C++

```
#include <iostream>
#include <memory>
#include <map>

using namespace std;
class Element
{
public:
    virtual unique_ptr<Element> clone() const = 0;
    virtual void render() const = 0;
    virtual ~Element() {}
};

class buttonEle : public Element
{
    string label;

public:
    buttonEle(string label1) : label(move(label1)) {}

    unique_ptr<Element> clone() const
    {
        cout << "cloning buttonEle..." << endl;
        return make_unique<buttonEle>(*this);
    }

    void render() const
    {
```

```
        cout << "redering button : " << label << endl;
    }
};

class textBoxEle : public Element
{
    string label;

public:
    textBoxEle(string label1) : label(move(label1)) {}

    unique_ptr<Element> clone() const
    {
        cout << "cloning textBoxEle..." << endl;
        return make_unique<textBoxEle>(*this);
    }

    void render() const
    {
        cout << "redering textBox : " << label << endl;
    }
};

class prototypeRegistory
{
private:
    map<string, unique_ptr<Element>> registory;

public:
    void registorElement(string type, unique_ptr<Element> ele)
    {
        registory[type] = move(ele);
    }

    unique_ptr<Element> getPrototype(string type)
    {
        auto it = registory.find(type);
        if (it != registory.end())
        {
            return it->second->clone();
        }
        return nullptr;
    }
};

int main()
{
    prototypeRegistory registry;
    registry.registorElement("BUTTON", make_unique<buttonEle>("Submit"));
    registry.registorElement("TextBox", make_unique<textBoxEle>("Enter your name"));

    auto clonedButton = registry.getPrototype("BUTTON");
    auto clonedTextBox = registry.getPrototype("TextBox");
```

```
    // Show configurations
    cout << "\nCloned UI Elements:" << endl;
    if (clonedButton)
        clonedButton->render();
    if (clonedTextBox)
        clonedTextBox->render();

    return 0;
}
```

Structural Design pattern

The Structural Design Patterns is a way of combining or arranging different classes and objects to form larger and complex structures to solve a particular problem in a better way. These patterns help in simplifying the design by identifying simple ways to realize relationships between entities.

Bridge Design Pattern in C++

Bridge pattern separates abstraction from implementation so both can evolve independently using composition instead of inheritance.

Bridge Design Pattern is a structural design pattern which helps to separate the following two aspects of a software system −

Abstraction − Provides the interface for client code, hiding the implementation details.

Implementation − Provides the concrete implementation of the abstraction.

By separating the above aspects, we can make both the abstraction and implementation independently expandable. This means that we can add new abstractions and implementations without affecting existing code.

Think of it like a Coffee shop. You love coffee but alone it would be plain. Sometimes you want Espresso, sometimes Latte, and sometimes you want to add Milk, Sugar, or Honey.

The coffee type (Espresso, Latte) is the abstraction, and the add-ons (Milk, Sugar, Honey) are the implementations. Instead of creating separate classes for every possible combination (like "EspressoWithMilk" or "LatteWithSugar"), the Bridge Pattern lets coffee delegate to add-ons through composition. This makes the system more flexible, that allows new coffee types or new add-ons to be added without changing existing code, and avoids the explosion of subclasses.

Abstraction

The Abtraction here is not same as in Abstract classes or Interfaces. It is a higher level of abstraction that separates the interface from the implementation.

We only see "what" we need to see, instead of "how" it is implemented. The Abstraction contains a reference to the Implementor.

Implementor

The Implementor defines the interface for implementation classes. This interface doesn't have to be exactly match to Abstraction's interface; in fact, the two interfaces can be different. Typically, the Implementor interface provides only primitive operations, and Abstraction defines higher-level operations based on those primitives.

We don't care about "what" it does, we care about "how" it is done. The Implementor doesn't contain any reference to the Abstraction.

```
#include <iostream>
using namespace std;

class AddOn
{
public:
   virtual void add() = 0;
   virtual ~AddOn() {}
};

class milk : public AddOn
{
public:
       void add()
       {
       cout << "with Milk";
   }
};
class sugar : public AddOn
{
public:
   void add()
   {
       cout << "with sugar";
   }
};
class honey : public AddOn
{
public:
   void add()
   {
       cout << "with honey";
   }
};

class coffe
{
protected:
   AddOn *addon;

public:
   coffe(AddOn *a) : addon(a) {}
   virtual void serve() const = 0;
   virtual ~coffe() {}
```

```
};

class Expresso : public coffe
{
public:
   Expresso(AddOn *addon) : coffe(addon) {}
   void serve() const
   {
      cout << "Serving Expresso ";
      addon->add();
      cout << endl;
   }
};

class Latte : public coffe
{
public:
   Latte(AddOn *addon) : coffe(addon) {}
   void serve() const
   {
      cout << "Serving Latte ";
      addon->add();
      cout << endl;
   }
};

int main()
{
   AddOn *milk1 = new milk();
   AddOn *sugar1 = new sugar();
   coffe *coffee = new Expresso(milk1);
   coffee->serve();
   coffe *coffeel = new Latte(sugar1);
   coffeel->serve();
   delete milk1;
   delete sugar1;
   delete coffee;
   delete coffeel;
   return 0;
}
```

Decorator Design Pattern in C++

A decorator is a Structural Design Pattern that allows us to add new functionalities to an existing object without changing or disturbing its structure. It lets us attach new behaviors to an object by placing that object inside another special wrapper object that contains the new behavior.

Let's understand in detail with an example. Imagine you have a simple text editor application with basic text features like writing and saving or editing text. Now, your manager wants you to add some new features like spell checking and grammar checking. If you are using the Decorator Design Pattern, you can easily add these new features without changing the existing code of the text editor. You can create

separate decorator classes for spell checking and grammar checking that wrap around the original text editor class.

Components of Decorator Design Pattern

Component − This is an interface or abstract class that defines the common methods for both the concrete component and the decorators. In our example, this would be the interface for the text editor.

Concrete Component − This is the class that implements the component interface and represents the original object that we want to add new functionalities to. In our example, this would be the basic text editor class.

Decorator − This is an abstract class that also implements the component interface and has a reference to a component object. The decorator class is responsible for adding new functionalities to the component object. In our example, this would be the abstract decorator class for the text editor.

Concrete Decorators − These are the classes that extend the decorator class and implement the new functionalities. In our example, these would be the spell checker and grammar checker classes that add their respective functionalities to the text editor.

```
#include <iostream>
using namespace std;

class MusicPlayer
{
public:
   virtual void play() = 0;
   virtual ~MusicPlayer() {}
};

class BasicMusicPlayer : public MusicPlayer
{
public:
   void play()
   {
      cout << "Playing Music" << endl;
   }
};

class MusicPlayerDecorator : public MusicPlayer
{
protected:
   MusicPlayer *player;

public:
   MusicPlayerDecorator(MusicPlayer *p) : player(p) {}
   void play()
   {
      player->play();
   }
   ~MusicPlayerDecorator()
   {
      delete player;
   }
};
```

```
class ShufleDecorator : public MusicPlayerDecorator
{
public:
   ShufleDecorator(MusicPlayer *p) : MusicPlayerDecorator(p) {}
   void play()
   {
      cout << "Shuffe Music.." << endl;
      player->play();
   }
};
class RepeatDecorator : public MusicPlayerDecorator
{
public:
   RepeatDecorator(MusicPlayer *p) : MusicPlayerDecorator(p) {}
   void play()
   {
      cout << "Repeat current Music.." << endl;
      player->play();
   }
};

int main()
{
   MusicPlayer *player = new BasicMusicPlayer();
   MusicPlayer *shuflePlayer = new ShufleDecorator(player);
   MusicPlayer *repeatPlayer = new RepeatDecorator(shuflePlayer);
   player->play();
   shuflePlayer->play();
   repeatPlayer->play();
   return 0;
}
```

When to Use Decorator Design Pattern?

Following are some scenarios where you should consider using the Decorator Design Pattern −

To add more functionalities to an object dynamically.

When you want to avoid class explosion due to multiple combinations of features.

To stick to the Single Responsibility Principle by dividing functionalities into separate classes.

For adding responsibilities to individual objects without affecting other objects of the same class.

To enhance the behavior of an object at runtime.

Proxy Design Pattern in C++

Proxy Design Pattern is a Structural Design Pattern that allows us to provide a substitute or we can say a placeholder for another object. In Proxy Design Pattern, a class represents the functionality of another class. A proxy receives client requests, does some work (access control, caching, etc.) and then passes the request to a service object (the original object).

Proxy Design Pattern is used when we need to create a wrapper for an object to control access to it. It is also used to add additional functionality to an object without changing its code.

Types of Proxy Design Pattern

Following are the different types of Proxy Design Pattern −

Remote Proxy − Lets you use something that's far away, maybe on another computer.

Virtual Proxy − Creates the real thing only when you actually need it, so it saves time and memory.

Protection Proxy − Checks if you're allowed to use something before giving you access.

Caching Proxy − Stores results and gives them back quickly if you ask for the same thing again.

Logging Proxy − Keeps a record of what happens when you use the real object.

Synchronization Proxy − Makes sure only one person can use something at a time to avoid clashes.

Smart Reference Proxy − Adds a few extra steps when you use something, like counting how many times it's used or loading it only when needed.

Components of Proxy Design Pattern

Subject − The basic rules or actions that both the real object and the helper use.

RealSubject − The real object that does the main job.

Proxy − The helper that talks for the real object and controls who can use it

Steps to Implement Proxy Design Pattern

Define the Subject interface that declares the common operations for both the real object and the proxy.

Create the RealSubject class that implements the Subject interface and simulates a database connection.

Create the Proxy class that also implements the Subject interface and controls access to the RealSubject. It will add caching functionality.

Use the Proxy class in the client code to access the database.

When to Use Proxy Design Pattern?

Control who can use something.

Add saving results or keeping records without changing the main object.

Delay making the real thing until it is needed.

Keep something safe from people who shouldn't use it.

Manage how and when something is used.

```
#include <iostream>
using namespace std;
```

```cpp
// subject
class Database
{
public:
    virtual void query(const string &sql) = 0;
    virtual ~Database() {}
};
// real subject
class RealDatabase : public Database
{
public:
    void query(const string &sql) override
    {
        cout << "Executing query : " << sql << endl;
    }
};

class ProxyDatabase : public Database
{
private:
    bool isCatched;
    string catchedResult;
    RealDatabase *realDB;

public:
    ProxyDatabase() : realDB(nullptr), isCatched(false) {}
    void query(const string &sql) override
    {
        if (!isCatched)
        {
            if (realDB == nullptr)
            {
                realDB = new RealDatabase();
            }
            isCatched = true;
        }
        else
        {
            cout << "Executing catched Result.." << endl;
        }
        realDB->query(sql);
    }
    ~ProxyDatabase()
    {
        delete realDB;
    }
};
int main()
{
    Database *db = new ProxyDatabase();
    db->query("SELECT* FROM users");
    db->query("SELECT* FROM users");
    delete db;
    return 0;
```

```
}
```

Adapter Design Pattern in C++

Adapter is a part of Structural Design Patterns. Adapter pattern works like a bridge between two incompatible interfaces. It involves a single class which is responsible for joining the functionalities of interfaces that are independent and not compatible with each other. This pattern is also known as Wrapper.

Adapter works as a middleman of two things that do not go along together. For example, your laptop has a USB port and you have a SD card that you want to connect to your laptop. Here, USB and SD are two incompatible interfaces. To make them compatible, you need an adapter that can convert SD to USB.

Types of Adapter Pattern

There are two types of Adapter pattern −

Class Adapter Pattern − In this, we use multiple inheritance to adapt one interface to another.

Object Adapter Pattern − We use composition to adapt one interface to another.

Class Adapter Pattern in C++

We use Class Adapter Pattern when we want to adopt one interface to another using multiple inheritance. In this pattern, the adapter class inherits from both the target interface and the adaptee class. This way, the adapter can override methods from the target interface and use methods from the adaptee class to provide the functionality we want.

```
#include <iostream>

using namespace std;

class ImediaPlayer
{
public:
   virtual void playMP3(string filename) = 0;
};

class OldCassatePlayer
{
public:
   void playCassate(string tape)
   {
      cout << "Playing cassette tape: " << tape << endl;
   }
};

class cassateToMP3Adapter : public ImediaPlayer, private OldCassatePlayer
{
public:
   void playMP3(string filename) override
   {
```

```
        string tape = filename + ".cassate";
        playCassate(tape);
    }
};
int main()
{
    ImediaPlayer *IP = new cassateToMP3Adapter();
    IP->playMP3("DIl_mang_eha_hai_mohlat");
    delete IP;
    return 0;
}
```

Object Adapter Pattern in C++

We use Object Adapter Pattern when we want to adopt one interface to another using composition. In this pattern, the adapter class contains an instance of the adaptee class and implements the target interface. The adapter class translates calls from the target interface to the adaptee class.

```
#include <iostream>

using namespace std;

class ImediaPlayer
{
public:
    virtual void playMP3(string name) = 0;
};

class cassatePlayer
{
public:
    void playCassate(string filename)
    {
        cout << "Playing cassate: " << filename << endl;
    }
};

class cassateToMP3 : public ImediaPlayer
{
private:
    cassatePlayer *cp;

public:
    cassateToMP3(cassatePlayer *cp1) : cp(cp1) {}
    void playMP3(string name) override
    {
        string tape = name + ".cassate";
        cp->playCassate(tape);
    }
};
int main()
{
    cassatePlayer *cp = new cassatePlayer();
```

```
    ImediaPlayer *ctm = new cassateToMP3(cp);
    ctm->playMP3("jasmine sanders");
    ctm->playMP3("saname re");
    delete ctm;
    delete cp;
    return 0;
}
```

Composite Design Pattern in C++

Composite Design Pattern is a structural design pattern that allows you to compose objects into tree like structures that represent part-whole hierarchies. This pattern lets clients treat any individual object and a composition of objects uniformly.

Sounds complicated? Let's break it down with an example.

Suppose you are building a graphic design application which allows users to create and manipulate various shapes like circles, rectangles, and lines.

You also want to allow user to group the shapes together to form new designs(like a house made of rectangles and triangles, or a tree made of circles and lines).

With the Composite Design Pattern, you can treat both individual shapes and groups of shapes uniformly. This means that you can perform operations like move, resize, or draw on both single shapes and groups of shapes without worrying about their specific types.

Components of Composite Design Pattern

The Composite Pattern is made up of multiple components. Take a look at the following image. Here, we can see the four main components of the Composite Design Pattern. The Component is an interface that has common methods for both Leaf and Composite. The Leaf is like a loner who doesn't have any children, while the Composite is like a parent who can have multiple children (which can be either Leaf or other Composite). And, lastly we have the Client, which is like the boss who interacts with the Component interface to perform operations on both Leaf and Composit

```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

// component
class FileSystemComponent
{
public:
    virtual void showDetails() = 0;
    virtual void add(shared_ptr<FileSystemComponent> component) {}
};

// leaf
class File : public FileSystemComponent
{
private:
    string name;
```

```
public:
   File(string filename) : name(filename) {}
   void showDetails() override
   {
      cout << "FileName : " << name << endl;
   }
};

// composite
class Directory : public FileSystemComponent
{
private:
   string DirName;
   vector<shared_ptr<FileSystemComponent>> children;

public:
   Directory(string dir) : DirName(dir) {}
   void add(shared_ptr<FileSystemComponent> component)
   {
      children.push_back(component);
   }
   void showDetails() override
   {
      cout << "Directory : " << DirName << endl;
      for (auto child : children)
      {
         child->showDetails();
      }
   }
};
int main()
{
   shared_ptr<FileSystemComponent> file1 = make_shared<File>("File1.txt");
   shared_ptr<FileSystemComponent> file2 = make_shared<File>("File2.txt");
   shared_ptr<FileSystemComponent> dir1 = make_shared<Directory>("Dir1");
   shared_ptr<FileSystemComponent> dir2 = make_shared<Directory>("Dir2");

   dir1->add(file1);
   dir2->add(file2);
   dir1->showDetails();
   dir2->showDetails();
   return 0;
}
```

When to Use Composite Design Pattern?

You should consider using the Composite Design Pattern in the following scenarios −

When you have a part-whole hierarchy in your application.

To represent tree structures, such as file systems or organizational hierarchies.

For treating individual objects and compositions of objects uniformly.

While simplifying client code by allowing it to interact with a single interface.

To add new types of components without changing existing code.

When you want to manage complex structures more easily.

Performing operations on both individual objects and groups of objects.

Flyweight Pattern | C++ Design Patterns

A flyweight pattern is a structural design pattern used to optimize memory usage and performance when dealing with a large number of objects that share some common characteristics. It achieves this by separating an object's intrinsic state (shared among multiple objects) from its extrinsic state (unique to each object) and storing the intrinsic state externally, typically within a Flyweight factory. This pattern is particularly useful when you need to create a significant number of similar objects and want to minimize the memory footprint.

Key Components

1. **Flyweight Interface or Base Class**: This defines the methods for accessing and manipulating the intrinsic state.
2. **Concrete Flyweight**: Implementations of the Flyweight interface that store and manage the intrinsic state. They are typically lightweight and capable of being shared.
3. **Flyweight Factory**: A factory class responsible for creating and managing flyweight objects. It ensures that flyweights are shared and reused when possible.

Use Cases

- Text processing, where individual characters can be represented as flyweights, and the text document contains references to these characters.
- Graphic design applications, where graphical elements like fonts, icons, or colors can be represented as flyweights.
- Game development, where objects like trees, rocks, or bullets can be managed as flyweights to reduce memory consumption.

```
#include <iostream>
#include <map>
#include <memory>
using namespace std;
/// flyweight
class TreeType
{
private:
    string name;
    string color;
    string texture;

public:
    TreeType(string t_name, string t_color, string t_texture) : name(t_name), color(t_color), tex-
ture(t_texture) {}

    void draw()
    {
        cout << "Drawing tree " << name << " with color " << color << " and with texture " << texture <<
endl;
    }
```

```
};

class TreeFactory
{
private:
    static map<string, unique_ptr<TreeType>> treeTypes;

public:
    static TreeType *getTreeType(string name, string color, string texture)
    {
        string key = name + color + texture;
        if (treeTypes.find(key) == treeTypes.end())
        {
            treeTypes[key] = make_unique<TreeType>(name, color, texture);
        }
        return treeTypes[key].get(); //.get() returns the raw pointer (TreeType*) managed by the unique_ptr.
    }
};

map<string, unique_ptr<TreeType>> TreeFactory::treeTypes;

int main()
{

    TreeType *t1 = TreeFactory::getTreeType("mango", "brown", "rough");
    TreeType *t2 = TreeFactory::getTreeType("banana", "green", "smooth");
    t1->draw();
    t2->draw();

    return 0;
}
```

Command Design Pattern in C++

The Command Design Pattern is a way to turn a request—like asking a light to turn on—into its own object. This means you can treat actions as things you can pass around, save for later, or even undo if you want.

In simple terms, it lets you wrap up an action and all the details it needs, so you can run it whenever you like. This makes your code more flexible and easier to manage, especially when you want to add features like undo or keep a history of what happened.

For example, in a text editor, actions like Copy, Paste, and Undo can each be their own command object. This makes it easy to run them, line them up, or take them back if you change your mind.

In the following image, you can see a remote control sending commands to a light source. The remote control (Invoker) doesn’t know how the Light (Receiver) works; it just tells it to turn on or off using Command objects.

Components of Command Design Pattern

Following are the main components of the Command Design Pattern −

Command − This is just an interface or abstract class with a method like execute(). It's a common way to say, "Here's what a command should look like."

ConcreteCommand − These are the actual command classes. Each one does something specific, like turning a light on or off. They know which object to work with and what action to perform.

Client − This is the part of your code that puts everything together. It creates the command objects and tells them which things they should control.

Invoker − Think of this as the remote control. It doesn't know what the command does, it just knows how to ask the command to do its job.

Receiver − This is the real worker. It's the object that actually does the work when a command is executed, like the light that turns on or off.

```
#include <iostream>
using namespace std;
// command interface
class command
{
public:
   virtual void exectue() = 0;
   virtual ~command() {}
};
// reciever class
class Light
{
public:
   void on()
   {
      cout << "lght On..." << endl;
   }
   void off()
   {
      cout << "light off..." << endl;
   }
};
// concrete class for light on
class LightOnCommand : public command
{
private:
   Light *l;

public:
   LightOnCommand(Light *light) : l(light) {}
   void exectue() override
   {
      l->on();
   }
};
// concrete class for light off
class LightOffCommand : public command
```

```
{
private:
    Light *l;

public:
    LightOffCommand(Light *light) : l(light) {}
    void exectue() override
    {
        l->off();
    }
};
// invoker class
class RemoteCommand
{
private:
    command *cmd;

public:
    void setCommand(command *c1)
    {
        cmd = c1;
    }
    void pressButton()
    {
        cmd->exectue();
    }
};
int main()
{
    Light *light = new Light();
    LightOnCommand *lightOn = new LightOnCommand(light);
    LightOffCommand *lightOff = new LightOffCommand(light);
    RemoteCommand *remote = new RemoteCommand();
    remote->setCommand(lightOn);
    remote->pressButton();
    remote->setCommand(lightOff);
    remote->pressButton();
    delete light;
    delete lightOn;
    delete lightOff;
    delete remote;

    return 0;
}
```

Facade Design Pattern in C++

Facade Design Pattern is a structural design pattern that provides a single and simplified interface to a complex system of classes, library or framework. It orginizes a complicated system by adding a simple interface to it.

Instead of having multiple objects to interact with, the client can interact with a single facade object which internally manages the interactions with the complex system.

Imagine the following big "Facade" building is like a magic "Play" button for a whole toy factory. You just press that one button, and inside, all the machines start working together automatically to make toys, without you needing to know how each one works.

Components of Facade Design Pattern

The Facade Design Pattern consists of the following three parts −

Facade − This is the main part the user talks to. It provides simplified interface to bunch of subsystems. It act as a middleman between client and complex system.

Complex System − Many classes that do the real work. These classes or we also call them subsystems are the ones who actually do the tasks. They can be hard to use directly without the facade.

Client − The user. Talks only to the facade, not the complex system. Means, the client doesn't need to know about the complex system, it just uses the facade to get things done.

```
#include<iostream>
using namespace std;

class TV
{
   public:
      void on()
      {
         cout<<"TV on.."<<endl;
      }
      void off()
      {
         cout<<"TV off.."<<endl;
      }
};

class SoundSystem
{
      public:
            void on()
            {
                  cout<<"SoundSystem on.."<<endl;
            }
            void off()
            {
                  cout<<"SoundSystem off.."<<endl;
            }
};


class DVDPlayer
{
      public:
            void on()
            {
                  cout<<"DVDPlayer on.."<<endl;
            }
            void off()
```

```
            {
                cout<<"DVDPlayer off.."<<endl;
            }

        void play(string movie)
        {
            cout<<"Playing movie "<< movie << endl;
        }
};

class HomeTheatreFacade
{
    private:
        TV* tv;
        SoundSystem* soundsystem;
        DVDPlayer* DVD;
    public:
        HomeTheatreFacade(TV* tv, SoundSystem* ss, DVDPlayer* dvd) : tv(tv), soundsystem(ss),
DVD(dvd) {}

        void watchMovie(string movie)
        {
            tv->on();
            soundsystem->on();
            DVD->on();
            DVD->play(movie);
        }
            void endMovie()
            {
                cout << "Shutting down the home theater..." << endl;
                DVD->off();
                soundsystem->off();
                tv->off();
            }

};

int main()
{
    TV *tv = new TV();
    SoundSystem* soundsystem = new SoundSystem();
    DVDPlayer* DVD = new DVDPlayer();
    HomeTheatreFacade* htf = new HomeTheatreFacade(tv, soundsystem, DVD);

    htf->watchMovie("3 Idiots");
    htf->endMovie();
    delete tv;
    delete DVD;
    delete htf;
    delete soundsystem;
    return 0;
}
```

When to Use Facade Design Pattern?

Following are some scenarios where you should consider using the Facade Design Pattern −

To make a complex system easier to use.

For reducing direct connections between client and system.

To organize code better.

Keeping client and system loosely connected.

To hide system details from the client.

Making a library or framework simpler to use.

Offer a simple interface for common tasks.

Behavioural Design Patterns in C++

Behavioural design patterns are ways of organising how different objects in a program communicate with each other. When a program becomes large, many objects need to exchange information or react to certain actions. If this communication is random or unplanned, the code becomes confusing. Behavioural design patterns give a simple and organised approach so every object knows when to speak, when to listen, and what its role is. This makes the whole program easier to understand and work with.

A good way to understand this is to picture a restaurant. A restaurant runs smoothly only because everyone inside knows how to behave and how to talk to each other. The waiter takes orders from customers, the chef cooks food, the cashier handles bills, and the manager watches over everything. The waiter does not suddenly start cooking, the chef does not run to take orders, and the customers don't go into the kitchen to ask what is happening. Everyone has a specific job and a clear way to interact. This simple structure keeps the restaurant calm even when many people are involved.

Now imagine if the behaviour rules inside the restaurant didn't exist. Customers might walk straight into the kitchen, chefs might run to tables arguing about orders, waiters might forget who to serve because everyone is shouting instructions. It would be a disaster. The same thing happens inside a program when communication is not planned properly. Behavioural design patterns prevent this problem by defining clean paths of communication, just like a restaurant defines who talks to whom and when.

Iterator Design Pattern in C++

Iterator Design Pattern is a behavioral design pattern that helps you move through a collection of items-like a list, stack, or tree-without needing to know how that collection is built on the inside. Imagine you have a box full of different objects, and you want to look at each one, one by one, without having to open the box and figure out how everything is arranged. The Iterator pattern gives you a simple way to do this, making it easier to work with all kinds of collections in a consistent way.

This pattern is especially useful when your application uses many different types of collections. Instead of writing separate code to loop through each type, you can use the Iterator pattern to create a common approach for all of them. This not only saves time but also makes your code cleaner and easier to update in the future. If you ever change how a collection is stored, you won't have to rewrite the code that goes through its elements.

Key Components of the Iterator Pattern

Iterator − Think of this as a guide that helps you move through a collection, one item at a time. The Iterator lays out the basic steps you need, like checking if there are more items to see (hasNext()) and getting the next item in line (next()). It doesn't care what's inside the collection or how it's organized—it just gives you a simple way to look at each item, one after another.

Concrete Iterator − This is the actual helper that knows how to walk through a specific collection, like a list of books or a playlist of songs. It remembers where you are in the collection and knows how to get to the next item. For example, if you're flipping through a stack of cards, the Concrete Iterator keeps track of which card you're on and helps you move to the next one.

Aggregate − This is just a fancy word for any group or collection of items—like a bookshelf full of books or a playlist full of songs. The Aggregate sets the rule that says, "If you want to look through my items, you need to use an iterator." It provides a way to get an iterator so you can start exploring the collection.

Concrete Aggregate − This is the actual collection itself, like your bookshelf or playlist. It holds all the items and knows how to give you an iterator that can walk through them. For example, your Library class might store a bunch of book titles and provide a way to get an iterator so you can read each title one by one.

```
#include <iostream>
#include <vector>
using namespace std;
// Iterator
class Iterator
{
public:
   virtual bool hasNext() = 0;
   virtual string Next() = 0;
   virtual ~Iterator() = default;
};
// Song Iterator
class SongIterator : public Iterator
{
private:
   vector<string> songs;
   int position;

public:
   SongIterator(vector<string> s, int pos) : songs(s), position(pos) {}
   bool hasNext() override
   {
      return position < songs.size();
   }
   string Next() override
   {
      return songs[position++];
   }
};
// Aggregate
class Aggregate
{
public:
   virtual Iterator *createIterator() = 0;
};
```

```
class PlayList : public Aggregate
{
private:
   vector<string> songs;

public:
   void addSong(string name)
   {
      songs.push_back(name);
   }
   Iterator *createIterator() override
   {
      return new SongIterator(songs, 0);
   }
};
int main()
{
   PlayList playlist;
   playlist.addSong("dil mang rha hai mohlat");
   playlist.addSong("main rahoon ya na rahoon");
   playlist.addSong("sanam re");

   Iterator *it = playlist.createIterator();
   cout << "My Playlist songs are..." << endl;
   while (it->hasNext())
   {
      cout << it->Next() << endl;
   }
   delete it;
   return 0;
}
```

Mediator Design Pattern in C++

Mediator Design Pattern is a way to make communication between different objects or classes much simpler. Instead of having every object talk directly to every other object, which can get very confusing, we use a special class called the mediator. The mediator acts like a middleman and handles all the messages between objects. This makes the code easier to manage and change later because the objects don't need to know about each other.

Let's make this even clearer with a simple example from real life. Imagine an airport. There are many airplanes, and they all need to communicate about when to take off or land. If every airplane tried to talk to every other airplane, it would be chaos! Instead, there is a control tower. The control tower is the mediator. All airplanes talk only to the control tower, and the tower tells each airplane what to do. This keeps everything organized and safe.

Steps to Implement Mediator Design Pattern in C++

Step 1: Create the Mediator Interface − This is like making a set of rules for how the middleman (mediator) will help others talk to each other. It just says what the mediator should be able to do, but not how.

Step 2: Make the Real Mediator Class − Here, you actually build the middleman. This class knows about everyone who needs to talk and helps them send messages to each other.

Step 3: Create the Colleague Interface − This is a simple plan for all the people (objects) who want to talk to each other. It says what each one should be able to do, like send a message.

Step 4: Build the Real Colleague Classes − Now you make the actual people (objects) who will use the mediator to talk. Each one uses the middleman instead of talking directly to others.

Step 5: See How It Works Together − Put everything together and show how the objects use the mediator to send messages. This is where you see the pattern in action, just like people using a group chat to talk instead of calling each other one by one.

```
#include <iostream>
#include <vector>
using namespace std;
// mediator Inteface
class ChatMediator
{
public:
   virtual void showMessage(string name, string message) = 0;
};
// concrete mediator
class ChatRoom : public ChatMediator
{
private:
   vector<string> users;

public:
   void addUser(string user)
   {
      users.push_back(user);
   }
   void showMessage(string name, string msg) override
   {
      cout << name << " : " << msg << endl;
   }
};
// colleague Interface
class User
{
protected:
   ChatMediator *med;
   string name;

public:
   User(ChatMediator *m, string n) : med(m), name(n) {}
   virtual void sendMessage(string msg) = 0;
};
// Concrete colleague
class ChatUser : public User
{
public:
   ChatUser(ChatMediator *m, string n) : User(m, n) {}
   void sendMessage(string msg)
   {
```

```
        med->showMessage(name, msg);
    }
};
int main()
{
    ChatRoom chatRoom;
    ChatUser user1(&chatRoom, "Alice");
    ChatUser user2(&chatRoom, "Bob");
    ChatUser user3(&chatRoom, "Charlie");
    chatRoom.addUser("Alice");
    chatRoom.addUser("Bob");
    chatRoom.addUser("Charlie");
    user1.sendMessage("Hello, everyone!");
    user2.sendMessage("Hi Alice!");
    user3.sendMessage("Hey folks, what's up?");

    return 0;
}
```

Memento Design Pattern in C++

Memento Design Pattern is a behavioral design pattern that allows an object to capture its internal state and save it externally so that it can be restored later without violating encapsulation. This pattern is particularly useful when you want to implement features like undo/redo functionality in applications.

For example, take an application where users can edit documents. By using the Memento Design Pattern, the application can save the state of the document at various points in time. If the user wants to undo their last action, the application can restore the document to its previous state using the saved memento. This way, the internal state of the document is preserved without exposing its details to other parts of the application.

Components of Memento Design Pattern

The Memento Design Pattern consists of three main components −

Originator − The main object you want to save and restore. It can make a memento (a snapshot of its current state) and later use that memento to go back to how it was before. For example, in a text editor, the Originator is the editor itself that holds your text.

Memento − A special object that remembers the state of the Originator at a certain time. Once made, its contents cannot be changed, so you can always trust it to bring things back exactly as they were. Think of it like a bookmark or a save point.

Caretaker − The helper that keeps track of all the mementos (saved states). It stores them safely and can ask the Originator to go back to a previous state when needed. The Caretaker never looks inside the mementos or changes them; it just holds onto them and gives them back when you want to undo or redo something.

```
#include <iostream>
#include <vector>
using namespace std;
// memmento
class TextMemento
```

```
{
    string textState;

public:
    TextMemento(string text) : textState(text) {}
    string getState() const
    {
        return textState;
    }
};
// Orginator
class TextEditor
{
    string text;

public:
    void type(string newText)
    {
        text += newText;
    }
    string getText() const
    {
        return text;
    }
    TextMemento save()
    {
        return TextMemento(text);
    }
    void restore(TextMemento memento)
    {
        text = memento.getState();
    }
};
// careTker
class EditorHstory
{
    vector<TextMemento> history;

public:
    void push(TextMemento mem)
    {
        history.push_back(mem);
    }
    TextMemento pop()
    {
        if (!history.empty())
        {
            TextMemento memento = history.back();
            history.pop_back();
            return memento;
        }
        return TextMemento("");
    }
};
```

```
int main()
{
   EditorHstory history;
   TextEditor editor;
   editor.type("lord ");
   history.push(editor.save());
   editor.type("shrikrishna");
   history.push(editor.save());
   cout << "get currentText : " << editor.getText() << endl;

   editor.restore(history.pop());
   cout << "After Undo: " << editor.getText() << endl;

   editor.restore(history.pop());
   cout << "After Undo: " << editor.getText() << endl;

   return 0;
}
```

When to Use Memento Design Pattern

The Memento Design Pattern is very useful in everyday programs where you want to let people fix mistakes or go back to something they did before. It is especially helpful for adding undo and redo features, which make programs much easier and safer to use.

This pattern is also great for keeping a history of changes. People can easily go back to an earlier version of their work if they change their mind or want to try something different. It helps make programs more friendly and less stressful to use.

Observer Design Pattern in C++

Observer Design Pattern is a way to make sure that when something important changes in one part of your program, all the other parts that care about it get updated right away. It's called a one-to-many relationship because one main object (the subject) can have many other objects (the observers) watching it. When the subject changes, it tells all its observers so they can update themselves. This is super helpful when you want different parts of your program to always show the latest information, but you don't want them to be tightly connected or mixed together.

For example, imagine you have a weather app with several display screens showing the current weather. The weather data is the subject, and the display screens are the observers. Whenever the weather changes, the weather data tells all the screens to update, so everyone always sees the newest weather. This way, you don't have to update each screen by hand because they all get the news automatically.

Sometimes people mix up the Observer Design Pattern with the Mediator Design Pattern. The difference is important: in the Observer Pattern, the subject keeps a list of observers and tells them when something changes. In the Mediator Pattern, there's a special mediator object in the middle that helps different objects talk to each other, so they don't talk directly. The Observer Pattern is all about keeping things in sync, while the Mediator Pattern is about organizing communication.

Components of Observer Design Pattern

The Observer Design Pattern consists of the following main components −

Subject − This is the main object that has some important information or state. When something changes inside the subject, it lets others know about it. The subject keeps a list of all the people (observers) who want to know when something changes. For example, in a weather app, the weather data is the subject because it knows the latest weather and tells others when it changes.

Observer − An observer is anyone who wants to know when the subject changes. Observers sign up with the subject, and whenever there is an update, the subject tells all its observers. In our weather app example, the display screens are observers because they show the latest weather whenever the weather data changes.

Concrete Subject − This is a real, working version of the subject. It actually stores the data and knows how to tell the observers when something changes. In the weather app, the WeatherData class is the concrete subject because it keeps the weather info and notifies the displays.

Concrete Observer − This is a real, working version of an observer. It knows how to react when the subject tells it about a change. In the weather app, the WeatherDisplay class is a concrete observer because it updates what it shows whenever the weather changes.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
// observer Interface
class Observer
{
public:
   virtual void update(float temperature, float humidity, float pressure) = 0;
   virtual ~Observer() {}
};
// Concrete Observer
class WheatherDisplay : public Observer
{
   float temperature;
   float humidity;
   float pressure;

public:
   void update(float temp, float hum, float pre) override
   {
      temperature = temp;
      humidity = hum;
      pressure = pre;
      display();
   }
   void display()
   {
      cout << "temperature : " << temperature << " humidity : " << humidity << " pressure : " << pres-
sure << endl;
   }
};
// Subject
class Subject
{
public:
```

```
    virtual void attach(Observer *observer) = 0;
    virtual void detach(Observer *observer) = 0;
    virtual void notify() = 0;
    virtual void setMeasurement(float temp, float hum, float pre) = 0;
    virtual ~Subject() {}
};
// concrete Subject
class WheatherData : public Subject
{
private:
    vector<Observer *> observers;
    float temperature;
    float humidity;
    float pressure;

public:
    void attach(Observer *obs) override
    {
        observers.push_back(obs);
    }
    void detach(Observer *obs) override
    {
        observers.erase(remove(observers.begin(), observers.end(), obs), observers.end());
    }
    void notify() override
    {
        for (Observer *observer : observers)
        {
            observer->update(temperature, humidity, pressure);
        }
    }
    void setMeasurement(float temp, float hum, float pre)
    {
        temperature = temp;
        humidity = hum;
        pressure = pre;
        notify();
    }
};
int main()
{
    Subject *subject = new WheatherData();
    Observer *Display1 = new WheatherDisplay();
    Observer *Display2 = new WheatherDisplay();
    subject->attach(Display1);
    subject->attach(Display2);
    subject->setMeasurement(12.5, 13, 22.7);
    subject->detach(Display1);
    subject->setMeasurement(15.2, 13.1, 29.7);
    delete subject;
    delete Display1;
    delete Display2;
    return 0;
}
```

State Design Pattern in C++

The State Design Pattern is a behavioral design pattern that allows an object to change its behavior when its internal state changes, without using large conditional statements.

The State Design Pattern is a way to let an object change how it behaves when its state changes. Instead of using lots of if-else or switch statements, you put the different behaviors into separate state classes. This makes your code cleaner and easier to understand.

In this pattern, you have a main class (called the context) that keeps track of its current state. When something happens, the context asks its current state what to do. If the state changes, the context just switches to a different state object. This way, the object can change its behavior at any time, just by changing its state.

For example, think about a simple vending machine. It can be in different states like Idle (waiting for money), HasMoney (money inserted), or Dispensing (giving out a product). Each state knows what to do when you press a button or insert money. By using the State Design Pattern, the vending machine can easily switch between states and always do the right thing, without complicated code.

Key Components of the State Design Pattern

The State Design Pattern is made up of a few important parts that work together to make your code easier to manage and understand. Here's what each part does:

Context − This is the main object you work with. It keeps track of what state it is in right now. When something happens, the context asks its current state what to do next. The context is also in charge of switching to a different state when needed.

State Interface − This is like a blueprint that says what actions every state should be able to do. All the different states must follow this blueprint and provide their own versions of these actions.

Concrete State Classes − These are the actual states your object can be in. Each state class has its own way of handling things. For example, the "Idle" state will do something different than the "Dispensing" state when you press a button.

State Transition: This is how the context changes from one state to another. When something happens (like inserting money), the context will switch to a new state so it can behave differently.

```
#include <iostream>
using namespace std;
// State Interface
class State
{
public:
   virtual void insertMoney() = 0;
   virtual void selectProduct() = 0;
   virtual void dispenseProduct() = 0;
   virtual ~State() {}
};

class VendingMachine;
// Idlestate
class IdleState : public State
{
private:
```

```
    VendingMachine *vendingmachine;

public:
    IdleState(VendingMachine *vm) : vendingmachine(vm) {}
    void insertMoney() override;
    void selectProduct() override
    {
        cout << "First Insert Money." << endl;
    }
    void dispenseProduct() override
    {
        cout << "First Insert Money and select product." << endl;
    }
};
// hasMOneystate
class HasMoneyState : public State
{
private:
    VendingMachine *vendingmachine;

public:
    HasMoneyState(VendingMachine *vm) : vendingmachine(vm) {}
    void insertMoney() override
    {
        cout << "Money has alread Inserted" << endl;
    }
    void selectProduct() override;
    void dispenseProduct() override
    {
        cout << "First select product." << endl;
    }
};
// dispensestate
class DispenseState : public State
{
private:
    VendingMachine *vendingmachine;

public:
    DispenseState(VendingMachine *vm) : vendingmachine(vm) {}
    void insertMoney() override
    {
        cout << "Waiting to dispense Product..." << endl;
    }
    void selectProduct() override
    {
        cout << "Waiting to dispense Product..." << endl;
    }
    void dispenseProduct() override;
};

// context // vending machine
class VendingMachine
{
```

```
private:
   State *currenstate;
   IdleState idlestate;
   HasMoneyState hasmoneystate;
   DispenseState dispensestate;

public:
   VendingMachine() : idlestate(this), hasmoneystate(this), dispensestate(this)
   {
      currenstate = &idlestate;
   }
   void setState(State *state)
   {
      currenstate = state;
   }
   void insertMoney() { currenstate->insertMoney(); }
   void selectProduct() { currenstate->selectProduct(); }
   void dispenseProduct() { currenstate->dispenseProduct(); }

   State *getIdleState() { return &idlestate; }
   State *getHasMoneyState() { return &hasmoneystate; }
   State *getDispenseState() { return &dispensestate; }
};

void IdleState::insertMoney()
{
   cout << "Inserting Money..." << endl;
   vendingmachine->setState(vendingmachine->getHasMoneyState());
}
void HasMoneyState::selectProduct()
{
   cout << "Product Selected..." << endl;
   vendingmachine->setState(vendingmachine->getDispenseState());
}
void DispenseState::dispenseProduct()
{
   cout << "Product Dispensing" << endl;
   vendingmachine->setState(vendingmachine->getIdleState());
}
int main()
{
   VendingMachine vm;
   vm.insertMoney();
   vm.dispenseProduct();
   return 0;
}
```

Real-World Examples of the State Design Pattern

The State Design Pattern is commonly used in many everyday software applications. Here are some practical examples −

User Interface Components − Buttons, sliders, and menus often behave differently based on their state (for example, enabled, disabled, or hovered). This pattern helps manage how these components react in each state.

Game Development − In video games, characters and objects have multiple states such as idle, running, jumping, or attacking. The State Design Pattern helps switch between these states and control their actions cleanly.

Network Protocols − A network connection can be in different states like connected, disconnected, or reconnecting. This pattern helps handle each state's behavior separately, making network management easier.

Workflow Management − In workflow systems, tasks may have states like pending, in progress, or completed. The pattern helps define how each state behaves and transitions to the next one.

Strategy Design Pattern in C++

The Strategy Design Pattern is a simple and powerful way to organize your code when you have different ways to perform the same task. It defines a family of algorithms, places each one in its own separate class, and lets you switch between them easily. This means you can change the algorithm's behavior without touching the main code that uses it.

For example, imagine you have a navigation app that can show routes based on different preferences: the fastest route, the shortest route, or the most scenic route. With the Strategy Design Pattern, you can create a separate class for each type of route and simply switch between them whenever needed, without changing how the main app works.

Steps to Implement the Strategy Design Pattern in C++

Define the Strategy Interface − Start by creating an abstract class that defines a common method for the algorithm. This interface acts like a blueprint that every strategy class will follow this same structure.

Implement Concrete Strategies − Create separate classes that inherit from the Strategy Interface and provide their own version of the algorithm. Each class will represent a different way of performing the same task.

Create the Context Class − This is the main class that will use one of the strategies. It holds a reference to a Strategy object and uses it to perform the calculation. The Context doesn't care which strategy it's using and it just calls the same method.

Use the Strategy Pattern − In the main function, create objects for the Context and the different Strategy classes. Show how the Context can switch between strategies dynamically for example, changing from addition to multiplication — without rewriting any logic in the Context itself.

Real-World Examples of the Strategy Design Pattern

The Strategy Design Pattern is used in many software programs to make it easy to change the way something works. Here are some simple examples of where it can be used −

In payment systems, such as online shopping apps, different payment methods like credit card, PayPal, or cryptocurrency can each be written in a separate class. The program can then choose the right payment method based on what the user wants.

In programs that need to sort data, different sorting methods like quicksort, mergesort, or bubblesort can be used as separate strategies. The program can decide which one to use depending on how big or small the data is.

In file compression software, different compression types such as ZIP, RAR, or TAR can each be written separately. The user can choose which one to use without changing the main program.

In video games, different character behaviors like attacking, defending, or staying still can each be made as their own strategy. The game can switch between them based on what is happening during the game.

```
#include <iostream>
#include <memory>
using namespace std;
class Strategy
{
public:
    virtual int doOperation(int, int) = 0;
    virtual ~Strategy() {}
};
class Addition : public Strategy
{
public:
    int doOperation(int a, int b) override
    {
        return a + b;
    }
};
class Subtraction : public Strategy
{
public:
    int doOperation(int a, int b) override
    {
        return a - b;
    }
};
class Context
{
private:
    unique_ptr<Strategy> strategy;

public:
    Context(unique_ptr<Strategy> stra) : strategy(move(stra)) {}
    void setStrategy(unique_ptr<Strategy> strategy)
    {
        this->strategy = move(strategy);
    }
    void executeStrategy(int a, int b)
    {
        cout << strategy->doOperation(a, b) << endl;
    }
};
int main()
{
```

```
    unique_ptr<Context> context = make_unique<Context>(make_unique<Addition>());
    context->executeStrategy(3, 5);
    context->setStrategy(make_unique<Subtraction>());
    context->executeStrategy(7, 3);
    return 0;
}
```

Template Method Design Pattern

Template Method Pattern introduces a template in a superclass that defines the steps of an algorithm. These steps may include both common tasks shared among subclasses and specific tasks that need customization. Subclasses then implement or override these steps to modify the algorithm according to their specific needs.

Key Components of Template Method Design Pattern in C++

Template Method: The template method is a method in an abstract base class (or interface) that defines the overall algorithm structure. It consists of a series of steps, some of which are implemented in the base class and others that are left abstract or virtual for concrete subclasses to implement.

Abstract Methods (or Hooks): These are abstract or virtual methods declared in the abstract base class (or interface) that represent steps of the algorithm to be implemented by concrete subclasses.

Concrete Classes: Concrete classes are derived from the abstract base class and provide concrete implementations for the abstract methods.

Template or Base Class: The template or base class is an abstract class (or interface) that declares the template method and may include some default implementations of steps.

Client: The client is the code that uses the template method pattern. It creates instances of concrete classes and invokes the template method to execute the algorithm

```
#include <iostream>
using namespace std;
// tempelate interface
class Meal
{
public:
//tempelate method
    void prepareMeal()
    {
        gatheringIngrediants();
        cookMeal();
        serveMeal();
    }
    virtual void gatheringIngrediants() = 0;
    virtual void cookMeal() = 0;
    void serveMeal()
    {
        cout << "Serving meal..." << endl;
    }
};
class Breakfast : public Meal
```

```
{
public:
   void gatheringIngrediants()
   {
      cout << "gatheringIngrediants eggs, bread and milk" << endl;
   }
   void cookMeal()
   {
      cout << "Frying eggs.boilding milk and making breakfast." << endl;
   }
};
class Lunch : public Meal
{
public:
   void gatheringIngrediants()
   {
      cout << "gatheringIngrediants vegitables and chicken" << endl;
   }
   void cookMeal()
   {
      cout << "steaming chicken , cutting vegitables and making lunch." << endl;
   }
};
int main()
{
   Meal *breakfast = new Breakfast();
   Meal *lunch = new Lunch();
   breakfast->prepareMeal();
   lunch->prepareMeal();
   return 0;
}
```

Visitor Design Pattern in C++

The visitor design pattern is a way of writing code that helps you in adding new features or new types of work to your classes without changing those classes again and again.

Imagine a school building. The school admin cannot repair wires, fix water pipes, clean the classrooms, or check the furniture. Instead, different people come to the school to do these jobs. An electrician comes to fix wires. A plumber comes to fix water pipes. A cleaner comes to clean the rooms. A carpenter comes to fix furniture. The school admin allows different workers to come inside and do their job.

This is the exact same idea in the visitor pattern. Your object is like the school building, and the visitor is like the worker who comes to do the job. The building has a door, and the objects have an accept method. The visitor enters through the accept method and then does the required work.

Components of the Visitor Pattern

Following are the main components of the visitor design pattern −

Visitor Interface − This is like a plan that tells which types of objects the visitor can work with. It has one visit function for each element type. For example, visitCircle(), visitSquare(), visitRectangle().

Concrete Visitor − These are actual classes that do real work. For example, an AreaCalculator visitor will calculate area. A PerimeterCalculator visitor will calculate perimeter. A DrawingVisitor might draw the shapes on screen.

Element Interface − This is the main interface that all elements must follow. It usually contains only one function: accept(visitor). Every concrete element must implement this.

Concrete Element − These are your real objects. For example, Circle, Square, and Rectangle. They store simple data like radius or side length and implement the accept method.

Object Structure − This is usually a list or collection of elements. The visitor will go through this structure and visit each element one by one.

Client − The client sets everything up. It creates shapes, creates visitors, and asks the elements to accept the visitors.

```
#include <iostream>
#include <vector>
using namespace std;
class Circle;
class Square;
class Rectangle;
class Visitor
{
public:
   virtual void visit(Circle *c) = 0;
   virtual void visit(Square *s) = 0;
   virtual void visit(Rectangle *r) = 0;
   virtual ~Visitor() {}
};
class Shape
{
public:
   virtual void accept(Visitor *v) = 0;
   virtual ~Shape() {}
};
class Circle : public Shape
{
private:
   double radius;

public:
   Circle(double r) : radius(r) {}
   double getRadius() { return radius; }
   void accept(Visitor *v) override
   {
      v->visit(this);
   }
};
class Square : public Shape
{
private:
   double side;
```

```
public:
    Square(double s) : side(s) {}
    double getSide() { return side; }
    void accept(Visitor *v) override
    {
        v->visit(this);
    }
};
class Rectangle : public Shape
{
private:
    double width, height;

public:
    Rectangle(double w, double h) : width(w), height(h) {}
    double getWidth() { return width; }
    double getHeight() { return height; }
    void accept(Visitor *v) override
    {
        v->visit(this);
    }
};
class AreaCalculator : public Visitor
{
public:
    void visit(Circle *c)
    {
        double area = 3.14 * c->getRadius() * c->getRadius();
        cout << "Area Circle : " << area << endl;
    }
    void visit(Square *s)
    {
        double area = s->getSide() * s->getSide();
        cout << "Area Square : " << area << endl;
    }
    void visit(Rectangle *r)
    {
        double area = r->getWidth() * r->getHeight();
        cout << "Area Rectangle : " << area << endl;
    }
};
class PerimeterCalculator : public Visitor
{
public:
    void visit(Circle *c)
    {
        double perimeter = 3.14 * 2 * c->getRadius();
        cout << "perimeter Circle : " << perimeter << endl;
    }
    void visit(Square *s)
    {
        double perimeter = 4 * s->getSide();
        cout << "perimeter Square : " << perimeter << endl;
    }
```

```
    void visit(Rectangle *r)
    {
        double perimeter = 2 * (r->getWidth() + r->getHeight());
        cout << "perimeter Rectangle : " << perimeter << endl;
    }
};

int main()
{
    vector<Shape *> shape;
    shape.push_back(new Circle(3));
    shape.push_back(new Square(5));
    shape.push_back(new Rectangle(2, 3));

    PerimeterCalculator perimetercalc;
    AreaCalculator areacalc;
    for (Shape *s : shape)
    {
        s->accept(&perimetercalc);
        s->accept(&areacalc);
    }
    for (Shape *s : shape)
    {
        delete s;
    }
    return 0;
}
```